

C/C++ & 嵌入式开发

- C 语言与数据结构
- Linux 系统编程
- C++ 开发
- 经典算法
- 嵌入式开发

C 语言与数据结构

一、C语言基础（1-10题）

1. C语言中int、short、long的区别？

- **解析：**三者均为整数类型，区别在于**占用内存大小**和**表示范围**（与平台相关，通常32位系统下）：
 - **short**：2字节，范围-32768 ~ 32767；
 - **int**：4字节，范围-2³¹ ~ 2³¹-1；
 - **long**：4字节（32位）或8字节（64位），范围不小于**int**。
- **注意：**C99标准规定**long long**至少8字节，可表示更大范围整数。

2. const关键字的作用？

- **解析：**用于定义**常量**，限制变量被修改，常见用法：
 - **const int a = 10;**：a的值不可修改；
 - **const int* p;** 指针指向的内容不可修改（*p不可改）；
 - **int* const p;** 指针本身不可修改（p的指向不可改）；
 - **const int* const p;** 指针及其指向的内容均不可修改。
- **作用：**增强代码可读性，防止意外修改，编译器可优化。

3. static关键字在C语言中的用法？

- **解析：****static**可修饰变量和函数，作用如下：
 - **修饰局部变量：**延长生命周期至程序结束（仅初始化一次），作用域仍为函数内；
 - **修饰全局变量：**限制作用域为当前文件，避免其他文件访问；
 - **修饰函数：**限制函数作用域为当前文件，避免外部调用冲突。

4. sizeof和strlen的区别？

特性	sizeof	strlen
本质	运算符（编译期计算）	库函数（运行期计算）
功能	计算变量/类型占用内存大小（字节）	计算字符串实际长度（不含\0）
参数	变量、类型、数组等	以\0结尾的字符串指针
示例	sizeof("abc")=4（含\0）	strlen("abc")=3

5. C语言中函数参数传递方式？

- **解析：**C语言仅支持**值传递**，具体表现：
 - 基本类型参数：函数内修改形参，不影响实参；
 - 指针参数：通过指针间接修改实参指向的内容（本质仍为值传递，传递的是指针地址）。

6. 什么是预编译？#define和typedef的区别？

- **预编译：**编译前的处理，包括宏替换、文件包含（#include）、条件编译（#ifdef）等。
- **区别：**
 - **#define：**宏定义，仅做文本替换（无类型检查），可定义常量、函数片段；
 - **typedef：**类型别名，为已有类型定义新名称（有类型检查），不能定义函数或常量。
 - 示例：**#define INT int*** 定义宏，**typedef int* INT** 定义指针类型别名，使用时前者可能导致歧义。

7. C语言中数组名和指针的关系？

- **联系：**数组名可视为指向数组首元素的**常量指针**（不可修改指向），如arr[i]等价于*(arr+i)。
- **区别：**
 - 数组名作为sizeof参数时，返回数组总字节数；指针作为sizeof参数时，返回指针本身大小（如8字节）；
 - 数组名不能被赋值（常量指针），指针可被赋值修改指向。

8. 什么是内存对齐？为什么需要内存对齐？

- **内存对齐：**数据在内存中的地址需满足一定规则（如int型地址为4的倍数）。
- **原因：**
 - 硬件限制：CPU访问内存时按块读取（如4字节），未对齐数据需多次访问，降低效率；
 - 平台兼容性：部分硬件不支持非对齐访问，可能导致程序崩溃。
- **控制：**通过**#pragma pack(n)**指定对齐字节数（n为2的幂）。

9. C语言中的结构体和联合体（union）的区别？

特性	结构体（struct）	联合体（union）
内存分配	各成员独立分配内存，总大小为成员大小之和（含对齐）	所有成员共享同一块内存，大小为最大成员的大小
访问	可同时访问所有成员	同一时间只能有效访问一个成员
用途	表示复杂对象（如学生信息）	节省内存（如同一空间存储不同类型数据）

10. volatile关键字的作用？

- **解析：**告知编译器变量的值可能被**意外修改**（如硬件中断、多线程），禁止编译器对该变量的优化（如缓存到寄存器），每次访问需从内存重新读取。
- **应用场景：**多线程共享变量、硬件寄存器地址映射变量。

二、指针与内存管理（11-20题）

11. 什么是野指针？如何避免？

- **野指针**：指向无效内存区域的指针（如未初始化、已释放的内存）。
- **危害**：访问野指针可能导致程序崩溃、数据损坏。
- **避免方法**：
 - 指针定义时初始化（如 `int* p = NULL;`）；
 - 释放内存后将指针置为 `NULL`；
 - 避免返回局部变量的地址（局部变量在函数结束后释放）。

12. 指针数组和数组指针的区别？

- **指针数组**：数组元素为指针，定义为 `int* arr[10];`（`arr`是数组，每个元素是 `int*`）；
- **数组指针**：指向数组的指针，定义为 `int (*p)[10];`（`p`是指针，指向含10个 `int`的数组）。
- 记忆：优先级 `[]` 高于 `*`，加括号改变优先级。

13. 函数指针的用途？举例说明。

- **函数指针**：指向函数的指针，格式 `返回类型 (*指针名)(参数列表)`。
- **用途**：实现回调函数（如排序时自定义比较规则）、函数表（根据条件调用不同函数）。
- 示例：

```
int add(int a, int b) { return a + b; }
int (*func_ptr)(int, int) = add; // 函数指针指向add
int result = func_ptr(2, 3); // 调用, result=5
```

14. 什么是内存泄漏？如何检测和避免？

- **内存泄漏**：动态分配的内存（`malloc/calloc`）未被释放，导致内存耗尽。
- **检测工具**：Linux下 `valgrind`，Windows下 Visual Studio 调试器。
- **避免方法**：
 - 分配内存后立即规划释放逻辑；
 - 使用 `goto` 或封装函数确保释放（如错误处理时）；
 - 避免多层嵌套分配内存。

15. `malloc`、`calloc`、`realloc` 的区别？

- `malloc(size)`：分配 `size` 字节内存，**未初始化**，返回指针；
- `calloc(n, size)`：分配 `n*size` 字节内存，**初始化为0**，返回指针；
- `realloc(ptr, new_size)`：调整 `ptr` 指向的内存大小为 `new_size`，可能移动内存，返回新地址（失败时返回 `NULL`，原内存不变）。

16. 如何用指针实现数组逆序？

- **思路**：用两个指针分别指向数组首尾，交换元素后向中间移动，直到相遇。
- 代码：

```
void reverse(int* arr, int len) {
    if (arr == NULL || len <= 1) return;
    int* left = arr;
    int* right = arr + len - 1;
    while (left < right) {
        int temp = *left;
        *left = *right;
        *right = temp;
        left++;
        right--;
    }
}
```

17. 什么是柔性数组？用途是什么？

- **柔性数组**：结构体最后一个成员为未指定大小的数组，如：

```
struct Data {
    int len;
    char data[]; // 柔性数组
};
```

- **用途**：动态分配结构体+数组的连续内存，节省空间且访问高效（避免二次指针）。
- **分配方式**：`struct Data* d = malloc(sizeof(struct Data) + 100);`（分配100字节给data）。

18. 指针的指针（二级指针）的应用场景？

- **解析**：用于修改指针本身（如函数中为指针分配内存、链表头节点修改）。
- **示例**：通过函数为指针分配内存：

```
void alloc_mem(int**p, int size) {
    *p = malloc(size * sizeof(int)); // 二级指针解引用得到一级指针
}

int main() {
    int* arr = NULL;
    alloc_mem(&arr, 10); // 传递指针地址
    return 0;
}
```

19. 栈内存和堆内存的区别？

特性	栈内存	堆内存
分配方式	编译器自动分配/释放（函数调用时）	手动malloc分配/free释放
大小限制	较小（如8MB）	较大（受系统内存限制）

特性	栈内存	堆内存
生长方向	向下生长（高地址→低地址）	向上生长（低地址→高地址）
效率	高（直接操作寄存器）	低（需查找空闲内存块）
碎片	无	频繁分配释放易产生碎片

20. 如何判断两个指针指向的内存是否重叠？

- 思路：假设指针p1指向大小s1的内存，p2指向大小s2的内存，重叠条件为：p1 < p2 + s2 && p2 < p1 + s1。
- 代码：

```
int is_overlap(const void* p1, size_t s1, const void* p2, size_t s2) {
    return (p1 < (char*)p2 + s2) && (p2 < (char*)p1 + s1);
}
```

三、数据结构：线性表（21-30题）

21. 链表和数组的区别？

特性	数组	链表
存储方式	连续内存	非连续（节点含指针）
访问效率	随机访问（O(1)）	顺序访问（O(n)）
插入/删除	需移动元素（O(n)）	仅修改指针（O(1)，已知前驱）
内存管理	大小固定（静态）	动态分配（大小可变）
空间开销	无额外开销	需存储指针（额外空间）

22. 如何实现单链表的反转？

- 思路：遍历链表，将每个节点的next指向其前驱节点。
- 代码（迭代法）：

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* reverse_list(Node* head) {
    Node *prev = NULL, *curr = head, *next = NULL;
    while (curr != NULL) {
        next = curr->next; // 保存下一个节点
        curr->next = prev; // 反转指针
        prev = curr;      // 移动前驱
    }
    return prev;
}
```

```

        curr = next;        // 移动当前
    }
    return prev; // 新头节点
}

```

23. 如何检测单链表中是否有环?

- **解法：快慢指针法**（Floyd判圈算法）：
 - 快指针每次走2步，慢指针每次走1步；
 - 若有环，两指针必在环内相遇；若无环，快指针先到达NULL。
- 代码：

```

int has_cycle(Node* head) {
    if (head == NULL || head->next == NULL) return 0;
    Node *slow = head, *fast = head->next;
    while (slow != fast) {
        if (fast == NULL || fast->next == NULL) return 0;
        slow = slow->next;
        fast = fast->next->next;
    }
    return 1;
}

```

24. 如何找到单链表中环的入口节点?

- **思路：**
 1. 用快慢指针确定有环后，慢指针回到头节点，快指针留在相遇点；
 2. 两指针均每次走1步，相遇点即为环入口。
- **证明：**设头到入口距离为 a ，入口到相遇点距离为 b ，环长为 c ，则快指针路程 $a + b + kc$ ，慢指针路程 $a + b$ ，因快=2×慢，得 $a = kc - b$ ，故头到入口与相遇点到入口距离相等。

25. 如何合并两个有序单链表为一个有序链表?

- **思路：**类似归并排序的合并步骤，用哨兵节点简化边界处理。
- 代码：

```

Node* merge_two_lists(Node* l1, Node* l2) {
    Node* dummy = (Node*)malloc(sizeof(Node)); // 哨兵节点
    Node* curr = dummy;
    while (l1 && l2) {
        if (l1->data < l2->data) {
            curr->next = l1;
            l1 = l1->next;
        } else {
            curr->next = l2;
            l2 = l2->next;
        }
    }
}

```

```

        curr = curr->next;
    }
    curr->next = l1 ? l1 : l2; // 拼接剩余节点
    return dummy->next;
}

```

26. 栈和队列的区别？如何用栈实现队列？

- **区别：**栈是“先进后出”（LIFO），队列是“先进先出”（FIFO）。
- **栈实现队列：**用两个栈`in`和`out`：
 - 入队：压入`in`栈；
 - 出队：若`out`栈为空，将`in`栈所有元素弹出并压入`out`栈，再弹出`out`栈顶元素。
- 代码核心逻辑：

```

typedef struct {
    Stack* in;
    Stack* out;
} MyQueue;

void push(MyQueue* q, int x) { stack_push(q->in, x); }
int pop(MyQueue* q) {
    if (stack_empty(q->out)) {
        while (!stack_empty(q->in)) {
            stack_push(q->out, stack_pop(q->in));
        }
    }
    return stack_pop(q->out);
}

```

27. 什么是双向链表？相比单链表有什么优势？

- **双向链表：**每个节点含`prev`（前驱指针）和`next`（后继指针），可双向遍历。
- **优势：**
 - 可从任意节点向前遍历（单链表仅能向后）；
 - 删除节点时无需遍历查找前驱（单链表需 $O(n)$ 找前驱）。

28. 如何找到单链表的倒数第k个节点？

- **思路：双指针法：**
 1. 快指针先向前走`k`步；
 2. 快慢指针同时向前走，快指针到达尾部时，慢指针即为目标节点。
- 代码：

```

Node* find_kth_from_end(Node* head, int k) {
    if (head == NULL || k <= 0) return NULL;
    Node *fast = head, *slow = head;
    for (int i = 0; i < k; i++) {

```

```

        if (fast == NULL) return NULL; // k大于链表长度
        fast = fast->next;
    }
    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}

```

29. 数组中出现次数超过一半的数字（多数元素）如何查找？

- **解法：摩尔投票法**（时间 $O(n)$ ，空间 $O(1)$ ）：
 - 遍历数组，维护一个候选值和计数，相同则计数+1，不同则-1；
 - 计数为0时更新候选值为当前元素；
 - 最终候选值即为多数元素（需验证是否超过一半）。
- 代码：

```

int majority_element(int* nums, int n) {
    int candidate = nums[0], count = 1;
    for (int i = 1; i < n; i++) {
        count = (nums[i] == candidate) ? count + 1 : count - 1;
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        }
    }
    // 验证
    count = 0;
    for (int i = 0; i < n; i++) if (nums[i] == candidate) count++;
    return count > n/2 ? candidate : -1;
}

```

30. 如何判断两个链表是否相交？若相交，找到交点。

- **思路：**
 1. 计算两链表长度 $len1$ 和 $len2$ ；
 2. 长链表指针先移动 $|len1-len2|$ 步，使两指针到尾部距离相等；
 3. 两指针同时移动，相遇点即为交点（若均为NULL则不相交）。

四、数据结构：树与图（31-40题）

31. 二叉树的前序、中序、后序遍历的区别？

- **定义：**以根节点（N）、左子树（L）、右子树（R）为参照：
 - 前序遍历： $N \rightarrow L \rightarrow R$ （先访问根）；
 - 中序遍历： $L \rightarrow N \rightarrow R$ （根在中间）；
 - 后序遍历： $L \rightarrow R \rightarrow N$ （最后访问根）。

- **示例：**对根为1，左子树2-3，右子树4-5的二叉树：
 - 前序：1 2 3 4 5；中序：2 3 1 4 5；后序：3 2 5 4 1。

32. 如何根据前序和中序遍历重建二叉树？

- **思路：**
 1. 前序遍历首元素为根节点；
 2. 在中序遍历中找到根节点位置，左侧为左子树，右侧为右子树；
 3. 递归重建左、右子树。
- **代码核心逻辑：**

```
typedef struct TreeNode {
    int val;
    struct TreeNode *left, *right;
} TreeNode;

TreeNode* build(int* pre, int pre_len, int* in, int in_len) {
    if (pre_len == 0) return NULL;
    int root_val = pre[0], root_idx = 0;
    while (root_idx < in_len && in[root_idx] != root_val) root_idx++;
    TreeNode* root = (TreeNode*)malloc(sizeof(TreeNode));
    root->val = root_val;
    root->left = build(pre + 1, root_idx, in, root_idx);
    root->right = build(pre + 1 + root_idx, pre_len - 1 - root_idx,
                       in + root_idx + 1, in_len - 1 - root_idx);
    return root;
}
```

33. 什么是二叉搜索树 (BST)？其特性是什么？

- **定义：**空树或满足以下条件的二叉树：
 - 左子树所有节点值 < 根节点值；
 - 右子树所有节点值 > 根节点值；
 - 左右子树均为BST。
- **特性：**中序遍历结果为**严格递增序列**，支持高效查找 ($O(h)$ ， h 为树高)。

34. 如何判断一棵二叉树是否为平衡二叉树？

- **平衡二叉树：**任意节点的左右子树高度差不超过1。
- **解法：**后序遍历，计算每个节点的左右子树高度，同时判断平衡性。
- **代码：**

```
int get_height(TreeNode* node) {
    if (node == NULL) return 0;
    int left_h = get_height(node->left);
    if (left_h == -1) return -1; // 左子树不平衡
    int right_h = get_height(node->right);
    if (right_h == -1) return -1; // 右子树不平衡
```

```
        if (abs(left_h - right_h) > 1) return -1; // 当前节点不平衡
        return 1 + max(left_h, right_h);
    }
    int is_balanced(TreeNode* root) {
        return get_height(root) != -1;
    }
}
```

35. 二叉树的层序遍历（广度优先遍历）如何实现？

- **思路：**用队列存储节点，出队时访问节点，同时入队其左右子节点。
- **代码：**

```
void level_order(TreeNode* root) {
    if (root == NULL) return;
    Queue* q = queue_create();
    queue_push(q, root);
    while (!queue_empty(q)) {
        TreeNode* node = queue_pop(q);
        printf("%d ", node->val); // 访问节点
        if (node->left) queue_push(q, node->left);
        if (node->right) queue_push(q, node->right);
    }
    queue_destroy(q);
}
```

36. 什么是红黑树？相比BST有什么优势？

- **红黑树：**自平衡BST，通过颜色规则（红/黑）和旋转操作维持平衡：
 - 根节点为黑；
 - 红节点的子节点必为黑；
 - 任意节点到其所有叶子的路径含相同数量黑节点。
- **优势：**最坏情况下查找/插入/删除时间复杂度为O(log n)（BST最坏为O(n)，如退化为链表）。

37. 图的深度优先搜索（DFS）和广度优先搜索（BFS）的区别？

特性	DFS	BFS
实现方式	栈（递归或显式栈）	队列
遍历顺序	沿路径深入，回溯探索其他路径	按层次逐层扩散
空间复杂度	O(h) (h为图深度)	O(w) (w为最大层宽度)
应用场景	拓扑排序、路径搜索	最短路径（无权图）、层序遍历

38. 如何实现图的邻接表表示？

- **邻接表：**用数组存储顶点，每个顶点对应一个链表存储相邻顶点。
- **代码：**

```
typedef struct EdgeNode {
    int adjvex; // 邻接顶点索引
    struct EdgeNode* next;
} EdgeNode;

typedef struct VertexNode {
    int data; // 顶点数据
    EdgeNode* firstedge; // 指向第一条边
} VertexNode;

typedef struct {
    VertexNode vertices[100]; // 顶点数组
    int num_vertices, num_edges; // 顶点数、边数
} Graph;
```

39. 什么是拓扑排序？适用于什么场景？

- **拓扑排序**：对有向无环图（DAG）的顶点进行排序，使所有有向边从前面的顶点指向后面的顶点。
- **实现**：基于入度表和队列，反复删除入度为0的顶点并更新邻接顶点入度。
- **场景**：任务调度（依赖关系）、课程安排（先修课）。

40. 如何判断有向图中是否存在环？

- **解法**：
 1. DFS：遍历中检测是否遇到**回边**（指向已访问但未回溯的顶点）；
 2. 拓扑排序：若无法生成包含所有顶点的拓扑序列，则存在环。

五、算法：排序与查找（41-50题）

41. 常见排序算法的时间复杂度和稳定性？

排序算法	平均时间复杂度	最坏时间复杂度	最好时间复杂度	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定

42. 实现快速排序算法。

- **思路**：选择基准元素，分区（小于基准放左，大于放右），递归排序左右分区。
- **代码**：

```

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
int partition(int* arr, int low, int high) {
    int pivot = arr[high]; // 选最后一个元素为基准
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
    }
    swap(&arr[i+1], &arr[high]); // 基准归位
    return i + 1;
}
void quick_sort(int* arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}

```

43. 实现归并排序算法。

- **思路：**分治思想，将数组分为两半，分别排序后合并。
- **代码：**

```

void merge(int* arr, int left, int mid, int right) {
    int n1 = mid - left + 1, n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
void merge_sort(int* arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

44. 二分查找的原理？如何实现？

- **原理：**对**有序数组**，每次与中间元素比较，缩小查找范围（左/右半区）。
- **代码（非递归）：**

```
int binary_search(int* arr, int n, int target) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // 避免溢出
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1; // 未找到
}
```

45. 如何在旋转排序数组中查找目标值（如[4,5,6,7,0,1,2]）？

- **思路：**二分查找变种，先判断左右半区是否有序，再确定目标所在区域。
- **代码：**

```
int search_rotated(int* nums, int n, int target) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) { // 左半区有序
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // 右半区有序
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

46. 什么是堆？大顶堆和小顶堆的区别？

- **堆：**完全二叉树，父节点与子节点满足特定关系。
- **大顶堆：**父节点值 $\geq$ 子节点值（根为最大值）；
- **小顶堆：**父节点值 $\leq$ 子节点值（根为最小值）。
- **应用：**堆排序、优先队列。

47. 如何用堆排序对数组排序？

- **步骤：**
 1. 构建大顶堆（从最后一个非叶子节点向上调整）；
 2. 交换根（最大值）与最后一个元素，缩小堆大小并调整堆；
 3. 重复步骤2直至排序完成。

48. 什么是哈希表？哈希冲突如何解决？

- **哈希表**：通过哈希函数将键映射到数组索引，实现 $O(1)$ 平均查找。
- **哈希冲突**：不同键映射到同一索引。
- **解决方法**：
 - 开放定址法：冲突时探测下一个空闲位置（如线性探测、二次探测）；
 - 链地址法：每个索引对应链表，冲突元素存入链表；
 - 再哈希法：用多个哈希函数，冲突时换函数。

49. 如何找到数组中第k大的元素？

- **解法**：
 1. 快速选择算法（快排变种）：平均 $O(n)$ ，最坏 $O(n^2)$ ；
 2. 堆排序：用小顶堆维护前k大元素， $O(n \log k)$ 。
- 快速选择代码核心：

```
int find_kth_largest(int* nums, int n, int k) {
    k = n - k; // 转化为找第k小 (0-based)
    int left = 0, right = n - 1;
    while (left < right) {
        int p = partition(nums, left, right);
        if (p < k) left = p + 1;
        else if (p > k) right = p - 1;
        else break;
    }
    return nums[k];
}
```

50. 如何求两个有序数组的中位数？

- **思路**：转化为求第k小元素 ($k=(m+n)/2$)，通过二分法排除不可能的区间。
- 示例：若 $nums1[i] < nums2[j]$ ，则 $nums1[0..i]$ 不可能是第k小，排除。

六、算法：动态规划与贪心（51-56题）

51. 什么是动态规划（DP）？核心思想是什么？

- **定义**：通过将复杂问题分解为重叠子问题，用表格存储子问题解（避免重复计算），解决最优化问题。
- **核心思想**：
 - 状态定义：确定子问题的表示（如 $dp[i]$ 的含义）；
 - 状态转移方程：子问题间的关系（如 $dp[i] = \max(dp[i-1], \dots)$ ）；
 - 初始条件：最小子问题的解。

52. 实现斐波那契数列的DP解法。

- **问题**： $f(n) = f(n-1) + f(n-2)$, $f(0)=0$, $f(1)=1$ 。
- **DP解法**（时间 $O(n)$ ，空间 $O(1)$ ）：

```
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

53. 最长递增子序列 (LIS) 问题如何解决?

- **DP解法:** $dp[i]$ 表示以 $nums[i]$ 结尾的LIS长度, $dp[i] = \max(dp[j]+1) (j < i \text{ 且 } nums[j] < nums[i])$, 时间 $O(n^2)$ 。
- 代码:

```
int length_of_lis(int* nums, int n) {
    if (n == 0) return 0;
    int dp[n], max_len = 1;
    for (int i = 0; i < n; i++) dp[i] = 1;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
            }
        }
        max_len = max(max_len, dp[i]);
    }
    return max_len;
}
```

54. 什么是贪心算法? 与DP的区别?

- **贪心算法:** 每步选择局部最优解, 希望得到全局最优 (适用于具有贪心选择性质的问题)。
- **区别:**
 - 贪心: 无后效性, 仅依赖当前状态, 无法回溯;
 - DP: 存储子问题解, 考虑所有可能选择, 可回溯。
- **应用:** 霍夫曼编码、活动选择问题、零钱兑换 (特定面值)。

55. 零钱兑换问题: 用最少硬币凑成金额 (硬币面值无限)。

- **DP解法:** $dp[i]$ 表示凑成金额 i 的最少硬币数, $dp[i] = \min(dp[i - coin] + 1)$ ($coin$ 为所有面值)。
- 代码:

```

int coin_change(int* coins, int coins_size, int amount) {
    int dp[amount + 1];
    for (int i = 0; i <= amount; i++) dp[i] = amount + 1;
    dp[0] = 0; // 金额0需0枚硬币
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < coins_size; j++) {
            if (coins[j] <= i) {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}

```

56. 0-1背包问题的DP解法？

- **问题：**n件物品，每件重量 $w[i]$ 、价值 $v[i]$ ，背包容量C，求最大价值（每件物品仅选一次）。
- **DP解法：** $dp[i][j]$ 表示前i件物品容量j的最大价值，转移方程：
 - $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$ （选或不选第i件）。
- **空间优化：**用一维数组 $dp[j]$ ，逆序更新（避免覆盖子问题解）。

Linux 系统编程

一、Linux文件操作（1-15题）

1. 什么是文件描述符（File Descriptor）？Linux中默认打开的文件描述符有哪些？

- **解析：**文件描述符（FD）是内核为进程打开的文件分配的非负整数标识，用于后续IO操作。
- **默认打开的3个FD：**
 - 0：标准输入（stdin）；
 - 1：标准输出（stdout）；
 - 2：标准错误（stderr）。
- **特性：**FD是进程级别的，不同进程可拥有相同FD但指向不同文件。

2. open()函数的常用参数和返回值？O_CREAT和O_EXCL组合的作用？

- **函数原型：**`int open(const char *pathname, int flags, mode_t mode);`
- **核心参数：**
 - **flags：**打开方式（如O_RDONLY只读、O_WRONLY只写、O_RDWR读写、O_CREAT文件不存在则创建）；
 - **mode：**当O_CREAT生效时，指定文件权限（如0644表示-rw-r--r--）。
- **返回值：**成功返回新FD，失败返回-1并设置errno。
- **O_CREAT | O_EXCL：**若文件已存在则open()失败（避免覆盖已有文件），常用于创建锁文件。

3. read()和write()函数的返回值含义？为什么可能返回小于请求字节数？

- **read(fd, buf, count):**

- 返回值：成功返回读取的字节数（0表示读到文件尾），失败返回-1。
- `write(fd, buf, count)`:
 - 返回值：成功返回写入的字节数，失败返回-1。
- 返回小于请求数的原因：
 - 读操作：管道/套接字缓冲区数据不足、被信号中断（需结合EINTR处理）；
 - 写操作：磁盘空间不足、套接字发送缓冲区满、被信号中断。

4. 什么是原子操作？`pread()`和`pwrite()`与`lseek()+read/write`的区别？

- 原子操作：不可分割的操作，要么完全执行，要么不执行（避免多线程/进程竞争）。
- 区别：
 - `lseek(fd, offset, SEEK_SET) + read(fd, buf, n)`：两步操作，可能被其他进程的`lseek`或IO操作打断，导致读写位置错误；
 - `pread(fd, buf, n, offset)/pwrite()`：原子操作，直接在指定`offset`读写，不影响当前文件指针，适合多线程共享FD场景。

5. Linux文件系统的inode包含哪些信息？与文件名的关系？

- inode（索引节点） 存储文件元数据：
 - 文件类型（普通文件、目录、设备等）、权限、所有者、大小、修改时间、数据块指针等。
- 与文件名的关系：
 - 文件名存于目录文件中，目录是“文件名→inode号”的映射表；
 - 一个inode可对应多个文件名（硬链接）， 但一个文件名仅对应一个inode。

6. 硬链接和软链接（符号链接）的区别？

特性	硬链接 (hard link)	软链接 (symbolic link)
本质	inode号相同的文件名	指向源文件路径的特殊文件
跨文件系统	不支持 (inode仅在同一文件系统有效)	支持
源文件删除	仍可访问 (inode引用计数>0)	失效 (指向不存在路径)
对目录	不支持 (避免循环引用)	支持
创建命令	<code>ln 源文件 链接文件</code>	<code>ln -s 源文件 链接文件</code>

7. `fcntl()`函数的主要用途？如何用其设置非阻塞IO？

- `fcntl()`用途：操作文件描述符的属性，如：
 - 修改文件状态标志（`F_SETFL`）；
 - 获取/设置文件描述符标志（`F_GETFD/F_SETFD`）；
 - 文件锁（`F_SETLK/F_SETLKW`）。
- 设置非阻塞IO：

```
int flags = fcntl(fd, F_GETFL, 0);           // 获取当前标志
fcntl(fd, F_SETFL, flags | O_NONBLOCK);     // 追加非阻塞标志
```

8. 什么是IO多路复用？常见的IO多路复用函数有哪些？

- **IO多路复用**：允许一个进程同时监控多个FD，当某个FD就绪（可读/可写）时再进行IO操作，避免阻塞在单个IO上。
- **常见函数**：`select()`、`poll()`、`epoll()`（Linux特有）。

9. `select()`函数的工作原理？存在哪些局限性？

- **原理**：
 - 传入读、写、异常FD集合（`fd_set`）和超时时间；
 - 内核阻塞等待，直到有FD就绪或超时，返回就绪FD数量；
 - 需轮询所有FD判断是否就绪。
- **局限性**：
 - FD数量限制（默认`FD_SETSIZE=1024`）；
 - 每次调用需复制FD集合到内核，开销大；
 - 就绪FD需轮询检测，效率低。

10. `poll()`与`select()`的区别？

- **数据结构**：`poll`用`struct pollfd`数组（每个元素含FD和事件掩码），`select`用三个独立`fd_set`；
- **FD数量**：`poll`无内置限制（仅受系统资源限制），`select`受`FD_SETSIZE`限制；
- **重用性**：`pollfd`数组可重用（事件掩码需手动重置），`select`的`fd_set`每次需重新初始化；
- **效率**：两者均需轮询检测就绪FD，性能相近（大规模FD时效率低）。

11. 什么是文件锁？`fcntl()`如何实现记录锁（字节范围锁）？

- **文件锁**：防止多进程同时修改文件导致的数据不一致，分为共享锁（读锁）和排他锁（写锁）。
- **`fcntl()`实现记录锁**：

```
struct flock fl = {
    .l_type = F_RDLCK,      // 共享锁 (F_WRLCK为排他锁, F_UNLCK为解锁)
    .l_whence = SEEK_SET,   // 偏移起始位置
    .l_start = 0,           // 锁起始偏移
    .l_len = 0,             // 锁长度 (0表示到文件尾)
    .l_pid = -1             // 持有锁的进程ID (获取时用)
};
fcntl(fd, F_SETLK, &fl); // 非阻塞获取锁, 失败返回-1且errno=EAGAIN
// F_SETLKW为阻塞版本 (W=wait)
```

12. `mmap()`函数的作用？如何用其实现文件共享？

- **`mmap()`**：将文件或设备内存映射到进程地址空间，实现“内存IO”（避免`read/write`拷贝）。
- **函数原型**：`void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);`
- **文件共享**：多进程`mmap`同一文件（`flags`含`MAP_SHARED`），修改映射区会同步到文件和其他进程的映射区（内核维护一致性）。

13. dup()和dup2()的作用？如何用其重定向标准输出？

- **作用：**复制文件描述符，指向同一打开文件描述（内核中的文件对象）。
 - `dup(oldfd)`：返回最小可用的新FD；
 - `dup2(oldfd, newfd)`：将`newfd`指向`oldfd`的文件（若`newfd`已打开则先关闭）。
- **重定向标准输出到文件：**

```
int fd = open("output.txt", O_WRONLY | O_CREAT, 0644);
dup2(fd, STDOUT_FILENO); // 标准输出 (1) 指向fd的文件
close(fd); // 可关闭原fd，复制的1仍有效
printf("Hello"); // 内容写入output.txt
```

14. Linux的IO模型有哪些？阻塞IO和非阻塞IO的区别？

- **IO模型：**阻塞IO、非阻塞IO、IO多路复用、信号驱动IO（`SIGIO`）、异步IO（`aio_*`）。
- **阻塞IO：**调用`read/write`后，进程挂起直到IO完成；
- **非阻塞IO：**调用`read/write`时，若IO未就绪立即返回-1且`errno=EAGAIN`，需轮询重试。

15. sync()、fsync()、fdatasync()的区别？

- `sync()`：将所有脏页（内存中修改未写入磁盘的数据）写入队列，异步刷新（不等待完成）；
- `fsync(fd)`：同步刷新`fd`对应的文件数据和元数据（如修改时间）到磁盘，阻塞直到完成；
- `fdatasync(fd)`：仅同步文件数据（不保证元数据），比`fsync`高效，适用于对数据一致性要求高但元数据不重要的场景（如数据库）。

二、进程管理（16-35题）

16. 进程和程序的区别？

- **程序：**静态的可执行文件（指令+数据），存储在磁盘；
- **进程：**程序的动态执行实例，包含PCB（进程控制块）、地址空间、打开文件等资源，是资源分配的基本单位。
- **关系：**一个程序可对应多个进程（如多个`ls`命令进程）。

17. Linux进程的生命周期包含哪些状态？

- **就绪态（TASK_RUNNING）：**进程已就绪，等待CPU调度；
- **运行态（TASK_RUNNING）：**进程正在CPU上执行（Linux中就绪和运行统一为该状态）；
- **阻塞态（TASK_INTERRUPTIBLE/TASK_UNINTERRUPTIBLE）：**等待资源（如IO），前者可被信号唤醒，后者不可；
- **暂停态（TASK_STOPPED）：**被信号暂停（如`SIGSTOP`），可被`SIGCONT`恢复；
- **僵尸态（EXIT_ZOMBIE）：**进程终止，父进程未回收其资源（PCB仍存在）。

18. fork()函数的作用？子进程和父进程的区别？

- **作用：**创建子进程，子进程是父进程的副本。

- **核心机制**：写时复制（COW）—— 初始时子进程与父进程共享内存，当任一进程修改数据时才复制该页，减少内存开销。
- **区别**：
 - `fork()`返回值：父进程得到子进程PID，子进程得到0；
 - PID不同，PPID（父进程ID）：子进程的PPID是父进程PID；
 - 资源：子进程复制父进程的FD、信号掩码等，但不共享内存（COW机制下初始共享）。

19. `vfork()`与`fork()`的区别？为什么`vfork()`后通常要调用`exec()`？

- **区别**：
 - 内存共享：`vfork()`子进程与父进程共享地址空间（不复制，无COW），`fork()`初始共享但修改时复制；
 - 调度：`vfork()`保证子进程先运行，父进程挂起直到子进程调用`exec()`或`exit()`。
- **需调用`exec()`的原因**：子进程共享父进程内存，若修改数据会直接影响父进程，`exec()`会替换子进程地址空间，避免冲突。

20. `exec`系列函数的作用？`execl()`和`execv()`的区别？

- **作用**：替换当前进程的代码段、数据段和堆栈，执行新程序（进程PID不变）。
- **区别**：参数传递方式不同：
 - `execl(path, arg0, arg1, ..., NULL)`：参数列表可变（需显式传每个参数，以NULL结尾）；
 - `execv(path, argv)`：参数存于数组`argv`（`argv`最后一个元素为NULL）。
- 示例：`execl("/bin/ls", "ls", "-l", NULL)` 等价于 `char* argv[] = {"ls", "-l", NULL}; execv("/bin/ls", argv);`。

21. 进程如何正常终止？僵尸进程产生的原因及解决方法？

- **正常终止方式**：
 - 调用`exit(status)`：清理资源后终止；
 - 主函数`return`：等效于`exit(return_value)`；
 - 调用`_exit(status)`：直接终止（不刷新缓冲区）。
- **僵尸进程原因**：子进程终止后，父进程未调用`wait()/waitpid()`回收其退出状态，导致PCB残留。
- **解决方法**：
 - 父进程主动调用`wait()/waitpid()`；
 - 父进程退出，僵尸进程由init（PID=1）收养并回收；
 - 注册SIGCHLD信号处理函数，在信号中回收子进程。

22. `wait()`和`waitpid()`的区别？如何非阻塞回收子进程？

- `wait(int *status)`：阻塞等待任意子进程终止，返回终止子进程PID。
- `waitpid(pid_t pid, int *status, int options)`：更灵活：
 - `pid`：指定子进程PID（-1表示任意）；
 - `options`：WNOHANG（非阻塞，若无子进程终止返回0）、WUNTRACED（关注暂停状态）。
- **非阻塞回收**：

```
pid_t pid;
int status;
while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
    // 处理终止的子进程 (pid)
}
```

23. Linux进程调度策略有哪些？

- **实时调度策略**（优先级1-99）：
 - **SCHED_FIFO**：先进先出，一旦获得CPU则一直运行直到主动放弃；
 - **SCHED_RR**：时间片轮转，相同优先级进程轮流执行。
- **普通进程调度策略**（优先级100-139，值越大优先级越低）：
 - **SCHED_OTHER**（默认）：基于CFS（完全公平调度器）；
 - **SCHED_BATCH**：批处理进程，优先级低于**SCHED_OTHER**；
 - **SCHED_IDLE**：空闲时运行，优先级最低。

24. 什么是孤儿进程？与僵尸进程的区别？

- **孤儿进程**：父进程先于子进程终止，子进程被init进程（PID=1）收养，init会负责回收其资源（不会成为僵尸）。
- **区别**：
 - 僵尸进程：子进程终止，父进程未回收；
 - 孤儿进程：父进程终止，子进程被收养，正常运行或终止后被回收。

25. 如何查看进程的状态和资源使用情况？常用命令有哪些？

- **ps**：查看进程快照（如**ps aux**显示所有进程详细信息）；
- **top/htop**：动态查看进程资源使用（CPU、内存、状态等）；
- **pstree**：以树状显示进程间父子关系；
- **/proc/[pid]/**：内核提供的进程信息伪文件系统（如**/proc/1234/status**含进程状态，**/proc/1234/fd**含打开的FD）。

26. 进程的虚拟地址空间布局？用户空间和内核空间如何划分？

- **虚拟地址空间**（32位系统为例，共4GB）：
 - 用户空间（0-3GB）：代码段（.text）、数据段（.data/.bss）、堆（向上生长）、栈（向下生长）、共享库；
 - 内核空间（3GB-4GB）：内核代码、数据、页表等，所有进程共享。
- 64位系统：用户空间通常为 $0-2^{47}$ 字节，内核空间为 $2^{47}-2^{64}$ 字节（不同架构可能不同）。

27. 什么是系统调用（System Call）？进程如何从用户态进入内核态？

- **系统调用**：用户进程请求内核提供服务的接口（如**open/fork**），是用户态与内核态交互的桥梁。
- **进入内核态的方式**：
 - 触发软中断（如x86的**int 0x80**或**syscall**指令）；
 - 内核验证系统调用号，执行对应服务，返回结果到用户态。

28. `setuid()`和`setgid()`的作用？有效UID和实际UID的区别？

- **UID类型：**
 - 实际UID (RUID)：进程创建者的UID；
 - 有效UID (EUID)：决定进程的权限（文件访问等）；
 - 保存的设置UID (SUID)：保存EUID的副本，用于权限切换。
- **`setuid(uid)`：**
 - 若进程有root权限 (EUID=0)，则RUID、EUID、SUID均设为uid；
 - 否则，仅当uid等于RUID或SUID时，EUID设为uid。
- 作用：实现权限临时切换（如passwd命令需要修改/etc/passwd，通过SUID位获取root权限）。

29. 什么是守护进程 (Daemon)？如何创建守护进程？

- **守护进程：**后台运行的特殊进程，独立于终端，通常用于提供服务（如sshd、httpd）。
- **创建步骤：**
 1. `fork()`创建子进程，父进程退出（脱离终端控制）；
 2. 子进程调用`setsid()`创建新会话（脱离原会话，成为会话组长）；
 3. 再次`fork()`（可选，确保进程不是会话组长，避免打开终端）；
 4. 切换工作目录（如`chdir("/")`）；
 5. 关闭不必要的FD（如0、1、2）；
 6. 重定向标准输入/输出/错误到/dev/null。

30. 进程间的上下文切换 (Context Switch) 包含哪些操作？代价如何？

- **操作：**
 - 保存当前进程的CPU寄存器状态（如PC、栈指针）；
 - 更新进程控制块（PCB）中的状态信息；
 - 将进程移出运行队列，加入对应状态队列（如就绪、阻塞）；
 - 选择下一个运行进程，加载其寄存器状态和内存映射；
 - 刷新TLB（Translation Lookaside Buffer，地址转换缓存）。
- **代价：**消耗CPU时间（约几微秒到几十微秒），频繁切换会降低系统吞吐量。

31. 什么是进程组和会话 (Session)？

- **进程组：**一组相关进程的集合，每个进程组有一个进程组ID (PGID)，由组长进程的PID决定。
- **会话：**一组进程组的集合，有一个会话ID (SID)，由创建会话的进程PID决定。
- **关系：**会话 → 进程组 → 进程（一个会话可含多个进程组，一个进程组可含多个进程）。
- **应用：**终端控制（一个会话通常关联一个终端，前台进程组接收终端输入）。

32. `kill()`函数的作用？如何向进程发送信号？

- **`kill(pid, sig)`：**向指定进程/进程组发送信号sig。
- **pid参数含义：**
 - `pid > 0`：发送给PID为pid的进程；
 - `pid = 0`：发送给与当前进程同进程组的所有进程；
 - `pid = -1`：发送给所有有权限发送的进程（除init）；
 - `pid < -1`：发送给进程组ID为-pid的所有进程。
- **示例：**`kill(1234, SIGKILL)` 强制终止PID=1234的进程。

33. 常见的信号及其作用？SIGTERM和SIGKILL的区别？

- 常见信号：
 - SIGINT (2)：终端中断 (Ctrl+C)；
 - SIGQUIT (3)：终端退出 (Ctrl+\)，产生core dump；
 - SIGKILL (9)：强制终止，不可捕获/忽略；
 - SIGTERM (15)：请求终止，可捕获/忽略（默认终止）；
 - SIGCHLD (17)：子进程状态变化（终止/暂停）；
 - SIGPIPE (13)：向已关闭的管道/套接字写数据。
- 区别：SIGTERM允许进程清理资源后退出，SIGKILL强制终止（用于无法正常退出的进程）。

34. 如何自定义信号处理函数？signal()和sigaction()的区别？

- signal(signum, handler)：简单注册信号处理函数，但行为在不同系统可能不一致（如是否自动重置 handler）。
- sigaction(signum, &act, &oldact)：更健壮，可设置更多选项（如SA_RESTART使系统调用被信号中断后自动重启）。
- 示例 (sigaction)：

```
struct sigaction act;
act.sa_handler = handler_func; // 处理函数
sigemptyset(&act.sa_mask);    // 信号处理期间屏蔽的信号集
act.sa_flags = SA_RESTART;    // 选项
sigaction(SIGINT, &act, NULL);
```

35. 什么是core dump？如何开启和分析core dump文件？

- core dump：进程异常终止时，内核将其内存状态写入磁盘文件（通常名为core），用于调试。
- 开启：ulimit -c unlimited（当前shell生效），或修改/etc/security/limits.conf永久生效。
- 分析：gdb 程序名 core，通过bt（backtrace）查看崩溃时的调用栈。

三、线程与线程同步（36-55题）

36. 线程与进程的区别？为什么说线程是轻量级进程？

- 区别：
 - 资源分配：进程是资源分配单位（独立地址空间、FD等），线程共享进程资源（地址空间、FD表）；
 - 调度：线程是调度单位（CPU分配对象）；
 - 开销：线程创建/销毁、上下文切换开销远小于进程（无需复制资源）。
- 轻量级原因：线程共享进程资源，创建时仅需分配少量私有资源（栈、寄存器），切换时无需切换地址空间。

37. 线程的私有资源有哪些？共享资源有哪些？

- 私有资源：线程ID（TID）、栈空间、寄存器状态、errno变量、信号掩码、线程特有数据（TSD）。

- **共享资源**：进程地址空间（代码、数据、堆）、文件描述符、信号处理函数、进程ID（PID）、用户ID（UID）。

38. 如何创建线程？`pthread_create()`的参数含义？

- **函数原型**：`int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);`
- **参数**：
 - `thread`：输出参数，返回新线程ID；
 - `attr`：线程属性（如栈大小、分离状态，`NULL`为默认）；
 - `start_routine`：线程入口函数（返回`void*`，参数`void*`）；
 - `arg`：传递给`start_routine`的参数。
- **返回值**：成功返回0，失败返回错误码（非-1，需用`strerror()`查看）。

39. 线程的分离状态（Detached）和可连接状态（Joinable）的区别？

- **可连接状态（默认）**：线程终止后，资源（如返回值）需通过`pthread_join()`回收，否则成为“僵尸线程”。
- **分离状态**：线程终止后自动释放资源，无需`pthread_join()`，且不能被连接。
- **设置方式**：
 - 创建时：`pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);`
 - 创建后：`pthread_detach(tid)`。

40. `pthread_join()`的作用？如何获取线程的返回值？

- **作用**：阻塞等待指定线程终止，回收其资源，并获取返回值。
- **函数原型**：`int pthread_join(pthread_t thread, void **retval);`
- **获取返回值**：线程入口函数通过`return`返回值，`pthread_join()`的`retval`接收（需强制类型转换）：

```
void* thread_func(void* arg) {
    return (void*)42; // 线程返回值
}

int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, thread_func, NULL);
    void* ret;
    pthread_join(tid, &ret);
    printf("返回值: %d\n", (int)ret); // 输出42
}
```

41. 什么是线程安全？如何保证线程安全？

- **线程安全**：多线程并发访问时，无需额外同步机制即可保证结果正确。
- **保证方式**：
 - 无状态设计（不使用全局/静态变量）；
 - 使用同步机制（互斥锁、读写锁等）保护共享资源；
 - 使用线程局部存储（TLS）存储线程私有数据。

42. 互斥锁 (Mutex) 的作用? `pthread_mutex_init()`、`pthread_mutex_lock()`、`pthread_mutex_unlock()` 的用法?

- **作用:** 保证同一时间只有一个线程访问共享资源 (互斥访问)。
- **用法:**

```
pthread_mutex_t mutex;  
pthread_mutex_init(&mutex, NULL); // 初始化 (默认属性)  
  
// 访问共享资源前加锁  
pthread_mutex_lock(&mutex);  
// 临界区操作 (如修改全局变量)  
pthread_mutex_unlock(&mutex); // 解锁  
  
pthread_mutex_destroy(&mutex); // 销毁
```

- **注意:** 必须成对加锁/解锁, 避免死锁; `pthread_mutex_lock()` 会阻塞直到获取锁。

43. 什么是死锁? 产生死锁的四个必要条件? 如何避免死锁?

- **死锁:** 多个线程互相等待对方释放资源, 导致永久阻塞。
- **四个必要条件:**
 1. 互斥: 资源只能被一个线程占用;
 2. 持有并等待: 线程持有部分资源, 同时等待其他资源;
 3. 不可剥夺: 资源不能被强制剥夺;
 4. 循环等待: 线程间形成资源等待环。
- **避免方法:**
 - 按固定顺序申请资源 (打破循环等待);
 - 限时等待 (如 `pthread_mutex_trylock()`);
 - 一次性申请所有资源。

44. 条件变量 (Condition Variable) 的作用? 与互斥锁的配合使用流程?

- **作用:** 用于线程间同步 (如等待某个条件成立), 允许线程阻塞等待, 直到被其他线程唤醒。
- **配合流程:**

```
pthread_mutex_t mutex;  
pthread_cond_t cond;  
int ready = 0; // 共享条件  
  
// 等待线程  
pthread_mutex_lock(&mutex);  
while (!ready) { // 循环检查 (避免虚假唤醒)  
    pthread_cond_wait(&cond, &mutex); // 释放锁并阻塞, 被唤醒后重新获取锁  
}  
// 条件成立后的操作  
pthread_mutex_unlock(&mutex);
```

```
// 唤醒线程
pthread_mutex_lock(&mutex);
ready = 1;
pthread_cond_signal(&cond); // 唤醒一个等待线程 (pthread_cond_broadcast唤醒所有)
pthread_mutex_unlock(&mutex);
```

- 注意: `pthread_cond_wait()` 必须在持有互斥锁时调用, 且需用 `while` 循环检查条件 (防止虚假唤醒)。

45. 读写锁 (Reader-Writer Lock) 的特点? 适用场景?

- 特点: 区分读操作和写操作, 允许多个读者同时读, 但写者需独占资源 (读-读不互斥, 读-写互斥, 写-写互斥)。
- 函数: `pthread_rwlock_rdlock()` (加读锁)、`pthread_rwlock_wrlock()` (加写锁)、`pthread_rwlock_unlock()` (解锁)。
- 适用场景: 读操作远多于写操作的场景 (如缓存、配置文件读取), 提高并发效率。

46. 信号量 (Semaphore) 与互斥锁的区别?

特性	互斥锁 (Mutex)	信号量 (Semaphore)
用途	保护临界区 (互斥访问)	控制资源访问数量 (如有限缓冲区)
所有权	加锁线程必须解锁 (线程关联)	可由不同线程加锁/解锁 (无关联)
值范围	0或1 (二元信号量)	非负整数 (可表示资源数量)
适用场景	单资源互斥	多资源竞争 (如线程池、生产者-消费者)

47. 线程特有数据 (TSD) 的作用? 如何使用?

- 作用: 为每个线程提供私有数据 (类似全局变量, 但线程独立), 解决线程安全问题。
- 使用步骤:
 1. `pthread_key_create(&key, destructor)`: 创建键 (`destructor` 为数据销毁函数);
 2. `pthread_setspecific(key, value)`: 为当前线程设置键值;
 3. `pthread_getspecific(key)`: 获取当前线程的键值;
 4. `pthread_key_delete(key)`: 删除键 (不自动调用析构函数)。

48. 什么是线程池? 线程池的优势?

- 线程池: 预先创建一定数量的线程, 统一管理并复用, 避免频繁创建/销毁线程的开销。
- 优势:
 - 减少线程创建/销毁的时间开销;
 - 控制并发线程数量, 避免资源耗尽;
 - 提高响应速度 (线程已就绪, 无需等待创建)。
- 核心组件: 线程队列、任务队列、任务添加/获取接口、线程管理逻辑。

49. 生产者-消费者模型的实现? 如何用线程和信号量实现?

- **模型**：生产者线程生成数据放入缓冲区，消费者线程从缓冲区取数据处理，需同步保证缓冲区操作安全。
- **实现（信号量）**：

```
#define BUF_SIZE 10
int buffer[BUF_SIZE];
int in = 0, out = 0;

sem_t empty; // 空槽数量 (初始BUF_SIZE)
sem_t full;   // 数据数量 (初始0)
pthread_mutex_t mutex; // 保护缓冲区访问

// 生产者
void* producer(void* arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&empty); // 等待空槽
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        in = (in + 1) % BUF_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full); // 增加数据计数
    }
}

// 消费者
void* consumer(void* arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&full); // 等待数据
        pthread_mutex_lock(&mutex);
        int data = buffer[out];
        out = (out + 1) % BUF_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty); // 增加空槽计数
    }
}
```

50. 什么是重入函数？如何编写可重入函数？

- **重入函数**：可被多个线程同时调用，或被信号中断后再次调用而不产生错误的函数。
- **编写原则**：
 - 不使用全局/静态变量；
 - 不使用`malloc/free`（内部可能用全局锁）；
 - 不调用不可重入函数；
 - 若必须使用共享资源，需通过同步机制保护。

51. `pthread_once()`的作用？

- **作用**：保证指定函数（初始化函数）在多线程环境下仅被执行一次（如初始化全局资源）。
- **用法**：

```
pthread_once_t once = PTHREAD_ONCE_INIT;
void init_func() {
    // 初始化操作 (如创建互斥锁、加载配置)
}
void* thread_func(void* arg) {
    pthread_once(&once, init_func); // 所有线程中仅执行一次init_func
    // 后续操作
}
```

52. 线程取消 (Thread Cancellation) 的流程? `pthread_cancel()`和`pthread_setcancelstate()`的作用?

- **流程:**
 1. 线程A调用`pthread_cancel(tid)`向线程B发送取消请求;
 2. 线程B在取消点 (如`pthread_join()`、`read()`等系统调用) 响应取消;
 3. 线程B执行清理函数 (`pthread_cleanup_push()`注册), 然后终止。
- **相关函数:**
 - `pthread_cancel(tid)`: 发送取消请求;
 - `pthread_setcancelstate(state, oldstate)`: 设置取消状态 (`PTHREAD_CANCEL_ENABLE`允许取消, `PTHREAD_CANCEL_DISABLE`禁止);
 - `pthread_setcanceltype(type, oldtype)`: 设置取消类型 (`PTHREAD_CANCEL_DEFERRED`延迟到取消点, `PTHREAD_CANCEL_ASYNCHRONOUS`立即取消)。

53. 如何调试多线程程序? 常用工具和方法?

- **工具:**
 - `gdb`: 支持线程调试 (`info threads`查看线程, `thread id`切换线程, `bt`查看调用栈);
 - `valgrind --tool=helgrind`: 检测线程竞争和死锁;
 - `pstack`: 打印进程的线程栈信息;
 - `top -H`: 查看线程CPU使用率。
- **方法:** 添加日志 (含线程ID)、使用断言检测数据一致性、复现并定位竞争条件。

54. 什么是线程局部存储 (TLS)? 与全局变量的区别?

- **TLS:** 为每个线程分配独立的变量实例, 线程间互不干扰。
- **声明方式:** C11标准`_Thread_local`, GCC扩展`__thread`。
- **区别:** 全局变量在进程内所有线程共享, TLS变量每个线程有独立副本。
- **示例:** `__thread int count = 0;` (每个线程的`count`独立)。

55. 多线程环境下`printf()`是否线程安全? 为什么?

- **不一定安全:** `printf()`涉及缓冲IO操作 (修改共享缓冲区), 若内部无同步机制, 多线程同时调用可能导致输出错乱。
- **实现依赖:** 部分libc (如glibc) 对`printf()`加了锁, 保证线程安全; 但从可移植性考虑, 建议手动加锁保护。

四、进程间通信 (IPC) (56-70题)

56. Linux下有哪些进程间通信（IPC）方式？

- **传统IPC**：管道（Pipe）、FIFO（命名管道）、信号（Signal）；
- **System V IPC**：消息队列、共享内存、信号量集；
- **POSIX IPC**：消息队列、共享内存、信号量、互斥锁、条件变量（跨进程）；
- **网络IPC**：套接字（Socket）。

57. 无名管道（Pipe）的特点？如何创建和使用？

- **特点**：
 - 半双工（单向通信），需两个管道实现双向通信；
 - 仅用于父子进程或兄弟进程（有共同祖先）；
 - 基于文件描述符，可使用`read/write`操作。
- **创建**：`int pipe(int pipefd[2]);`（`pipefd[0]`为读端，`pipefd[1]`为写端）。
- **示例（父子进程通信）**：

```
int fd[2];
pipe(fd);
if (fork() == 0) { // 子进程写
    close(fd[0]); // 关闭读端
    write(fd[1], "hello", 5);
    close(fd[1]);
} else { // 父进程读
    close(fd[1]); // 关闭写端
    char buf[10];
    read(fd[0], buf, 10);
    close(fd[0]);
}
```

58. FIFO（命名管道）与无名管道的区别？

- **相同点**：均为半双工，数据先进先出，操作方式相同（`read/write`）。
- **区别**：
 - 存在形式：FIFO有文件名（存在于文件系统），无名管道无；
 - 通信范围：FIFO可用于任意进程（只要知道文件名），无名管道仅限亲缘进程；
 - 创建方式：FIFO用`mkfifo()`创建，无名管道用`pipe()`创建。

59. 如何使用FIFO实现两个非亲缘进程通信？

- **步骤**：
 1. 进程A：`mkfifo("myfifo", 0666)` 创建FIFO；
 2. 进程A：`open("myfifo", O_WRONLY)` 打开FIFO写端；
 3. 进程B：`open("myfifo", O_RDONLY)` 打开FIFO读端（若写端未打开则阻塞）；
 4. 进程A写入数据，进程B读取数据；
 5. 通信结束后关闭FD，删除FIFO（`unlink("myfifo")`）。

60. System V消息队列的特点？如何创建和发送/接收消息？

- **特点:**
 - 消息有类型（可按类型接收，实现优先级）；
 - 数据在内核中存储，进程退出后消息可保留；
 - 有最大消息大小和队列总大小限制。
- **操作函数:**
 - `msgget(key_t key, int msgflg)`: 创建或获取消息队列（返回msgid）；
 - `msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg)`: 发送消息；
 - `msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg)`: 接收消息。
- **消息结构:**

```
struct msgbuf {
    long mtype; // 消息类型 (>0)
    char mtext[1024]; // 消息数据
};
```

61. System V共享内存的优点？使用流程？

- **优点:** 进程间共享内存区域，无需数据拷贝（IPC中效率最高）。
- **使用流程:**
 1. `shmget(key_t key, size_t size, int shmflg)`: 创建或获取共享内存（返回shmid）；
 2. `shmat(int shmid, const void *shmaddr, int shmflg)`: 将共享内存映射到进程地址空间（返回映射地址）；
 3. 进程通过映射地址直接读写共享内存；
 4. `shmdt(const void *shmaddr)`: 解除映射；
 5. `shmctl(int shmid, int cmd, struct shmid_ds *buf)`: 删除共享内存（cmd=IPC_RMID）。
- **注意:** 需配合同步机制（如信号量）防止数据竞争。

62. System V信号量集的作用？`semget()`、`semop()`的用法？

- **作用:** 用于进程间同步（如控制共享资源的访问数量），支持对多个信号量原子操作。
- **操作函数:**
 - `semget(key_t key, int nsems, int semflg)`: 创建或获取信号量集（`nsems`为信号量数量）；
 - `semop(int semid, struct sembuf *sops, size_t nsops)`: 原子执行一组操作（`sops`为操作数组）。
- **操作结构:**

```
struct sembuf {
    unsigned short sem_num; // 信号量索引（在集中的位置）
    short sem_op;           // 操作（正数：V操作，负数：P操作，0：等待为0）
    short sem_flg;          // 标志（如IPC_NOWAIT非阻塞）
};
```

63. 信号 (Signal) 作为IPC的特点？局限性？

- **特点：**异步通知机制，可携带简单信息（信号编号）。
- **局限性：**
 - 信息少（仅信号编号，无法传递复杂数据）；
 - 信号可能丢失（如相同信号未处理时会被合并）；
 - 处理函数需简洁（避免重入问题）。

64. 共享内存与消息队列的区别？

特性	共享内存	消息队列
数据传输	直接读写内存（无拷贝）	内核中转（数据从用户态→内核→用户态）
效率	最高	中等
同步	需额外机制（如信号量）	自带同步（ <code>msgsnd/msgrcv</code> 阻塞）
数据结构	无格式，需用户定义	有消息类型，结构化
持久性	内核中存在，直到被删除	内核中存在，直到被读取或删除

65. 什么是UNIX域套接字？与网络套接字的区别？

- **UNIX域套接字：**用于同一主机上的进程通信，基于文件系统路径标识，不经过网络协议栈。
- **与网络套接字的区别：**
 - 地址格式：UNIX域用`struct sockaddr_un`（含路径），网络用`struct sockaddr_in`（含IP和端口）；
 - 效率：UNIX域更高（无协议处理、校验和等开销）；
 - 范围：UNIX域仅限本地进程，网络套接字可跨主机。

66. 如何用共享内存+信号量实现进程间同步通信？

- **流程：**
 1. 创建共享内存（存储数据）和信号量集（含两个信号量：`empty`控制写，`full`控制读）；
 2. 写进程：`semop`操作`empty`（P操作）→ 写共享内存 → `semop`操作`full`（V操作）；
 3. 读进程：`semop`操作`full`（P操作）→ 读共享内存 → `semop`操作`empty`（V操作）；
 4. 通信结束后释放资源。

67. POSIX IPC与System V IPC的区别？

特性	System V IPC	POSIX IPC
标识方式	键（ <code>key_t</code> ）+ ID	名字（字符串）
创建/打开	分离函数（如 <code>msgget</code> ）	统一函数（如 <code>mq_open</code> ）
可移植性	较差（不同系统实现有差异）	较好（遵循POSIX标准）
通知机制	无（需配合信号）	支持异步通知（如 <code>mq_notify</code> ）
关闭/删除	需显式删除（如 <code>shmctl(IPC_RMID)</code> ）	引用计数，最后关闭时可自动删除

68. 什么是内存映射文件（Memory-Mapped File）？如何用于进程通信？

- **内存映射文件**：将磁盘文件映射到进程地址空间，通过内存操作实现文件IO。
- **进程通信**：多个进程映射同一文件，修改映射区会同步到文件，实现进程间共享数据（类似共享内存，但数据持久化到磁盘）。
- **优势**：数据自动持久化，适合需要持久化的共享数据。

69. 各种IPC方式的效率对比？

- **从高到低排序**：
 1. 共享内存（无数据拷贝）；
 2. 内存映射文件（仅首次映射开销）；
 3. UNIX域套接字（本地通信，无网络开销）；
 4. 管道/FIFO（内核缓冲，一次拷贝）；
 5. 消息队列（内核存储，两次拷贝）；
 6. 信号（仅传递编号，不适合大量数据）。

70. 如何选择合适的IPC方式？

- **数据量小、需异步通知**：信号；
- **亲缘进程、简单通信**：无名管道；
- **非亲缘进程、简单通信**：FIFO；
- **大量数据、高效通信**：共享内存（+信号量同步）；
- **需要消息类型、结构化数据**：消息队列；
- **跨主机通信**：网络套接字；
- **本地跨进程、类似网络编程**：UNIX域套接字。

五、网络编程与epoll（71-90题）

71. TCP和UDP的区别？适用场景分别是什么？

特性	TCP	UDP
连接性	面向连接（三次握手建立连接）	无连接
可靠性	可靠（重传、确认、拥塞控制）	不可靠（无重传，可能丢失）
有序性	保证数据按序到达	不保证
速度	较慢（开销大）	较快（开销小）
拥塞控制	有	无
适用场景	文件传输、HTTP、邮件（需可靠）	视频通话、直播、DNS（需快速）

72. TCP的三次握手过程？为什么需要三次握手？

- **三次握手**：
 1. 客户端→服务器：SYN（seq=x），客户端进入SYN_SENT状态；
 2. 服务器→客户端：SYN+ACK（seq=y, ack=x+1），服务器进入SYN_RCVD状态；

3. 客户端→服务器: ACK (ack=y+1) , 双方进入ESTABLISHED状态。

- **原因:** 防止过期的连接请求报文被服务器接收, 导致错误建立连接。三次握手可确保双方均确认“对方已准备好”。

73. TCP的四次挥手过程? 为什么需要四次挥手?

- **四次挥手:**

1. 客户端→服务器: FIN (关闭请求, seq=u) , 客户端进入FIN_WAIT_1;
2. 服务器→客户端: ACK (ack=u+1) , 服务器进入CLOSE_WAIT, 客户端进入FIN_WAIT_2;
3. 服务器→客户端: FIN (seq=v) , 服务器进入LAST_ACK;
4. 客户端→服务器: ACK (ack=v+1) , 客户端进入TIME_WAIT, 服务器进入CLOSED; 客户端等待2MSL后进入CLOSED。

- **原因:** TCP是全双工通信, 双方需分别关闭各自的发送通道, 故需要四次挥手 (FIN和ACK分两次发送) 。

74. TCP的TIME_WAIT状态的作用? 时长是多少?

- **作用:**

1. 确保最后一个ACK被服务器接收 (若丢失, 服务器会重发FIN, 客户端可重发ACK) ;
2. 防止过期报文干扰新连接 (等待2MSL让网络中残留的报文失效) 。

- **时长:** 2MSL (Maximum Segment Lifetime, 报文最大生存时间, 通常为2分钟) 。

75. 什么是TCP的滑动窗口机制? 作用是什么?

- **滑动窗口:** TCP用于流量控制的机制, 接收方通过告知发送方自己的接收窗口大小 (rwnd) , 控制发送方的发送速率。

- **作用:**

- 实现流量控制 (避免接收方缓冲区溢出) ;
- 支持批量发送 (连续发送多个报文, 无需等待每个ACK) , 提高效率。

76. 什么是TCP的拥塞控制? 包含哪些算法?

- **拥塞控制:** 防止网络因过多数据而拥塞, 通过动态调整发送窗口 (cwnd) 实现。

- **核心算法:**

- 慢启动 (Slow Start) : 初始cwnd小, 每收到一个ACK就加倍, 直到达到慢启动阈值 (sssthresh) ;
- 拥塞避免 (Congestion Avoidance) : cwnd达到sssthresh后, 每次RTT增加1, 缓慢增长;
- 拥塞发生 (超时) : sssthresh设为cwnd/2, cwnd重置为1, 重新慢启动;
- 快速重传/恢复: 收到3个重复ACK, 认为报文丢失, sssthresh设为cwnd/2, cwnd设为sssthresh, 进入快速恢复。

77. socket编程的基本流程 (TCP服务器和客户端) ?

- **TCP服务器:**

1. `socket()`: 创建套接字 (`AF_INET`, `SOCK_STREAM`) ;
2. `bind()`: 绑定IP和端口;
3. `listen()`: 监听连接 (设置backlog) ;

4. `accept()`: 阻塞等待客户端连接 (返回新的通信套接字) ;
5. `read()/write()`: 与客户端通信;
6. `close()`: 关闭套接字。

- **TCP客户端:**

1. `socket()`: 创建套接字;
2. `connect()`: 连接服务器;
3. `read()/write()`: 与服务器通信;
4. `close()`: 关闭套接字。

78. `socket()`函数的参数含义? `SOCK_STREAM`和`SOCK_DGRAM`的区别?

- **函数原型:** `int socket(int domain, int type, int protocol);`
- **参数:**
 - `domain`: 地址族 (`AF_INET`为IPv4, `AF_INET6`为IPv6, `AF_UNIX`为UNIX域) ;
 - `type`: 套接字类型 (`SOCK_STREAM`为TCP, `SOCK_DGRAM`为UDP) ;
 - `protocol`: 协议 (通常为0, 由前两个参数决定) 。
- **区别:** `SOCK_STREAM`提供面向连接、可靠的字节流服务; `SOCK_DGRAM`提供无连接、不可靠的数据包服务。

79. `bind()`函数绑定失败的可能原因?

- 端口被占用 (`Address already in use`) : 需等待端口释放或设置`SO_REUSEADDR`选项;
- 权限不足: 绑定1024以下端口需root权限;
- IP地址无效或不属于本机;
- 套接字已绑定到其他地址。

80. `listen()`函数的`backlog`参数含义?

- `backlog`: 指定处于SYN_RCVD状态的未完成连接队列的最大长度 (实际长度可能受系统限制, 如Linux中为`min(backlog, /proc/sys/net/core/somaxconn)`) 。
- 若队列满, 新的连接请求会被忽略 (客户端可能收到`ECONNREFUSED`) 。

81. UDP编程与TCP编程的区别?

- **服务器流程:** UDP无需`listen()`和`accept()`, 直接用`recvfrom()/sendto()`收发数据 (需指定对方地址) ;
- **连接性:** TCP有`connect()`建立连接, UDP可调用`connect()`绑定对方地址 (但仍无连接, 仅简化收发) ;
- **数据边界:** UDP保留消息边界 (`recvfrom()`一次返回一个完整消息), TCP是字节流 (无边界) ;
- **可靠性:** UDP需自行处理丢包、重传, TCP由协议保证。

82. 什么是IO复用? `epoll`相比`select`和`poll`有哪些优势?

- **IO复用:** 允许一个进程同时监控多个文件描述符 (如套接字), 当某个FD就绪时再进行IO操作。
- **`epoll`优势:**
 - **FD数量:** 无上限 (仅受系统内存), `select/poll`有上限;

- **效率**：epoll用红黑树存储FD，就绪FD通过链表返回，无需轮询（O(1)获取就绪FD），select/poll需轮询所有FD（O(n)）；
- **内存拷贝**：epoll的`epoll_ctl`仅在注册/修改时拷贝FD信息，select/poll每次调用都需拷贝整个FD集合；
- **触发方式**：epoll支持边缘触发（ET）和水平触发（LT），select/poll仅支持LT。

83. epoll的工作原理？`epoll_create()`、`epoll_ctl()`、`epoll_wait()`的作用？

- **原理**：
 1. `epoll_create()`创建epoll实例（内核数据结构，含红黑树和就绪链表）；
 2. `epoll_ctl()`向实例添加/修改/删除需要监控的FD及事件（如EPOLLIN可读）；
 3. `epoll_wait()`阻塞等待，返回就绪的FD及事件（从就绪链表获取）。
- **函数作用**：
 - `epoll_create(size)`：创建epoll实例（`size`参数已忽略，返回epoll FD）；
 - `epoll_ctl(epfd, op, fd, event)`：管理监控的FD（`op`为EPOLL_CTL_ADD/MOD/DEL）；
 - `epoll_wait(epfd, events, maxevents, timeout)`：等待就绪事件（`events`存储结果，`timeout`为超时毫秒数）。

84. epoll的水平触发（LT）和边缘触发（ET）的区别？

- **水平触发（LT，默认）**：只要FD有数据可读/可写，`epoll_wait()`就会返回该FD（适合阻塞IO，操作简单）。
- **边缘触发（ET）**：仅当FD状态从不可就绪变为可就绪时，`epoll_wait()`才返回（需用非阻塞IO，效率更高，需一次性处理完数据）。
- **示例**：FD缓冲区有100字节，LT模式下`epoll_wait()`会多次返回，直到数据读完；ET模式下仅返回一次，需循环读取直到EAGAIN。

85. 如何用epoll实现高并发服务器？

- **步骤**：
 1. 创建监听套接字，设置SO_REUSEADDR选项，绑定并监听；
 2. 创建epoll实例，将监听套接字添加到epoll（EPOLLIN事件，LT模式）；
 3. 进入循环：调用`epoll_wait()`获取就绪FD；
 4. 若就绪FD是监听套接字，调用`accept()`接收新连接，将新套接字设为非阻塞，添加到epoll（EPOLLIN事件，ET模式）；
 5. 若就绪FD是通信套接字，循环读取数据（直到EAGAIN），处理后发送响应；
 6. 关闭断开的连接，从epoll中移除。

86. 什么是粘包？TCP粘包的原因及解决方法？

- **粘包**：TCP是字节流，多次发送的数据可能被合并成一个包接收，或一个包被拆分。
- **原因**：
 - 发送方：Nagle算法（合并小数据包）；
 - 接收方：缓冲区未滿，未及时通知应用层。
- **解决方法**：
 - 固定包长；
 - 包头部添加长度字段；
 - 使用分隔符（如\r\n）；

- 关闭Nagle算法 (`setsockopt(sock, IPPROTO_TCP, TCP_NODELAY, &on, sizeof(on))`)。

87. `setsockopt()`函数的作用？常用的套接字选项有哪些？

- **作用：**设置或获取套接字的选项（如缓冲区大小、超时时间、地址复用）。
- **常用选项：**
 - `SO_REUSEADDR`：允许重用本地地址和端口（解决Address already in use）；
 - `SO_RCVBUF`/`SO_SNDBUF`：设置接收/发送缓冲区大小；
 - `SO_RCVTIMEO`/`SO_SNDTIMEO`：设置接收/发送超时时间；
 - `TCP_NODELAY`：关闭Nagle算法（减少延迟）。

88. 什么是IO模型中的“同步”与“异步”？“阻塞”与“非阻塞”？

- **同步IO：**进程需主动等待IO操作完成（如`read`阻塞或轮询非阻塞FD）；
- **异步IO：**进程发起IO后继续执行，IO完成后内核通知进程（如`aio_read`）；
- **阻塞IO：**IO未就绪时，进程挂起等待；
- **非阻塞IO：**IO未就绪时，立即返回错误（`EAGAIN`），进程不挂起。

89. 什么是keepalive机制？如何设置TCP的keepalive参数？

- **keepalive：**检测TCP连接是否存活的机制，定期发送探测报文，若对方无响应则关闭连接。
- **设置参数：**
 - `TCP_KEEPIDLE`：空闲时间（多久未活动后开始探测，默认7200秒）；
 - `TCP_KEEPINTVL`：探测间隔（默认75秒）；
 - `TCP_KEEPCNT`：探测次数（默认9次，失败则关闭连接）。
- 示例：`setsockopt(sock, SOL_TCP, TCP_KEEPIDLE, &idle, sizeof(idle))`；

90. 如何实现一个简单的HTTP服务器？

- **核心步骤：**
 1. 创建TCP监听套接字，绑定80端口，监听连接；
 2. 接收客户端连接（`accept()`）；
 3. 读取HTTP请求（如`GET /index.html HTTP/1.1`）；
 4. 解析请求，获取资源路径；
 5. 读取对应文件内容，构建HTTP响应（状态行+头部+空行+ body）；
 6. 发送响应给客户端，关闭连接（HTTP/1.0默认短连接）。

六、其他Linux系统相关（91-100题）

91. 什么是虚拟内存？作用是什么？

- **虚拟内存：**操作系统为进程提供的抽象地址空间，使进程认为自己独占连续内存，实际映射到物理内存或磁盘（swap）。
- **作用：**
 - 进程隔离（地址空间独立）；
 - 扩大可用内存（借助磁盘swap）；
 - 内存保护（权限控制）；
 - 高效内存管理（分页机制）。

92. 分页 (Paging) 和分段 (Segmentation) 的区别?

- **分页**: 将虚拟内存和物理内存分为固定大小的页 (如4KB), 通过页表映射, 内存分配灵活, 可能产生内存碎片。
- **分段**: 按逻辑意义 (如代码段、数据段) 划分, 大小可变, 通过段表映射, 避免内存碎片, 但可能产生外碎片。
- Linux主要使用分页机制, 结合段表实现地址转换。

93. 什么是页表? TLB的作用?

- **页表**: 存储虚拟页号到物理页号的映射关系, 每个进程有独立页表。
- **TLB (Translation Lookaside Buffer)**: 页表缓存, 存储最近使用的页表项, 减少CPU访问内存页表的次数, 提高地址转换速度。

94. Linux的swap分区作用? 何时会使用swap?

- **swap分区**: 磁盘上的一块区域, 作为物理内存的补充。
- **使用场景**: 物理内存不足时, 内核将不常用的内存页 (inactive pages) 换出到swap, 释放物理内存给活跃进程。
- 过度使用swap会导致系统性能下降 (磁盘IO远慢于内存)。

95. strace工具的作用? 如何用它调试系统调用问题?

- **作用**: 跟踪进程执行的系统调用和接收的信号, 输出调用参数、返回值和错误信息。
- **用法**: `strace ./program` 跟踪程序的系统调用; `strace -p pid` 跟踪已有进程。
- **应用**: 调试程序崩溃原因 (如open失败的errno)、分析程序IO行为。

96. lsof工具的作用? 如何查看某个进程打开的文件?

- **作用**: 列出系统中打开的文件 (包括普通文件、FD、套接字等)。
- **查看进程打开的文件**: `lsof -p pid` (pid为进程ID)。
- **常用选项**: `lsof filename` 查看打开该文件的进程; `lsof -i :80` 查看占用80端口的进程。

97. 什么是inode节点号? 如何通过inode查找文件?

- **inode节点号**: 文件系统中唯一标识文件的号码 (类似身份证)。
- **通过inode查找文件**: `find /path -inum inode_number` (在指定路径下查找inode为inode_number的文件)。

98. Linux的信号驱动IO (Signal-Driven IO) 的工作原理?

- **原理**:
 1. 进程设置套接字为信号驱动IO模式 (`fcntl`设置`O_ASYNC`), 并指定信号 (通常`SIGIO`) 的处理函数;
 2. 内核在IO就绪时发送`SIGIO`信号给进程;
 3. 进程在信号处理函数中执行IO操作 (如`read`)。
- **特点**: 非阻塞, 无需轮询, 但信号处理复杂, 不适合高并发。

99. 什么是异步IO (AIO) ? Linux下如何使用AIO?

- **异步IO**：进程发起IO请求后继续执行，IO完成后内核通过回调或信号通知进程，全程不阻塞。
- **Linux AIO使用**：通过libaio库函数 (io_setup、io_submit、io_getevents)：

```
io_context_t ctx;
io_setup(128, &ctx); // 初始化AIO上下文

struct iocb cb, *cbs[1];
io_prep_pread(&cb, fd, buf, size, offset); // 准备读请求
cbs[0] = &cb;
io_submit(ctx, 1, cbs); // 提交请求

struct io_event events[1];
io_getevents(ctx, 1, 1, events, NULL); // 获取完成的事件
```

100. 如何排查Linux系统的性能问题? 常用工具和指标?

- **CPU问题**：top/mpstat (查看CPU使用率、负载)，vmstat (查看上下文切换)；
- **内存问题**：free (内存使用)，vmstat (swap in/out)，top (进程内存占用)；
- **磁盘IO问题**：iostat (磁盘读写速率、IOPS)，iotop (进程磁盘IO)；
- **网络问题**：netstat/ss (连接状态、端口占用)，iftop (网络流量)，tcpdump (抓包分析)。

C++ 开发

一、C++语法基础 (1-15题)

1. C++与C的主要区别是什么?

- **解析**：C++是C的超集，新增核心特性：
 - 面向对象编程 (类、继承、多态)；
 - 泛型编程 (模板)；
 - 异常处理 (try/catch/throw)；
 - 命名空间 (namespace)；
 - 引用 (&)、constexpr、auto等关键字；
 - 标准库扩展 (STL容器、算法)。
- **设计思想差异**：C是面向过程，C++支持多范式 (面向对象、泛型、过程式)。

2. 指针与引用的区别?

特性	指针 (Pointer)	引用 (Reference)
定义	存储变量地址的变量	变量的别名 (必须初始化绑定到变量)
空值	可指向nullptr	无空引用 (必须绑定有效对象)
重指向	可改变指向的对象	一旦绑定, 不可更改

特性	指针 (Pointer)	引用 (Reference)
算术操作	支持指针运算 (++、--、+n)	不支持 (本质是对象本身)
初始化	可延迟初始化	必须在定义时初始化
sizeof	返回指针本身的大小 (如8字节)	返回绑定对象的大小

3. const关键字的用法？

- **修饰变量：**`const int a = 5;` 变量值不可修改（常量）。
- **修饰指针：**
 - `const int* p;` 指针指向的内容不可改（常量指针）；
 - `int* const p;` 指针本身不可改（指针常量）；
 - `const int* const p;` 指针及其指向内容均不可改。
- **修饰函数参数：**`void func(const int& x);` 防止参数被修改，避免拷贝（传引用）。
- **修饰函数返回值：**`const int func();` 返回值不可作为左值。
- **修饰类成员函数：**`class A { void func() const; };` 成员函数不可修改类的成员变量（mutable修饰的除外）。

4. static关键字的用法？

- **全局static变量：**作用域限于当前文件，避免全局命名冲突。
- **局部static变量：**生命周期延长至程序结束，仅初始化一次（线程不安全）。
- **static类成员变量：**属于类而非对象，所有对象共享，需在类外初始化（`int A::count = 0;`）。
- **static类成员函数：**无this指针，只能访问static成员，可通过类名直接调用（`A::func()`）。

5. 什么是函数重载（Overload）？ 重载的条件是什么？

- **函数重载：**同一作用域内，函数名相同但参数列表不同的函数。
- **条件：**参数个数、类型、顺序不同（返回值类型不同不构成重载）。
- **原理：**编译器通过参数列表生成不同的函数名（名字修饰，Name Mangling）。

6. 什么是默认参数？ 默认参数的规则？

- **默认参数：**函数声明时为参数指定默认值，调用时可省略该参数。
- **规则：**
 - 默认参数必须从右向左定义（`void func(int a, int b=5, int c=10);` 合法，`void func(int a=5, int b);` 非法）；
 - 声明与定义中默认参数不能重复（通常在声明中指定）；
 - 函数重载时避免默认参数导致歧义。

7. 内联函数（inline）的作用？ 与宏定义的区别？

- **作用：**编译器将函数体插入调用处，避免函数调用开销（压栈、跳转等），适用于短小函数。
- **与宏的区别：**
 - 内联函数由编译器处理，有类型检查，宏由预处理器处理，无类型检查；
 - 内联函数可调试，宏不可调试；
 - 内联函数支持复杂逻辑（如循环），宏复杂时易出错（需加括号）。

- 注意：`inline`是建议，编译器可能忽略（如函数体过大）。

8. 什么是extern "C"? 作用是什么?

- **extern "C"**: 告知编译器按C语言规则编译代码（不进行名字修饰）。
- **作用**: 实现C++与C代码的混合编程（如C++调用C库函数）。
- 示例:

```
extern "C" {  
    #include "c_lib.h" // C头文件  
    void c_func(); // 按C规则编译  
}
```

9. 左值 (Lvalue) 与右值 (Rvalue) 的区别?

- **左值**: 可放在赋值号左边, 有明确内存地址, 可被取址 (如变量、数组元素)。
- **右值**: 只能放在赋值号右边, 无持久内存地址 (如字面量、表达式结果、临时对象)。
- C++11后右值细分为: 纯右值 (如`3+4`)、将亡值 (如临时对象)。

10. 什么是野指针? 如何避免?

- **野指针**: 指向已释放内存或非法地址的指针 (未初始化、指向内存被释放后未置空)。
- **避免方法**:
 - 指针定义时初始化 (`int* p = nullptr;`) ;
 - 内存释放后及时置空 (`delete p; p = nullptr;`) ;
 - 使用智能指针 (`unique_ptr/shared_ptr`) ;
 - 避免返回局部变量的指针/引用。

11. struct与class的区别?

- **默认访问权限**: struct默认public, class默认private。
- **默认继承方式**: struct默认public继承, class默认private继承。
- 其他功能完全一致 (均可定义成员变量、函数、模板等)。
- 习惯用法: struct用于数据聚合 (如POD类型), class用于面向对象设计。

12. 什么是POD类型?

- **POD (Plain Old Data)**: 满足以下条件的类型:
 - 平凡类型 (有默认构造函数、析构函数、拷贝构造函数等, 且均为编译器生成) ;
 - 标准布局 (非静态成员无访问权限差异, 无虚函数, 继承关系简单)。
- 特性: 可通过`memcpy`拷贝, 与C语言结构体兼容。

13. 什么是内存泄漏? 常见原因及检测方法?

- **内存泄漏**: 动态分配的内存未释放, 导致系统内存耗尽。
- **常见原因**:
 - `new/malloc`后未对应`delete/free`;

- 异常导致释放代码未执行;
- 循环引用 (如`shared_ptr`互相引用) 。
- **检测方法:**
 - 工具: Valgrind (Linux) 、 Visual Leak Detector (Windows) ;
 - 代码规范: 使用智能指针, 避免手动管理内存。

14. 什么是栈溢出? 原因是什么?

- **栈溢出:** 程序使用的栈空间超过系统限制。
- **原因:**
 - 递归调用过深 (每次递归压栈消耗栈空间) ;
 - 局部变量过大 (如`char buf[1024*1024*10];` 超过栈大小) 。
- 解决: 递归改循环, 大变量改用堆内存 (`new/malloc`) 。

15. C++中的类型转换有哪些? 各自的用途?

- **static_cast:** 编译期转换, 用于基本类型 (`int→double`)、派生类→基类 (上行转换)、`void*`与其他指针互转。
- **dynamic_cast:** 运行期转换, 用于基类→派生类 (下行转换), 需基类有虚函数, 失败返回`nullptr` (指针) 或抛出异常 (引用) 。
- **const_cast:** 移除变量的`const`属性 (`const int*→int*`), 仅用于指针/引用。
- **reinterpret_cast:** 底层二进制转换 (如`int→int*`), 风险高, 不保证可移植性。

二、面向对象编程 (16-35题)

16. 面向对象的三大特性?

- **封装:** 将数据 (成员变量) 和操作 (成员函数) 捆绑, 通过访问权限 (`public/private/protected`) 隐藏内部实现, 仅暴露接口。
- **继承:** 子类继承父类的属性和方法, 实现代码复用, 支持扩展 (如`class Dog : public Animal`) 。
- **多态:** 同一接口 (如虚函数) 在不同对象上表现出不同行为, 分为编译期多态 (函数重载、模板) 和运行期多态 (虚函数) 。

17. 访问权限控制 (`public/protected/private`) 的区别?

- **public:** 类内、类外、子类均可访问。
- **protected:** 类内、子类可访问, 类外不可访问。
- **private:** 仅类内可访问, 子类、类外均不可访问。
- 注意: 子类继承时, 父类的`private`成员不可见 (无论继承方式), `protected`成员在子类中仍为`protected` (`public`继承) 。

18. 继承方式 (`public/protected/private`) 的影响?

- **public继承:** 父类`public`→子类`public`, 父类`protected`→子类`protected` (is-a关系, 如`Student is a Person`) 。
- **protected继承:** 父类`public/protected`→子类`protected`。
- **private继承:** 父类`public/protected`→子类`private` (has-a关系, 更接近组合) 。

19. 什么是虚函数？实现原理是什么？

- **虚函数**：被`virtual`修饰的成员函数，支持运行期多态（子类重写后，通过基类指针/引用调用时，实际执行子类版本）。
- **实现原理**：
 - 类中存在虚函数时，编译器为类生成虚函数表（vtable），存储虚函数地址；
 - 每个对象包含一个虚表指针（vptr），指向类的vtable；
 - 调用虚函数时，通过vptr找到vtable，再调用对应函数（动态绑定）。

20. 纯虚函数与抽象类的作用？

- **纯虚函数**：`virtual void func() = 0;` 无实现的虚函数，强制子类重写。
- **抽象类**：包含纯虚函数的类，不能实例化（`Animal a;` 错误），只能作为基类被继承。
- **作用**：定义接口规范（如`Shape`类的`draw()`纯虚函数，子类`Circle/Rectangle`实现具体绘制）。

21. 虚析构函数的作用？

- **作用**：当通过基类指针删除子类对象时，确保子类析构函数被调用（避免内存泄漏）。
- **示例**：

```
class Base {
public:
    virtual ~Base() { cout << "Base destructor"; } // 虚析构
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived destructor"; }
};
// 正确调用: Derived::~~Derived() → Base::~~Base()
Base* p = new Derived();
delete p;
```

22. 什么是多态？如何实现运行期多态？

- **多态**：同一操作作用于不同对象，产生不同结果。
- **运行期多态实现条件**：
 - 基类定义虚函数；
 - 子类重写（override）该虚函数（函数名、参数、返回值需完全一致，协变返回值除外）；
 - 通过基类指针或引用调用该虚函数。

23. 什么是函数隐藏（Name Hiding）？与重写（Override）的区别？

- **函数隐藏**：子类定义与父类同名但参数不同的函数，或非虚函数同名同参数，父类函数被隐藏（不构成重写）。
- **区别**：
 - 重写：父类函数是虚函数，子类函数与父类同原型（`virtual`可省略）；

- 隐藏：父类函数非虚函数，或子类函数参数不同，调用时由指针/引用的类型决定（编译期绑定）。

24. 什么是菱形继承？如何解决菱形继承的二义性和数据冗余？

- **菱形继承**：子类间接继承同一基类多次（如D继承B和C，B和C均继承A），导致基类成员在子类中冗余且访问二义性。
- **解决方法**：虚继承（`virtual`）：

```
class A {};  
class B : virtual public A {}; // 虚继承A  
class C : virtual public A {}; // 虚继承A  
class D : public B, public C {}; // D中A的成员仅一份
```

- 原理：虚继承的子类通过虚基类表（vbptr）间接访问基类成员，避免数据冗余。

25. 组合（Composition）与继承（Inheritance）的区别？如何选择？

- **组合**：类中包含另一个类的对象（`class Car { Engine engine; };`），体现has-a关系。
- **继承**：子类继承父类（`class Car : public Vehicle`），体现is-a关系。
- **选择原则**：
 - 需复用接口且扩展功能→继承；
 - 需复用实现且低耦合→组合（优先，符合“多用组合，少用继承”原则）。

26. 什么是拷贝构造函数？何时会被调用？

- **拷贝构造函数**：用已存在的对象初始化新对象的构造函数（`A(const A& other);`）。
- **调用场景**：
 - 对象作为函数参数按值传递；
 - 对象作为函数返回值按值返回；
 - 显式用对象初始化新对象（`A b(a);` 或 `A b = a;`）。
- 注意：若类中存在指针成员，需手动实现拷贝构造函数（深拷贝），避免浅拷贝导致的内存问题。

27. 深拷贝与浅拷贝的区别？

- **浅拷贝**：默认拷贝构造函数/赋值运算符的行为，仅复制成员变量的值（指针变量复制地址，不复制指向的内容）。
- **深拷贝**：手动实现，复制指针指向的内容（重新分配内存），确保两个对象独立。
- 示例（深拷贝）：

```
class A {  
private:  
    int* data;  
public:  
    A(const A& other) { // 深拷贝构造  
        data = new int(*other.data);  
    }  
}
```

```
A& operator=(const A& other) { // 深拷贝赋值
    if (this != &other) {
        delete data;
        data = new int(*other.data);
    }
    return *this;
}
};
```

28. 赋值运算符重载的注意事项？

- 需检查自赋值 (`if (this == &other) return *this;`) ;
- 释放当前对象的资源 (避免内存泄漏) ;
- 返回自身引用 (`return *this;`) , 支持链式赋值 (`a = b = c;`) ;
- 若类中存在指针成员, 需实现深拷贝。

29. 构造函数、析构函数、拷贝构造函数、赋值运算符的调用顺序？

- 示例: `A a; A b(a); b = a;`
 1. `a`的构造函数;
 2. `b`的拷贝构造函数 (用`a`初始化) ;
 3. `b`的赋值运算符 (`b = a`) ;
 4. 程序结束: `b`的析构函数 → `a`的析构函数 (析构顺序与构造相反) 。

30. 什么是移动构造函数和移动赋值运算符？

- **移动构造**: `A(A&& other)`, 用右值 (将亡对象) 初始化新对象, 转移资源所有权 (而非拷贝) , 提高效率。
- **移动赋值**: `A& operator=(A&& other)`, 类似移动构造, 用于右值赋值。
- 示例:

```
class A {
private:
    int* data;
public:
    A(A&& other) : data(other.data) {
        other.data = nullptr; // 转移资源, 避免被析构释放
    }
};
```

- 触发场景: 用临时对象初始化或赋值 (如 `A a = createA();`, `createA()`返回临时对象) 。

31. 什么是explicit关键字? 作用是什么?

- **explicit**: 修饰单参数构造函数, 禁止隐式类型转换 (仅允许显式转换) 。
- 示例:

```
class A {
public:
    explicit A(int x) {} // 禁止隐式转换
};
A a = 5; // 错误 (隐式转换被禁止)
A b(5); // 正确 (显式构造)
```

32. 什么是友元 (friend) ? 使用场景和缺点?

- **友元**: 允许类外函数或其他类访问当前类的private/protected成员。
- **场景**:
 - 运算符重载 (如`operator<<`需访问类成员) ;
 - 工具类需要访问目标类的内部数据。
- **缺点**: 破坏封装性, 增加耦合度, 谨慎使用。

33. 什么是this指针? 用途是什么?

- **this指针**: 隐含在非static成员函数中的指针, 指向当前对象 (`this`的类型为`ClassName* const`) 。
- **用途**:
 - 区分成员变量与函数参数 (`this->x = x;`) ;
 - 返回当前对象 (`return *this;`, 用于链式调用) ;
 - 在成员函数中传递当前对象地址。

34. mutable关键字的作用?

- **mutable**: 修饰类的成员变量, 允许在const成员函数中修改该变量。
- 示例:

```
class A {
private:
    mutable int count; // 可在const函数中修改
public:
    void func() const {
        count++; // 合法
    }
};
```

35. 什么是RTTI (运行时类型识别) ? 如何实现?

- **RTTI**: 在运行时获取对象的类型信息。
- **实现方式**:
 - `dynamic_cast`: 用于基类向派生类的安全转换;
 - `typeid`: 返回`type_info`对象, 可比较类型 (`typeid(*p) == typeid(Derived)`) 。
- 注意: 需类有虚函数 (启用vtable, 支持RTTI) 。

三、设计模式 (36-45题)

36. 什么是设计模式？设计模式的分类？

- **设计模式**：解决特定场景下软件设计问题的成熟方案，提高代码复用性、可维护性。
- **分类**：
 - 创建型：单例、工厂、抽象工厂、建造者、原型；
 - 结构型：适配器、装饰器、代理、外观、桥接、组合、享元；
 - 行为型：观察者、策略、模板方法、迭代器、状态、命令等。

37. 单例模式（Singleton）的实现？优缺点？

- **核心**：确保类只有一个实例，并提供全局访问点。
- **线程安全实现（懒汉式）**：

```
class Singleton {
private:
    static Singleton* instance;
    Singleton() = default; // 私有构造，禁止外部实例化
    ~Singleton() = default;
    Singleton(const Singleton&) = delete; // 禁止拷贝
    Singleton& operator=(const Singleton&) = delete; // 禁止赋值
public:
    static Singleton* getInstance() {
        static std::once_flag flag;
        std::call_once(flag, []() { // 线程安全初始化
            instance = new Singleton();
        });
        return instance;
    }
};
Singleton* Singleton::instance = nullptr;
```

- **优点**：控制实例数量，节省资源；
- **缺点**：耦合度高，测试困难，可能隐藏依赖关系。

38. 工厂模式（Factory）的种类和区别？

- **简单工厂**：一个工厂类根据参数创建不同产品（违反开放封闭原则，新增产品需修改工厂）。
- **工厂方法**：每个产品对应一个工厂子类（`ProductAFactory`创建`ProductA`），符合开放封闭原则。
- **抽象工厂**：创建一系列相关产品（如“华为工厂”生产“华为手机”和“华为平板”），支持产品族扩展。

39. 观察者模式（Observer）的作用？实现要点？

- **作用**：定义对象间的一对多依赖，当一个对象（主题）状态变化时，所有依赖它的对象（观察者）自动收到通知并更新。
- **实现要点**：
 - 主题（Subject）：维护观察者列表，提供添加、删除观察者及通知方法；
 - 观察者（Observer）：定义更新接口，由具体观察者实现。
- **应用**：GUI事件监听、消息订阅系统。

40. 装饰器模式 (Decorator) 的作用? 与继承的区别?

- **作用**: 动态给对象添加额外功能, 不改变其接口 (如给咖啡加奶、加糖)。
- **实现**: 装饰器类与被装饰类实现同一接口, 装饰器持有被装饰对象的引用, 在调用原方法前后添加新逻辑。
- **与继承的区别**: 装饰器是动态组合, 灵活扩展 (运行时添加功能); 继承是静态扩展 (编译期确定)。

41. 代理模式 (Proxy) 的作用? 常见类型?

- **作用**: 为对象提供代理 (替身), 控制对原对象的访问。
- **常见类型**:
 - 远程代理: 代表远程对象 (如RPC代理);
 - 虚拟代理: 延迟加载 heavy 对象 (如大图加载前显示占位符);
 - 安全代理: 控制访问权限。

42. 策略模式 (Strategy) 的作用?

- **作用**: 定义算法家族, 封装每个算法, 使它们可互换 (如排序算法可切换为冒泡、快速排序)。
- **优点**: 避免多重条件判断 (`if-else/switch`), 符合开放封闭原则。

43. 模板方法模式 (Template Method) 的作用?

- **作用**: 在基类中定义算法骨架, 将可变步骤延迟到子类实现 (如框架中的生命周期方法)。
- **示例**:

```
class AbstractClass {
public:
    void templateMethod() { // 算法骨架
        step1();
        step2(); // 子类实现
        step3();
    }
protected:
    void step1() { ... }
    virtual void step2() = 0; // 纯虚函数, 子类实现
    void step3() { ... }
};
```

44. 迭代器模式 (Iterator) 的作用? STL中如何体现?

- **作用**: 提供一种方法遍历容器元素, 无需暴露容器内部结构。
- **STL体现**: 所有容器均提供`begin()/end()`方法返回迭代器, 通过`*it`访问元素, `++it`移动, 统一遍历接口 (如`for (auto it = vec.begin(); it != vec.end(); ++it)`)。

45. 单例模式中如何防止对象被拷贝或赋值?

- 将拷贝构造函数和赋值运算符声明为私有且不实现, 或C++11后用`delete`删除:

```
class Singleton {
private:
    Singleton(const Singleton&) = delete; // 禁止拷贝
    Singleton& operator=(const Singleton&) = delete; // 禁止赋值
};
```

四、模板编程 (46-55题)

46. 函数模板与类模板的定义方式?

- 函数模板:

```
template <typename T> // 或 class T
T max(T a, T b) {
    return a > b ? a : b;
}
// 调用: max(3, 5) → int版, max(3.1, 2.5) → double版
```

- 类模板:

```
template <typename T, int N>
class Array {
private:
    T data[N];
public:
    T& operator[](int i) { return data[i]; }
};
// 实例化: Array<int, 10> arr;
```

47. 模板特化 (Specialization) 的作用?

- 作用: 为特定类型提供模板的自定义实现 (覆盖通用模板)。
- 函数模板特化:

```
template <typename T>
void print(T value) { cout << value; }

template <> // 特化const char*
void print<const char*>(const char* value) {
    cout << "String: " << value;
}
```

- 类模板特化:

```
template <typename T>
class Stack { ... };

template <> // 特化Stack<bool>
class Stack<bool> { ... }; // 优化bool存储 (用bit)
```

48. 模板偏特化 (Partial Specialization) 的作用?

- **作用：**对模板参数的部分条件进行特化（不能用于函数模板，仅类模板）。
- 示例：

```
template <typename T, typename U>
class Pair { ... };

template <typename T> // 偏特化：第二个参数为int
class Pair<T, int> { ... };

template <typename T, typename U> // 偏特化：两个参数均为指针
class Pair<T*, U*> { ... };
```

49. 什么是模板元编程 (Template Metaprogramming) ?

- **模板元编程：**以模板为工具，在编译期执行计算的编程范式（代码运行在编译阶段）。
- 示例（编译期计算阶乘）：

```
template <int N>
struct Factorial {
    static const int value = N * Factorial<N-1>::value;
};

template <> // 特化终止递归
struct Factorial<0> {
    static const int value = 1;
};

// 编译期计算：Factorial<5>::value → 120
```

50. 什么是SFINAE (Substitution Failure Is Not An Error) ?

- **SFINAE：**模板参数替换失败时，编译器不报错，而是尝试其他可行的重载版本。
- 应用：条件编译（根据类型特性选择不同实现），如判断类型是否有`begin()`方法：

```
#include <type_traits>

template <typename T>
```

```
auto has_begin(T&& t) -> decltype(t.begin(), std::true_type{}) {  
    return std::true_type{};  
}  
  
std::false_type has_begin(...) {  
    return std::false_type{};  
}
```

51. 模板参数的类型有哪些？

- **类型参数**： `typename T` 或 `class T`（表示类型）；
- **非类型参数**： `int N`、`double D`（表示常量值，需编译期已知）；
- **模板模板参数**：模板作为参数（`template <typename T> class Container`）。

52. 如何使用模板实现泛型算法？

- 示例（泛型交换函数）：

```
template <typename T>  
void swap(T& a, T& b) {  
    T temp = std::move(a); // 利用移动语义提高效率  
    a = std::move(b);  
    b = std::move(temp);  
}
```

- 特点：与容器无关，通过迭代器访问元素（如STL的`std::sort`可排序任何提供随机访问迭代器的容器）。

53. 模板的优缺点？

- **优点**：代码复用（一份代码支持多种类型），泛型编程，编译期类型检查，可实现模板元编程（编译期计算）。
- **缺点**：编译时间长（模板实例化增加编译负担），错误信息复杂（模板展开错误难以定位），二进制膨胀（不同类型实例化生成多份代码）。

54. 什么是可变参数模板（Variadic Templates）？

- **可变参数模板**：接受任意数量、任意类型参数的模板（C++11引入）。
- 示例（递归打印参数）：

```
void print() {} // 终止递归  
  
template <typename T, typename... Args>  
void print(T first, Args... rest) {  
    cout << first << " ";  
    print(rest...); // 递归展开参数包  
}
```

```
// 调用: print(1, "hello", 3.14); → 输出 "1 hello 3.14 "
```

55. 如何限制模板只能用于特定类型?

- **方法1:** 静态断言 (`static_assert`) + 类型特性:

```
#include <type_traits>

template <typename T>
class MyClass {
    static_assert(std::is_integral_v<T>, "T must be integral type");
};
```

- **方法2:** SFINAE (通过返回类型或参数限制):

```
template <typename T, typename =
std::enable_if_t<std::is_floating_point_v<T>>>
void func(T value) { ... } // 仅接受浮点类型
```

五、STL (56-75题)

56. STL的六大组件是什么?

- **容器 (Containers)**: 存储数据 (如 `vector`、`list`、`map`) ;
- **算法 (Algorithms)**: 操作数据 (如 `sort`、`find`、`copy`) ;
- **迭代器 (Iterators)**: 连接容器与算法 (如 `begin()`、`end()`) ;
- **仿函数 (Functors)**: 行为类似函数的对象 (如 `std::less`) ;
- **适配器 (Adapters)**: 转换接口 (如 `stack`、`queue` 是容器适配器) ;
- **分配器 (Allocators)**: 内存管理 (如 `std::allocator`) 。

57. STL容器的分类?

- **序列容器**: 按顺序存储, 元素位置由插入顺序决定 (`vector`、`deque`、`list`、`array`、`forward_list`) ;
- **关联容器**: 按键 (key) 排序存储, 支持快速查找 (`set`、`map`、`multiset`、`multimap`) ;
- **无序关联容器** (C++11): 哈希表实现, 无序 (`unordered_set`、`unordered_map`等) ;
- **容器适配器**: 基于其他容器封装 (`stack`、`queue`、`priority_queue`) 。

58. vector的底层实现? 优缺点?

- **底层**: 动态数组 (连续内存), 维护指向数据的指针、大小 (size)、容量 (capacity) 。
- **扩容机制**: 容量不足时, 分配新内存 (通常为原容量的2倍), 拷贝旧数据, 释放旧内存。
- **优点**: 随机访问效率高 ($O(1)$), 尾部插入/删除效率高 ($O(1)$) ;
- **缺点**: 中间插入/删除效率低 ($O(n)$), 扩容有性能开销 (拷贝数据) 。

59. vector的reserve()与resize()的区别？

- **reserve(n)**：预分配至少n个元素的容量（不改变size，避免多次扩容）；
- **resize(n)**：调整容器大小为n（若n>size，新增元素默认初始化；若n<size，删除多余元素）。

60. list的底层实现？与vector的区别？

- **底层**：双向链表（非连续内存），每个节点含数据、prev指针、next指针。
- **与vector的区别**：
 - 内存：list非连续，vector连续；
 - 访问：list不支持随机访问（O(n)），vector支持（O(1)）；
 - 插入/删除：list中间操作高效（O(1)，只需修改指针），vector中间操作低效（O(n)）；
 - 迭代器：list迭代器在插入/删除后仍有效（除非指向被删除节点），vector扩容后迭代器失效。

61. deque的底层实现？适用场景？

- **底层**：分段连续内存（多个固定大小的缓冲区，通过中控器数组管理）。
- **特点**：支持首尾两端高效插入/删除（O(1)），随机访问效率接近vector（O(1)）。
- **适用场景**：需要频繁在首尾操作的场景（如双端队列、滑动窗口）。

62. map与unordered_map的区别？

特性	map	unordered_map
底层实现	红黑树（平衡二叉搜索树）	哈希表
有序性	按键升序排列	无序
查找效率	O(log n)	平均O(1)，最坏O(n)
插入/删除	O(log n)	平均O(1)，最坏O(n)
内存占用	较低（仅存节点）	较高（哈希表有负载因子）
键要求	需定义比较运算符（<）	需定义哈希函数和相等运算符（==）

63. set与multiset的区别？

- **set**：键唯一，插入重复键会失败（返回的pair中second为false）；
- **multiset**：键可重复，允许插入多个相同键。
- 底层均为红黑树，支持按键排序和范围查找。

64. 什么是迭代器失效？哪些操作会导致迭代器失效？

- **迭代器失效**：迭代器指向的元素被删除或内存位置改变，继续使用会导致未定义行为。
- **导致失效的操作**：
 - vector：扩容（所有迭代器失效）、插入/删除元素（失效点及之后迭代器失效）；
 - list：删除元素（仅指向该元素的迭代器失效），插入元素（迭代器均有效）；
 - map/unordered_map：删除元素（仅指向该元素的迭代器失效），插入元素（map迭代器有效，unordered_map可能因rehash失效）。

65. STL迭代器的分类？

- **输入迭代器**：只读，单向移动（如`istream_iterator`）；
- **输出迭代器**：只写，单向移动（如`ostream_iterator`）；
- **前向迭代器**：可读写，单向移动（如`forward_list`的迭代器）；
- **双向迭代器**：可读写，双向移动（如`list`、`map`的迭代器）；
- **随机访问迭代器**：可读写，随机访问（如`vector`、`deque`的迭代器，支持`+n`、`-n`）。

66. STL算法的分类？常用算法有哪些？

- **分类**：
 - 非修改型算法（如`find`、`count`、`for_each`）；
 - 修改型算法（如`copy`、`transform`、`replace`）；
 - 排序算法（如`sort`、`stable_sort`、`partial_sort`）；
 - 集合算法（如`set_union`、`set_intersection`）。
- **常用算法**：
 - `std::sort`：排序（需随机访问迭代器）；
 - `std::find`：查找元素；
 - `std::for_each`：遍历元素并应用函数；
 - `std::transform`：转换元素；
 - `std::accumulate`：累加元素（需包含`<numeric>`）。

67. 如何自定义STL算法的比较函数？

- 方法1：函数指针：

```
bool greater(int a, int b) { return a > b; }
std::sort(vec.begin(), vec.end(), greater); // 降序排序
```

- 方法2：仿函数（functor）：

```
struct Greater {
    bool operator()(int a, int b) { return a > b; }
};
std::sort(vec.begin(), vec.end(), Greater());
```

- 方法3：lambda表达式（C++11）：

```
std::sort(vec.begin(), vec.end(), [](int a, int b) { return a > b; });
```

68. stack和queue的底层容器是什么？如何指定？

- **stack**：默认底层容器是`deque`，可指定为`vector`或`list`：

```
std::stack<int, std::vector<int>> st; // 用vector实现stack
```

- **queue**: 默认底层容器是`deque`, 可指定为`list` (不可用`vector`, 因`vector`无`pop_front`高效实现):

```
std::queue<int, std::list<int>> q; // 用list实现queue
```

69. priority_queue的底层实现? 如何改变优先级?

- **底层**: 默认用`vector`作为底层容器, 维护一个最大堆 (每次弹出最大元素)。
- **改变优先级**: 指定仿函数为`std::greater`实现最小堆:

```
std::priority_queue<int, std::vector<int>, std::greater<int>> min_heap;
```

70. 什么是适配器 (Adapter)? STL中有哪些适配器?

- **适配器**: 转换已有组件的接口, 使其符合新的需求 (不改变功能)。
- **STL适配器**:
 - 容器适配器: `stack` (适配为栈)、`queue` (适配为队列)、`priority_queue` (适配为优先队列);
 - 迭代器适配器: `reverse_iterator` (反向迭代器)、`insert_iterator` (插入迭代器);
 - 仿函数适配器: `not1` (取反一元仿函数)、`bind1st` (绑定第一个参数)。

71. 什么是仿函数 (Functor)? 与函数指针的区别?

- **仿函数**: 重载`operator()`的类/结构体, 实例化后可像函数一样调用。
- **与函数指针的区别**:
 - 仿函数可携带状态 (成员变量), 函数指针不能;
 - 仿函数是编译期确定的, 可内联优化, 函数指针调用通常不能内联;
 - 仿函数支持模板特化, 更灵活。

72. STL中的空间配置器 (Allocator) 的作用?

- **作用**: 负责容器的内存分配与释放 (封装`new/delete`), 隔离内存管理细节。
- **标准配置器**: `std::allocator`, 提供`allocate(n)` (分配n个元素内存) 和`deallocate(p, n)` (释放内存) 接口。
- 自定义配置器可优化特定场景 (如内存池减少碎片)。

73. 如何遍历STL容器? 有哪些方法?

- 方法1: 迭代器:

```
for (auto it = vec.begin(); it != vec.end(); ++it) { ... }
```

- 方法2：范围for循环（C++11）：

```
for (const auto& elem : vec) { ... }
```

- 方法3：算法for_each：

```
std::for_each(vec.begin(), vec.end(), [](int x) { cout << x; });
```

74. 什么是区间（Range）？STL中如何表示区间？

- **区间**：由两个迭代器定义的元素范围，`[begin, end)`（包含begin指向的元素，不包含end指向的元素）。
- 优点：统一容器的访问方式，算法只需接收区间即可操作，无需关心具体容器类型。

75. STL容器的线程安全性？

- **STL容器本身不是线程安全的**：
 - 多线程读：安全；
 - 单线程写，其他线程读/写：不安全（可能导致迭代器失效、数据竞争）。
- **保证线程安全的方法**：手动加锁（如`std::mutex`）保护容器访问。

六、Boost库与网络编程（76-85题）

76. Boost库的作用？常用组件有哪些？

- **作用**：提供高质量的C++库，补充STL不足，许多组件被纳入C++标准（如`shared_ptr`来自`boost::shared_ptr`）。
- **常用组件**：
 - 智能指针：`boost::shared_ptr`、`boost::weak_ptr`；
 - 网络：`boost::asio`（异步IO，网络编程）；
 - 字符串处理：`boost::algorithm`；
 - 容器：`boost::array`、`boost::unordered_map`（C++11前的无序容器）；
 - 其他：`boost::thread`（线程库）、`boost::function`（函数对象）。

77. boost::asio的核心概念？如何实现异步操作？

- **核心概念**：`io_context`（IO服务上下文）、`socket`（套接字）、`endpoint`（端点）、`strand`（保证异步操作顺序执行）。
- **异步操作实现**：通过回调函数（或`std::future`），操作完成后由`io_context`调用回调：

```
#include <boost/asio.hpp>
using boost::asio::ip::tcp;

void handle_read(const boost::system::error_code& ec, std::size_t len) {
```

```

        if (!ec) { /* 处理数据 */ }
    }

    int main() {
        boost::asio::io_context io;
        tcp::socket sock(io);
        char buf[1024];
        sock.async_read_some(boost::asio::buffer(buf), handle_read); // 异步读
        io.run(); // 运行IO服务，等待异步操作完成
    }

```

78. boost::shared_ptr与std::shared_ptr的区别？

- 两者功能基本一致（共享所有权的智能指针），`std::shared_ptr`是C++11采纳自`boost::shared_ptr`。
- 细微区别：`boost::shared_ptr`支持不完全类型的删除器，可用于pimpl模式更灵活。

79. 什么是Pimpl模式（Pointer to Implementation）？

- **Pimpl**：将类的实现细节（成员变量、私有函数）封装在一个独立的实现类中，对外只暴露一个指向实现类的指针。
- 优点：降低编译依赖（头文件无需包含实现细节的头文件），隐藏实现，二进制兼容。
- 示例：

```

// MyClass.h
#include <memory>
class MyClass {
private:
    class Impl; // 前向声明
    std::unique_ptr<Impl> pimpl; // 指向实现
public:
    MyClass();
    void func();
};

// MyClass.cpp
class MyClass::Impl {
public:
    int data; // 实现细节
    void func_impl() { ... }
};

MyClass::MyClass() : pimpl(std::make_unique<Impl>()) {}
void MyClass::func() { pimpl->func_impl(); }

```

80. C++网络编程中如何处理粘包问题？

- **粘包**：TCP字节流中，多次发送的数据被合并或拆分（见Linux网络编程部分）。

- **解决方法**（C++实现）：
 - 固定包长：读取固定字节数；
 - 包头+包体：包头包含包长（如4字节整数），先读包头再读对应长度的包体；
 - 分隔符：如\r\n，读取到分隔符为止。

81. 如何用C++实现一个简单的TCP服务器？

- 步骤：
 1. 创建socket (`socket(AF_INET, SOCK_STREAM, 0)`) ；
 2. 绑定地址端口 (`bind()`) ；
 3. 监听连接 (`listen()`) ；
 4. 接受连接 (`accept()`) ；
 5. 读写数据 (`recv()/send()`) ；
 6. 关闭连接 (`close()`) 。
- 示例代码：

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <cstring>
#include <iostream>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);
    bind(server_fd, (sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 5);

    int client_fd = accept(server_fd, nullptr, nullptr);
    char buf[1024];
    recv(client_fd, buf, sizeof(buf), 0);
    std::cout << "Received: " << buf << std::endl;

    close(client_fd);
    close(server_fd);
    return 0;
}
```

82. 同步IO与异步IO在C++网络编程中的区别？

- **同步IO**：调用`recv()/send()`后，进程阻塞直到操作完成（或超时）。
- **异步IO**：调用`async_read()/async_write()`后立即返回，操作完成后通过回调通知（如`boost::asio`）。
- 应用：同步IO实现简单，适合连接少的场景；异步IO效率高，适合高并发（如服务器）。

83. 什么是IOCP (Windows) 和epoll (Linux) ？ C++中如何使用？

- **IOCP (Input Output Completion Port)** : Windows的异步IO模型, 高效处理大量并发IO。
- **epoll**: Linux的IO复用模型 (见Linux网络编程部分)。
- **C++使用**: 直接调用系统API (`epoll_create/epoll_ctl/epoll_wait`), 或通过封装库 (如 `boost::asio` 自动适配不同平台的高效IO模型)。

84. 如何在C++中实现超时控制?

- **方法1**: `setsockopt`设置`SO_RCVTIMEO/SO_SNDTIMEO`:

```
struct timeval timeout {3, 0}; // 3秒超时
setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));
```

- **方法2**: `select/epoll`设置超时参数:

```
struct timeval tv {3, 0};
fd_set readfds;
FD_SET(sock, &readfds);
select(sock+1, &readfds, nullptr, nullptr, &tv); // 超时返回0
```

85. 什么是线程池? C++中如何实现简单的线程池?

- **线程池**: 预先创建多个线程, 从任务队列中获取任务执行, 避免频繁创建线程的开销。
- **实现要点**:
 - 任务队列 (`std::queue`, 需互斥锁保护);
 - 工作线程 (循环从队列取任务执行);
 - 同步机制 (`std::mutex`互斥锁, `std::condition_variable`条件变量)。
- 核心代码:

```
#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>

class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex mtx;
    std::condition_variable cv;
    bool stop;
public:
    ThreadPool(size_t num_threads) : stop(false) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
```

```

        {
            std::unique_lock<std::mutex> lock(mtx);
            cv.wait(lock, [this] { return stop ||
!tasks.empty(); });
            if (stop && tasks.empty()) return;
            task = std::move(tasks.front());
            tasks.pop();
        }
        task();
    }
    });
}
}
~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(mtx);
        stop = true;
    }
    cv.notify_all();
    for (auto& w : workers) w.join();
}
template <typename F>
void enqueue(F&& f) {
    {
        std::unique_lock<std::mutex> lock(mtx);
        tasks.emplace(std::forward<F>(f));
    }
    cv.notify_one();
}
};

```

七、C++11及以后新特性 (86-100题)

86. C++11的智能指针有哪些？各自的用途？

- **unique_ptr**：独占所有权，不可复制（可移动），适合管理单个对象（`std::unique_ptr<int> p = std::make_unique<int>(5);`）。
- **shared_ptr**：共享所有权，通过引用计数管理生命周期，最后一个持有者释放内存（`std::shared_ptr<int> p = std::make_shared<int>(5);`）。
- **weak_ptr**：弱引用，不增加**shared_ptr**的引用计数，用于解决**shared_ptr**的循环引用问题（`std::weak_ptr<int> wp = p;`）。

87. 为什么优先使用**make_shared**而不是直接**new**？

- **make_shared**优点：
 - 一次内存分配（同时分配对象和引用计数），`new+shared_ptr`需两次分配；
 - 避免内存泄漏风险（如**`f(shared_ptr<int>(new int), g())`**中，若**`g()`**抛出异常，`new int`可能未被释放）；
 - 代码更简洁（`auto p = make_shared<int>(5);`）。

88. 什么是右值引用 (Rvalue Reference) ? 作用是什么?

- **右值引用**: 用`&&`表示, 绑定到右值 (临时对象、将亡值)。
- **作用**:
 - 实现移动语义 (转移资源, 避免拷贝);
 - 实现完美转发 (保持参数的左值/右值属性)。
- 示例: `int&& r = 5;` (绑定到字面量右值)。

89. 移动语义 (Move Semantics) 的作用? 与拷贝语义的区别?

- **移动语义**: 将右值对象的资源 (如堆内存) 转移给新对象, 原对象变为空状态 (不分配新资源)。
- **与拷贝语义的区别**: 拷贝语义复制资源 (深拷贝, 开销大), 移动语义转移资源 (开销小)。
- 应用: `vector` 扩容时移动元素, `unique_ptr` 的转移 (`u2 = std::move(u1);`)。

90. 什么是完美转发 (Perfect Forwarding) ? 如何实现?

- **完美转发**: 在模板函数中, 将参数原封不动地转发给其他函数 (保持左值/右值、`const`属性)。
- **实现**: 通过`std::forward`结合右值引用:

```
template <typename T>
void forward_func(T&& arg) {
    other_func(std::forward<T>(arg)); // 完美转发
}
```

- 作用: 减少函数重载, 简化模板代码 (如工厂函数创建对象时转发参数)。

91. lambda表达式的语法和作用?

- **语法**: `[捕获列表](参数列表) mutable noexcept -> 返回类型 { 函数体 }`
- **捕获列表**:
 - `[]`: 不捕获;
 - `[=]`: 按值捕获所有外部变量;
 - `[&]`: 按引用捕获所有外部变量;
 - `[x, &y]`: 按值捕获x, 按引用捕获y。
- **作用**: 定义匿名函数, 简化代码 (如作为算法的回调函数)。
- 示例: `auto add = [](int a, int b) { return a + b; };`

92. auto关键字的作用? 使用场景?

- **作用**: 自动推导变量类型 (编译期确定)。
- **场景**:
 - 简化长类型声明 (如`auto it = vec.begin();`);
 - 存储lambda表达式 (lambda类型匿名, 必须用`auto`);
 - 避免类型拼写错误。
- 注意: `auto`不能用于函数参数、数组类型推导。

93. decltype关键字的作用? 与auto的区别?

- **作用：**推导表达式的类型（不执行表达式）。
- **示例：**

```
int a = 5;
decltype(a) b = 10; // b的类型为int
decltype(a + 3.14) c = 2.5; // c的类型为double
```

- **与auto的区别：**auto推导变量类型时需初始化，decltype仅推导表达式类型，无需初始化；auto会忽略引用和const，decltype保留。

94. constexpr关键字的作用？

- **作用：**声明编译期常量或函数（可在编译期计算）。
- **示例：**

```
constexpr int square(int x) { return x * x; }
constexpr int num = square(5); // 编译期计算为25
```

- **优势：**提高性能（编译期计算），增强类型安全，可用于模板参数。

95. 范围for循环（Range-based for Loop）的用法？

- **语法：**for（元素声明：容器）{ 循环体 }
- **示例：**

```
std::vector<int> vec = {1, 2, 3};
for (int x : vec) { std::cout << x; } // 只读
for (int& x : vec) { x *= 2; } // 修改元素
```

- **要求：**容器需提供begin()和end()方法，返回迭代器。

96. C++11的线程库（std::thread）的基本用法？

- **创建线程：**std::thread t(func, args...);
- **等待线程结束：**t.join();
- **分离线程：**t.detach();（线程后台运行，不阻塞当前线程）
- **获取线程ID：**t.get_id();
- **示例：**

```
#include <thread>
void func(int x) { std::cout << x; }
int main() {
    std::thread t(func, 5);
    t.join(); // 等待t执行完毕
}
```

```
    return 0;
}
```

97. C++11的互斥锁 (std::mutex) 和条件变量 (std::condition_variable) 的用法?

- **互斥锁**：保护共享资源，std::lock_guard (自动加锁解锁)：

```
std::mutex mtx;
int shared_data = 0;

void increment() {
    std::lock_guard<std::mutex> lock(mtx); // 构造加锁，析构解锁
    shared_data++;
}
```

- **条件变量**：线程同步 (等待条件成立)：

```
std::condition_variable cv;
bool ready = false;

void wait_func() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return ready; }); // 等待ready为true
}

void notify_func() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }
    cv.notify_one(); // 唤醒一个等待线程
}
```

98. C++17的结构化绑定 (Structured Binding) 的作用?

- **作用**：同时声明多个变量，绑定到结构体、数组、pair等的成员。
- **示例**：

```
std::pair<int, std::string> p = {1, "hello"};
auto [id, name] = p; // id=1, name="hello"

int arr[] = {10, 20};
auto [a, b] = arr; // a=10, b=20
```

99. C++20的概念 (Concepts) 的作用?

- **作用：**限制模板参数的类型，使错误信息更清晰，替代SFINAE的复杂用法。
- **示例：**

```
template <typename T>
concept Integral = std::is_integral_v<T>;

template <Integral T> // 仅接受整数类型
T add(T a, T b) { return a + b; }
```

100. C++11到C++20的重要新特性总结？

- **C++11：**智能指针、右值引用、移动语义、lambda、auto、decltype、线程库、范围for；
- **C++14：**泛型lambda、返回类型推导、make_unique；
- **C++17：**结构化绑定、if constexpr、折叠表达式、文件系统库；
- **C++20：**概念（Concepts）、协程（Coroutines）、模块（Modules）、范围库（Ranges）。

经典算法

一、算法基础与概念（1-10题）

1. 什么是时间复杂度和空间复杂度？如何分析？

- **时间复杂度：**描述算法执行时间随输入规模增长的变化趋势，通常用大O符号表示。
- **空间复杂度：**描述算法所需存储空间随输入规模增长的变化趋势。
- **分析方法：**
 - 时间复杂度：关注循环嵌套层数和递归深度，忽略常数项和低阶项。
 - 空间复杂度：考虑变量、数组、栈递归等占用的空间。
- 示例：for(i=0;i<n;i++) for(j=0;j<n;j++) 时间复杂度为 $O(n^2)$ 。

2. 什么是递归？递归的优缺点是什么？

- **递归：**函数调用自身解决问题的方法，通常包含基线条件（终止条件）和递归条件。
- **优点：**代码简洁，问题描述直观（如树、图的遍历）。
- **缺点：**可能导致栈溢出，时间和空间开销较大，存在重复计算风险。
- 示例：斐波那契数列递归实现

```
int fib(int n) {
    if (n <= 1) return n; // 基线条件
    return fib(n-1) + fib(n-2); // 递归条件
}
```

3. 什么是动态规划？它与分治法的区别是什么？

- **动态规划：**将复杂问题分解为重叠子问题，通过存储子问题解（记忆化）避免重复计算。
- **与分治法区别：**

- 分治法适用于子问题独立的场景（如归并排序）。
- 动态规划适用于子问题重叠的场景，通过缓存中间结果优化效率。
- 核心思想：最优子结构和重叠子问题。

4. 什么是贪心算法？它的适用场景是什么？

- **贪心算法**：在每一步选择中都采取当前状态下最优的选择，希望导致全局最优。
- **适用场景**：问题具有贪心选择性质（局部最优导致全局最优）和最优子结构，如：
 - 哈夫曼编码
 - 活动选择问题
 - 最小生成树（Kruskal、Prim算法）
- 局限性：不能保证所有问题都得到最优解（如背包问题）。

5. 什么是回溯法？它的基本思想是什么？

- **回溯法**：一种通过探索所有可能的候选解来找出所有解的算法，若候选解被确认不是可行解，则回溯并尝试其他候选解。
- **基本思想**：深度优先搜索 + 剪枝，常用于解决组合优化问题。
- 应用：N皇后问题、子集和问题、全排列等。

6. 什么是哈希表？哈希冲突如何解决？

- **哈希表**：通过哈希函数将键映射到存储位置的数据结构，平均查找时间复杂度为 $O(1)$ 。
- **哈希冲突**：不同的键通过哈希函数得到相同的存储位置。
- **解决方法**：
 - 开放定址法：线性探测、二次探测、双重哈希。
 - 链地址法：将哈希值相同的元素存储在同一个链表中。
 - 再哈希法：使用多个哈希函数。
 - 建立公共溢出区：将冲突元素放入另一个表中。

7. 什么是二叉查找树（BST）？它的时间复杂度是多少？

- **BST**：一种二叉树，左子树所有节点值小于根节点值，右子树所有节点值大于根节点值，左右子树也为BST。
- **时间复杂度**：
 - 平均情况：查找、插入、删除均为 $O(\log n)$ 。
 - 最坏情况（退化为链表）： $O(n)$ 。
- 应用：索引、映射表等。

8. 什么是平衡二叉树？常见的平衡二叉树有哪些？

- **平衡二叉树**：左右子树高度差不超过1的二叉树，避免BST在最坏情况下退化为链表。
- **常见类型**：
 - AVL树：左右子树高度差不超过1，通过旋转保持平衡。
 - 红黑树：通过颜色规则（红节点的子节点为黑节点等）维持平衡，旋转操作较少，实际应用更广泛（如C++ STL中的map、set）。

9. 什么是堆？最大堆和最小堆有什么区别？

- **堆**：一种完全二叉树，每个节点的值大于等于（或小于等于）其孩子节点的值。
- **最大堆**：父节点值 $\geq$ 子节点值，根节点为最大值。
- **最小堆**：父节点值 $\leq$ 子节点值，根节点为最小值。
- 应用：优先队列、堆排序、Top K问题。

10. 什么是图的DFS和BFS？它们的时间复杂度是多少？

- **DFS（深度优先搜索）**：从起点出发，尽可能深入搜索，无法继续时回溯。
- **BFS（广度优先搜索）**：从起点出发，逐层扩展搜索，使用队列实现。
- **时间复杂度**：均为 $O(V + E)$ ，其中V是顶点数，E是边数。
- 应用：
 - DFS：拓扑排序、连通分量、解决迷宫问题。
 - BFS：最短路径（无权图）、层次遍历。

二、字符串处理（11-20题）

11. 实现strlen函数

```
size_t my_strlen(const char* str) {  
    if (str == nullptr) return 0;  
    size_t len = 0;  
    while (*str++ != '\0') {  
        len++;  
    }  
    return len;  
}
```

- 解析：遍历字符串直到遇到'\0'，计数字符个数，注意空指针判断。

12. 实现strcpy函数

```
char* my_strcpy(char* dest, const char* src) {  
    if (dest == nullptr || src == nullptr) return nullptr;  
    char* ret = dest;  
    while ((*dest++ = *src++) != '\0');  
    return ret;  
}
```

- 解析：将源字符串复制到目标字符串，包括'\0'，返回目标字符串起始地址。

13. 实现strcmp函数

```
int my_strcmp(const char* str1, const char* str2) {  
    while (*str1 && *str2 && *str1 == *str2) {  
        str1++;  
    }
```

```
        str2++;
    }
    return *str1 - *str2;
}
```

- 解析：逐个字符比较，返回第一个不同字符的ASCII码差值，相等返回0。

14. 字符串反转

```
void reverse_string(char* str) {
    if (str == nullptr) return;
    char* left = str;
    char* right = str + strlen(str) - 1;
    while (left < right) {
        swap(*left++, *right--);
    }
}
```

- 解析：双指针法，首尾交换字符，向中间移动直到相遇。

15. 判断字符串是否为回文

```
bool is_palindrome(const string& s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] != s[right]) return false;
        left++;
        right--;
    }
    return true;
}
```

- 解析：双指针从两端向中间比较，若所有对应字符相等则为回文。

16. 字符串匹配 (KMP算法)

```
vector<int> compute_lps(const string& pattern) {
    int m = pattern.size();
    vector<int> lps(m, 0);
    int len = 0; // 最长前缀后缀长度
    for (int i = 1; i < m; ) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
```

```

        if (len != 0) {
            len = lps[len - 1];
        } else {
            lps[i] = 0;
            i++;
        }
    }
}
return lps;
}

int kmp_search(const string& text, const string& pattern) {
    int n = text.size(), m = pattern.size();
    if (m == 0) return 0;
    vector<int> lps = compute_lps(pattern);
    int i = 0, j = 0; // i: text索引, j: pattern索引
    while (i < n) {
        if (text[i] == pattern[j]) {
            i++;
            j++;
            if (j == m) return i - j; // 找到匹配
        } else {
            if (j != 0) {
                j = lps[j - 1]; // 利用最长前缀后缀避免重复比较
            } else {
                i++;
            }
        }
    }
    return -1; // 未找到
}

```

- 解析：KMP算法通过预处理模式串得到最长前缀后缀数组（LPS），在匹配失败时避免回溯主串，时间复杂度 $O(n + m)$ 。

17. 最长公共前缀

```

string longest_common_prefix(vector<string>& strs) {
    if (strs.empty()) return "";
    string prefix = strs[0];
    for (int i = 1; i < strs.size(); i++) {
        int j = 0;
        while (j < prefix.size() && j < strs[i].size() && prefix[j] == strs[i][j])
        {
            j++;
        }
        prefix = prefix.substr(0, j);
        if (prefix.empty()) break;
    }
    return prefix;
}

```

- 解析：以第一个字符串为基准，逐个与其他字符串比较，更新公共前缀。

18. 字符串的排列

```
void backtrack(string& s, vector<string>& res, int start) {
    if (start == s.size()) {
        res.push_back(s);
        return;
    }
    unordered_set<char> used;
    for (int i = start; i < s.size(); i++) {
        if (used.count(s[i])) continue; // 去重
        used.insert(s[i]);
        swap(s[start], s[i]);
        backtrack(s, res, start + 1);
        swap(s[start], s[i]); // 回溯
    }
}

vector<string> permutation(string s) {
    vector<string> res;
    backtrack(s, res, 0);
    return res;
}
```

- 解析：回溯法生成全排列，使用哈希集合避免重复元素导致的相同排列。

19. 字符串转换整数 (atoi)

```
int myAtoi(string s) {
    int i = 0, n = s.size();
    // 跳过空格
    while (i < n && s[i] == ' ') i++;
    if (i == n) return 0;

    // 判断符号
    int sign = 1;
    if (s[i] == '+' || s[i] == '-') {
        sign = (s[i] == '+') ? 1 : -1;
        i++;
    }

    // 转换数字
    long long res = 0;
    while (i < n && isdigit(s[i])) {
        res = res * 10 + (s[i] - '0');
        // 处理溢出
        if (res > INT_MAX) {
```

```

        return (sign == 1) ? INT_MAX : INT_MIN;
    }
    i++;
}
return sign * res;
}

```

- 解析：处理空格、符号、数字转换及溢出情况，按照题目要求实现类似atoi的功能。

20. 最长回文子串

```

string longestPalindrome(string s) {
    if (s.empty()) return "";
    int start = 0, end = 0;
    for (int i = 0; i < s.size(); i++) {
        // 奇数长度回文
        int len1 = expandAroundCenter(s, i, i);
        // 偶数长度回文
        int len2 = expandAroundCenter(s, i, i + 1);
        int len = max(len1, len2);
        if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substr(start, end - start + 1);
}

// 辅助函数：从中心向两边扩展
int expandAroundCenter(string& s, int left, int right) {
    while (left >= 0 && right < s.size() && s[left] == s[right]) {
        left--;
        right++;
    }
    return right - left - 1;
}

```

- 解析：中心扩展法，对每个字符（奇数长度）和每对相邻字符（偶数长度）向两边扩展，寻找最长回文子串，时间复杂度 $O(n^2)$ 。

三、数组与矩阵 (21-35题)

21. 数组中的第K个最大元素

```

int findKthLargest(vector<int>& nums, int k) {
    // 使用最小堆，堆大小为k
    priority_queue<int, vector<int>, greater<int>> min_heap;
    for (int num : nums) {

```

```
        min_heap.push(num);
        if (min_heap.size() > k) {
            min_heap.pop();
        }
    }
    return min_heap.top();
}
```

- 解析：使用最小堆维护前k个最大元素，堆顶即为第k个最大元素，时间复杂度 $O(n \log k)$ 。

22. 合并两个有序数组

```
void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    // 从后往前合并，避免覆盖nums1中的元素
    while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j]) {
            nums1[k--] = nums1[i--];
        } else {
            nums1[k--] = nums2[j--];
        }
    }
    // 处理nums2剩余元素
    while (j >= 0) {
        nums1[k--] = nums2[j--];
    }
}
```

- 解析：双指针从尾部开始比较，将较大元素放入nums1的末尾，时间复杂度 $O(m + n)$ 。

23. 两数之和

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> num_map;
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (num_map.find(complement) != num_map.end()) {
            return {num_map[complement], i};
        }
        num_map[nums[i]] = i;
    }
    return {}; // 题目保证有解，可省略
}
```

- 解析：哈希表存储已遍历元素及其索引，对每个元素查找 $target - num$ 是否存在，时间复杂度 $O(n)$ 。

24. 三数之和

```
vector<vector<int>> threeSum(vector<int>& nums) {
    vector<vector<int>> res;
    sort(nums.begin(), nums.end());
    int n = nums.size();
    for (int i = 0; i < n - 2; i++) {
        // 跳过重复元素
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int left = i + 1, right = n - 1;
        int target = -nums[i];
        while (left < right) {
            int sum = nums[left] + nums[right];
            if (sum == target) {
                res.push_back({nums[i], nums[left], nums[right]});
                // 跳过重复元素
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++;
                right--;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
    }
    return res;
}
```

- 解析：排序后固定一个元素，双指针寻找另外两个元素，注意去重，时间复杂度 $O(n^2)$ 。

25. 数组中出现次数超过一半的数字

```
int majorityElement(vector<int>& nums) {
    // 摩尔投票法
    int candidate = nums[0];
    int count = 1;
    for (int i = 1; i < nums.size(); i++) {
        if (nums[i] == candidate) {
            count++;
        } else {
            count--;
            if (count == 0) {
                candidate = nums[i];
                count = 1;
            }
        }
    }
    return candidate;
}
```

- 解析：摩尔投票法，相同元素计数加1，不同减1，计数为0时更换候选元素，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

26. 旋转数组

```
void rotate(vector<int>& nums, int k) {
    int n = nums.size();
    k %= n; // 处理k >= n的情况
    // 反转整个数组
    reverse(nums.begin(), nums.end());
    // 反转前k个元素
    reverse(nums.begin(), nums.begin() + k);
    // 反转剩余元素
    reverse(nums.begin() + k, nums.end());
}
```

- 解析：三次反转法，先反转整个数组，再分别反转前k个和剩余元素，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

27. 寻找数组中消失的数字

```
vector<int> findDisappearedNumbers(vector<int>& nums) {
    int n = nums.size();
    // 将出现的数字对应索引标记为负数
    for (int i = 0; i < n; i++) {
        int index = abs(nums[i]) - 1;
        if (nums[index] > 0) {
            nums[index] = -nums[index];
        }
    }
    // 正数对应的索引+1即为消失的数字
    vector<int> res;
    for (int i = 0; i < n; i++) {
        if (nums[i] > 0) {
            res.push_back(i + 1);
        }
    }
    return res;
}
```

- 解析：利用数组本身作为哈希表，通过正负标记记录数字是否出现，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

28. 盛最多水的容器

```
int maxArea(vector<int>& height) {
    int left = 0, right = height.size() - 1;
```

```
int max_area = 0;
while (left < right) {
    // 计算当前面积: 宽度 * 高度 (取较矮的一边)
    int area = (right - left) * min(height[left], height[right]);
    max_area = max(max_area, area);
    // 移动较矮的一边, 寻找更大的面积
    if (height[left] < height[right]) {
        left++;
    } else {
        right--;
    }
}
return max_area;
}
```

- 解析: 双指针法, 从两端向中间移动, 每次移动较矮的一边, 时间复杂度 $O(n)$ 。

29. 矩阵置零

```
void setZeroes(vector<vector<int>>& matrix) {
    int m = matrix.size();
    int n = matrix[0].size();
    bool first_row_zero = false, first_col_zero = false;

    // 检查第一行是否有零
    for (int j = 0; j < n; j++) {
        if (matrix[0][j] == 0) {
            first_row_zero = true;
            break;
        }
    }

    // 检查第一列是否有零
    for (int i = 0; i < m; i++) {
        if (matrix[i][0] == 0) {
            first_col_zero = true;
            break;
        }
    }

    // 用第一行和第一列标记零的位置
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }

    // 根据标记置零
```

```

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }

    // 处理第一行
    if (first_row_zero) {
        for (int j = 0; j < n; j++) {
            matrix[0][j] = 0;
        }
    }

    // 处理第一列
    if (first_col_zero) {
        for (int i = 0; i < m; i++) {
            matrix[i][0] = 0;
        }
    }
}

```

- 解析：使用第一行和第一列作为标记，记录需要置零的行和列，最后处理第一行和第一列本身，空间复杂度 $O(1)$ 。

30. 螺旋矩阵

```

vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> res;
    if (matrix.empty()) return res;
    int m = matrix.size(), n = matrix[0].size();
    int top = 0, bottom = m - 1;
    int left = 0, right = n - 1;

    while (top <= bottom && left <= right) {
        // 从左到右遍历上边界
        for (int j = left; j <= right; j++) {
            res.push_back(matrix[top][j]);
        }
        top++;

        // 从上到下遍历右边界
        for (int i = top; i <= bottom; i++) {
            res.push_back(matrix[i][right]);
        }
        right--;

        // 从右到左遍历下边界（需检查是否与上边界重合）
        if (top <= bottom) {
            for (int j = right; j >= left; j--) {

```

```

        res.push_back(matrix[bottom][j]);
    }
    bottom--;
}

// 从下到上遍历左边界（需检查是否与右边界重合）
if (left <= right) {
    for (int i = bottom; i >= top; i--) {
        res.push_back(matrix[i][left]);
    }
    left++;
}
}
return res;
}

```

- 解析：按顺时针方向逐层遍历，通过调整边界控制遍历范围。

31. 搜索二维矩阵

```

bool searchMatrix(vector<vector<int>>& matrix, int target) {
    if (matrix.empty() || matrix[0].empty()) return false;
    int m = matrix.size(), n = matrix[0].size();
    // 二分查找：将二维矩阵视为一维数组
    int left = 0, right = m * n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int row = mid / n;
        int col = mid % n;
        if (matrix[row][col] == target) {
            return true;
        } else if (matrix[row][col] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return false;
}

```

- 解析：将有序二维矩阵视为一维有序数组，使用二分查找，时间复杂度 $O(\log(mn))$ 。

32. 对角线遍历

```

vector<int> findDiagonalOrder(vector<vector<int>>& matrix) {
    vector<int> res;
    if (matrix.empty() || matrix[0].empty()) return res;
    int m = matrix.size(), n = matrix[0].size();
    int row = 0, col = 0;

```

```
bool up = true; // 标记当前方向是否向上

while (res.size() < m * n) {
    res.push_back(matrix[row][col]);
    if (up) {
        // 向上移动: 行减1, 列加1
        if (col == n - 1) { // 到达右边界, 向下移动一行
            row++;
            up = false;
        } else if (row == 0) { // 到达上边界, 向右移动一列
            col++;
            up = false;
        } else {
            row--;
            col++;
        }
    } else {
        // 向下移动: 行加1, 列减1
        if (row == m - 1) { // 到达下边界, 向右移动一列
            col++;
            up = true;
        } else if (col == 0) { // 到达左边界, 向下移动一行
            row++;
            up = true;
        } else {
            row++;
            col--;
        }
    }
}

return res;
}
```

- 解析: 模拟对角线遍历过程, 通过方向标记 (向上/向下) 和边界判断调整移动方向。

33. 杨辉三角

```
vector<vector<int>> generate(int numRows) {
    vector<vector<int>> res;
    for (int i = 0; i < numRows; i++) {
        vector<int> row(i + 1, 1); // 每行第一个和最后一个元素为1
        // 填充中间元素: 等于上一行相邻两个元素之和
        for (int j = 1; j < i; j++) {
            row[j] = res[i - 1][j - 1] + res[i - 1][j];
        }
        res.push_back(row);
    }
    return res;
}
```

- 解析：利用杨辉三角特性，每行元素由上一行元素计算得出，时间复杂度 $O(n^2)$ 。

34. 买卖股票的最佳时机

```
int maxProfit(vector<int>& prices) {
    if (prices.empty()) return 0;
    int min_price = prices[0];
    int max_profit = 0;
    for (int price : prices) {
        // 更新最低价格
        if (price < min_price) {
            min_price = price;
        }
        // 计算当前价格卖出的利润，更新最大利润
        else if (price - min_price > max_profit) {
            max_profit = price - min_price;
        }
    }
    return max_profit;
}
```

- 解析：一次遍历，记录历史最低价格，计算每天卖出的最大利润，时间复杂度 $O(n)$ 。

35. 最大子序和

```
int maxSubArray(vector<int>& nums) {
    if (nums.empty()) return 0;
    int current_sum = nums[0];
    int max_sum = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        // 选择当前元素或当前元素加上之前的和
        current_sum = max(nums[i], current_sum + nums[i]);
        // 更新最大和
        max_sum = max(max_sum, current_sum);
    }
    return max_sum;
}
```

- 解析：动态规划，current_sum表示以当前元素结尾的最大子序和，时间复杂度 $O(n)$ 。

四、链表 (36-45题)

36. 反转链表

```
struct ListNode {
    int val;
    ListNode *next;
```

```

ListNode() : val(0), next(nullptr) {}
ListNode(int x) : val(x), next(nullptr) {}
ListNode(int x, ListNode *next) : val(x), next(next) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr != nullptr) {
        ListNode* next = curr->next; // 保存下一个节点
        curr->next = prev; // 反转指针
        prev = curr; // 移动prev
        curr = next; // 移动curr
    }
    return prev; // prev成为新的头节点
}

```

- 解析：迭代法，通过三个指针反转链表指针方向，时间复杂度 $O(n)$ 。

37. 合并两个有序链表

```

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode* dummy = new ListNode(0); // 虚拟头节点
    ListNode* curr = dummy;
    while (l1 != nullptr && l2 != nullptr) {
        if (l1->val <= l2->val) {
            curr->next = l1;
            l1 = l1->next;
        } else {
            curr->next = l2;
            l2 = l2->next;
        }
        curr = curr->next;
    }
    // 连接剩余节点
    curr->next = (l1 != nullptr) ? l1 : l2;
    ListNode* res = dummy->next;
    delete dummy; // 释放虚拟节点
    return res;
}

```

- 解析：双指针法，比较两个链表节点值，较小的节点加入结果链表，时间复杂度 $O(m + n)$ 。

38. 链表的中间结点

```

ListNode* middleNode(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    // 快指针每次走两步，慢指针每次走一步

```

```
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
return slow;
}
```

- 解析：快慢指针法，快指针到达尾部时，慢指针恰好指向中间节点，时间复杂度 $O(n)$ 。

39. 链表中倒数第k个节点

```
ListNode* getKthFromEnd(ListNode* head, int k) {
    ListNode* fast = head;
    ListNode* slow = head;
    // 快指针先移动k步
    for (int i = 0; i < k; i++) {
        fast = fast->next;
    }
    // 快慢指针同时移动，直到快指针到达尾部
    while (fast != nullptr) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
```

- 解析：快慢指针法，快指针先移动k步，然后两指针同步移动，时间复杂度 $O(n)$ 。

40. 环形链表的检测

```
bool hasCycle(ListNode *head) {
    if (head == nullptr || head->next == nullptr) return false;
    ListNode* slow = head;
    ListNode* fast = head->next;
    // 快慢指针相遇则有环
    while (slow != fast) {
        if (fast == nullptr || fast->next == nullptr) return false;
        slow = slow->next;
        fast = fast->next->next;
    }
    return true;
}
```

- 解析：快慢指针法，若有环，快指针最终会追上慢指针，时间复杂度 $O(n)$ 。

41. 相交链表

```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {  
    if (headA == nullptr || headB == nullptr) return nullptr;  
    ListNode* a = headA;  
    ListNode* b = headB;  
    // 当a和b到达尾部时, 分别指向另一个链表的头部, 直到相遇  
    while (a != b) {  
        a = (a == nullptr) ? headB : a->next;  
        b = (b == nullptr) ? headA : b->next;  
    }  
    return a;  
}
```

- 解析：双指针法，两指针分别遍历两个链表，到达尾部后切换到另一个链表，相遇点即为相交节点，时间复杂度 $O(m + n)$ 。

42. 删除链表的倒数第N个节点

```
ListNode* removeNthFromEnd(ListNode* head, int n) {  
    ListNode* dummy = new ListNode(0, head); // 虚拟头节点, 处理删除头节点的情况  
    ListNode* fast = dummy;  
    ListNode* slow = dummy;  
  
    // 快指针先移动n+1步  
    for (int i = 0; i <= n; i++) {  
        fast = fast->next;  
    }  
  
    // 快慢指针同时移动, 直到快指针到达尾部  
    while (fast != nullptr) {  
        slow = slow->next;  
        fast = fast->next;  
    }  
  
    // 删除slow的下一个节点  
    ListNode* to_delete = slow->next;  
    slow->next = slow->next->next;  
    delete to_delete;  
  
    ListNode* res = dummy->next;  
    delete dummy;  
    return res;  
}
```

- 解析：快慢指针法，快指针先移动 $n + 1$ 步，然后两指针同步移动，慢指针指向待删除节点的前一个节点，时间复杂度 $O(n)$ 。

43. 两两交换链表中的节点

```

ListNode* swapPairs(ListNode* head) {
    if (head == nullptr || head->next == nullptr) return head;
    ListNode* dummy = new ListNode(0, head);
    ListNode* prev = dummy;
    ListNode* first = head;
    ListNode* second = head->next;

    while (first != nullptr && second != nullptr) {
        // 交换节点
        prev->next = second;
        first->next = second->next;
        second->next = first;

        // 移动指针
        prev = first;
        first = first->next;
        if (first != nullptr) {
            second = first->next;
        }
    }

    ListNode* res = dummy->next;
    delete dummy;
    return res;
}

```

- 解析：迭代法，使用三个指针（prev, first, second）完成两两交换，时间复杂度O(n)。

44. K个一组翻转链表

```

// 辅助函数：反转链表，返回新的头和尾
pair<ListNode*, ListNode*> reverse(ListNode* head, ListNode* tail) {
    ListNode* prev = tail->next;
    ListNode* curr = head;
    while (prev != tail) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return {tail, head};
}

ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* prev = dummy;

    while (head != nullptr) {
        // 检查剩余节点是否有k个
        ListNode* tail = prev;

```

```

        for (int i = 0; i < k; i++) {
            tail = tail->next;
            if (tail == nullptr) { // 不足k个, 不翻转
                ListNode* res = dummy->next;
                delete dummy;
                return res;
            }
        }

        ListNode* next_head = tail->next;
        // 翻转当前k个节点
        auto [new_head, new_tail] = reverse(head, tail);
        // 连接翻转后的链表
        prev->next = new_head;
        new_tail->next = next_head;
        // 移动指针
        prev = new_tail;
        head = next_head;
    }

    ListNode* res = dummy->next;
    delete dummy;
    return res;
}

```

- 解析：先检查剩余节点是否足够k个，足够则翻转，否则直接返回，时间复杂度O(n)。

45. 复制带随机指针的链表

```

struct Node {
    int val;
    Node *next;
    Node *random;
    Node(int x) : val(x), next(nullptr), random(nullptr) {}
};

Node* copyRandomList(Node* head) {
    if (head == nullptr) return nullptr;

    // 第一步：在每个节点后插入复制节点
    Node* curr = head;
    while (curr != nullptr) {
        Node* copy = new Node(curr->val);
        copy->next = curr->next;
        curr->next = copy;
        curr = copy->next;
    }

    // 第二步：设置复制节点的random指针
    curr = head;
    while (curr != nullptr) {

```

```
        if (curr->random != nullptr) {
            curr->next->random = curr->random->next;
        }
        curr = curr->next->next;
    }

    // 第三步：分离原链表和复制链表
    curr = head;
    Node* dummy = new Node(0);
    Node* copy_curr = dummy;
    while (curr != nullptr) {
        copy_curr->next = curr->next;
        copy_curr = copy_curr->next;
        curr->next = curr->next->next;
        curr = curr->next;
    }

    Node* res = dummy->next;
    delete dummy;
    return res;
}
```

- 解析：三步法，先插入复制节点，再设置random指针，最后分离两个链表，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

五、栈与队列 (46-55题)

46. 用栈实现队列

```
class MyQueue {
private:
    stack<int> in_stack; // 输入栈
    stack<int> out_stack; // 输出栈

    // 将输入栈元素转移到输出栈
    void transfer() {
        while (!in_stack.empty()) {
            out_stack.push(in_stack.top());
            in_stack.pop();
        }
    }

public:
    MyQueue() {}

    void push(int x) {
        in_stack.push(x);
    }

    int pop() {
        if (out_stack.empty()) {
```

```
        transfer();
    }
    int val = out_stack.top();
    out_stack.pop();
    return val;
}

int peek() {
    if (out_stack.empty()) {
        transfer();
    }
    return out_stack.top();
}

bool empty() {
    return in_stack.empty() && out_stack.empty();
}
};
```

- 解析：使用两个栈，输入栈负责push，输出栈负责pop和peek，需要时将输入栈元素转移到输出栈。

47. 用队列实现栈

```
class MyStack {
private:
    queue<int> q1; // 主队列
    queue<int> q2; // 辅助队列

public:
    MyStack() {}

    void push(int x) {
        q2.push(x);
        // 将q1元素转移到q2
        while (!q1.empty()) {
            q2.push(q1.front());
            q1.pop();
        }
        // 交换q1和q2
        swap(q1, q2);
    }

    int pop() {
        int val = q1.front();
        q1.pop();
        return val;
    }

    int top() {
        return q1.front();
    }
}
```

```
bool empty() {  
    return q1.empty();  
}  
};
```

- 解析：使用两个队列，每次push时将新元素加入辅助队列，再将主队列元素转移过来，最后交换两个队列。

48. 有效的括号

```
bool isValid(string s) {  
    stack<char> st;  
    unordered_map<char, char> mapping = {{'}', '('}, {'}', '{'}, {'}', '['}, {'}', '['}, {'}', '['}};  
  
    for (char c : s) {  
        if (mapping.count(c)) { // 右括号  
            if (st.empty() || st.top() != mapping[c]) {  
                return false;  
            }  
            st.pop();  
        } else { // 左括号  
            st.push(c);  
        }  
    }  
    // 栈为空说明所有括号都匹配  
    return st.empty();  
}
```

- 解析：栈用于存储左括号，遇到右括号时检查是否与栈顶左括号匹配，时间复杂度O(n)。

49. 最小栈

```
class MinStack {  
private:  
    stack<int> data_stack; // 存储数据  
    stack<int> min_stack; // 存储当前最小值  
  
public:  
    MinStack() {  
        min_stack.push(INT_MAX); // 初始化最小值为最大整数  
    }  
  
    void push(int val) {  
        data_stack.push(val);  
        // 压入当前最小值 (当前val与栈顶最小值的较小者)  
        min_stack.push(min(val, min_stack.top()));  
    }  
}
```

```
void pop() {
    data_stack.pop();
    min_stack.pop();
}

int top() {
    return data_stack.top();
}

int getMin() {
    return min_stack.top();
}
};
```

- 解析：使用辅助栈存储当前最小值，每次push时同步更新辅助栈，getMin操作时间复杂度O(1)。

50. 栈的压入、弹出序列

```
bool validateStackSequences(vector<int>& pushed, vector<int>& popped) {
    stack<int> st;
    int j = 0; // 弹出序列索引
    for (int num : pushed) {
        st.push(num);
        // 检查栈顶是否与弹出序列当前元素匹配
        while (!st.empty() && st.top() == popped[j]) {
            st.pop();
            j++;
        }
    }
    // 栈为空说明匹配成功
    return st.empty();
}
```

- 解析：模拟入栈过程，每次入栈后检查是否可以弹出，最后栈为空则序列有效。

51. 滑动窗口最大值

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    vector<int> res;
    deque<int> dq; // 双端队列，存储索引，对应元素单调递减

    for (int i = 0; i < nums.size(); i++) {
        // 移除超出窗口的元素
        if (!dq.empty() && dq.front() < i - k + 1) {
            dq.pop_front();
        }

        // 移除队列中比当前元素小的元素
```

```
        while (!dq.empty() && nums[dq.back()] < nums[i]) {
            dq.pop_back();
        }

        dq.push_back(i);

        // 窗口形成后，记录最大值（队首元素）
        if (i >= k - 1) {
            res.push_back(nums[dq.front()]);
        }
    }
    return res;
}
```

- 解析：使用单调队列，队列中存储的索引对应的元素保持递减，队首即为窗口最大值，时间复杂度 $O(n)$ 。

52. 设计循环队列

```
class MyCircularQueue {
private:
    vector<int> data;
    int front; // 队首索引
    int rear; // 队尾索引的下一个位置
    int size; // 队列容量

public:
    MyCircularQueue(int k) {
        size = k + 1; // 额外一个空间用于区分空和满
        data.resize(size);
        front = 0;
        rear = 0;
    }

    bool enqueue(int value) {
        if (isFull()) return false;
        data[rear] = value;
        rear = (rear + 1) % size;
        return true;
    }

    bool dequeue() {
        if (isEmpty()) return false;
        front = (front + 1) % size;
        return true;
    }

    int Front() {
        if (isEmpty()) return -1;
        return data[front];
    }
}
```

```
int Rear() {
    if (isEmpty()) return -1;
    return data[(rear - 1 + size) % size];
}

bool isEmpty() {
    return front == rear;
}

bool isFull() {
    return (rear + 1) % size == front;
}
};
```

- 解析：使用数组实现，预留一个额外空间区分队列空和满的状态，时间复杂度 $O(1)$ 。

53. 基本计算器

```
int calculate(string s) {
    stack<int> st;
    int num = 0;
    int sign = 1; // 1表示正, -1表示负
    int res = 0;

    for (char c : s) {
        if (isdigit(c)) {
            num = num * 10 + (c - '0'); // 处理多位数
        } else if (c == '+') {
            res += sign * num;
            num = 0;
            sign = 1;
        } else if (c == '-') {
            res += sign * num;
            num = 0;
            sign = -1;
        } else if (c == '(') {
            // 保存当前结果和符号, 重置res和sign
            st.push(res);
            st.push(sign);
            res = 0;
            sign = 1;
        } else if (c == ')') {
            res += sign * num;
            num = 0;
            // 乘以括号前的符号
            res *= st.top();
            st.pop();
            // 加上括号前的结果
            res += st.top();
            st.pop();
        }
    }
    return res;
}
```

```

    }
}
// 处理最后一个数字
res += sign * num;
return res;
}

```

- 解析：使用栈处理括号，遇到'('时保存当前结果和符号，遇到')'时计算括号内结果并与外部合并。

54. 字符串解码

```

string decodeString(string s) {
    stack<int> num_stack;    // 存储数字
    stack<string> str_stack; // 存储字符串
    string curr_str;
    int curr_num = 0;

    for (char c : s) {
        if (isdigit(c)) {
            curr_num = curr_num * 10 + (c - '0'); // 处理多位数
        } else if (c == '[') {
            // 保存当前数字和字符串，重置
            num_stack.push(curr_num);
            str_stack.push(curr_str);
            curr_num = 0;
            curr_str = "";
        } else if (c == ']') {
            // 弹出数字和字符串，重复curr_str
            int num = num_stack.top();
            num_stack.pop();
            string prev_str = str_stack.top();
            str_stack.pop();
            string temp;
            for (int i = 0; i < num; i++) {
                temp += curr_str;
            }
            curr_str = prev_str + temp;
        } else {
            // 普通字符
            curr_str += c;
        }
    }
    return curr_str;
}

```

- 解析：使用两个栈分别存储数字和字符串，遇到']'时弹出并重复字符串，时间复杂度 $O(n)$ ， n 为解码后字符串长度。

55. 课程表 (拓扑排序)

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> adj(numCourses); // 邻接表
    vector<int> in_degree(numCourses, 0); // 入度

    // 构建图和入度数组
    for (auto& p : prerequisites) {
        int course = p[0];
        int prereq = p[1];
        adj[prereq].push_back(course);
        in_degree[course]++;
    }

    queue<int> q;
    // 入度为0的节点入队
    for (int i = 0; i < numCourses; i++) {
        if (in_degree[i] == 0) {
            q.push(i);
        }
    }

    int count = 0; // 记录已学习的课程数
    while (!q.empty()) {
        int curr = q.front();
        q.pop();
        count++;
        // 减少相邻节点的入度
        for (int next : adj[curr]) {
            in_degree[next]--;
            if (in_degree[next] == 0) {
                q.push(next);
            }
        }
    }

    // 所有课程都能学习则无环
    return count == numCourses;
}
```

- 解析：拓扑排序判断有向图是否有环，通过入度和队列实现，时间复杂度 $O(V + E)$ 。

六、树与图 (56-70题)

56. 二叉树的前序遍历

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

```
TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
};

// 递归版
void preorder(TreeNode* root, vector<int>& res) {
    if (root == nullptr) return;
    res.push_back(root->val);
    preorder(root->left, res);
    preorder(root->right, res);
}

vector<int> preorderTraversal(TreeNode* root) {
    vector<int> res;
    preorder(root, res);
    return res;
}

// 迭代版
vector<int> preorderTraversal(TreeNode* root) {
    vector<int> res;
    if (root == nullptr) return res;
    stack<TreeNode*> st;
    st.push(root);
    while (!st.empty()) {
        TreeNode* node = st.top();
        st.pop();
        res.push_back(node->val);
        // 右子树先入栈，左子树后入栈，保证左子树先访问
        if (node->right != nullptr) {
            st.push(node->right);
        }
        if (node->left != nullptr) {
            st.push(node->left);
        }
    }
    return res;
}
```

- 解析：前序遍历顺序为根-左-右，迭代版使用栈实现，先压右子树再压左子树。

57. 二叉树的中序遍历

```
// 递归版
void inorder(TreeNode* root, vector<int>& res) {
    if (root == nullptr) return;
    inorder(root->left, res);
    res.push_back(root->val);
    inorder(root->right, res);
}
```

```
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> res;
    inorder(root, res);
    return res;
}

// 迭代版
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    while (curr != nullptr || !st.empty()) {
        // 遍历到最左节点
        while (curr != nullptr) {
            st.push(curr);
            curr = curr->left;
        }
        // 访问节点
        curr = st.top();
        st.pop();
        res.push_back(curr->val);
        // 处理右子树
        curr = curr->right;
    }
    return res;
}
```

- 解析：中序遍历顺序为左-根-右，迭代版先遍历到最左节点，再访问并处理右子树。

58. 二叉树的后序遍历

```
// 递归版
void postorder(TreeNode* root, vector<int>& res) {
    if (root == nullptr) return;
    postorder(root->left, res);
    postorder(root->right, res);
    res.push_back(root->val);
}

vector<int> postorderTraversal(TreeNode* root) {
    vector<int> res;
    postorder(root, res);
    return res;
}

// 迭代版
vector<int> postorderTraversal(TreeNode* root) {
    vector<int> res;
    if (root == nullptr) return res;
    stack<TreeNode*> st;
    TreeNode* prev = nullptr; // 记录上一个访问的节点
```

```
TreeNode* curr = root;
while (curr != nullptr || !st.empty()) {
    // 遍历到最左节点
    while (curr != nullptr) {
        st.push(curr);
        curr = curr->left;
    }
    curr = st.top();
    // 右子树为空或已访问，则访问当前节点
    if (curr->right == nullptr || curr->right == prev) {
        res.push_back(curr->val);
        st.pop();
        prev = curr;
        curr = nullptr;
    } else {
        // 处理右子树
        curr = curr->right;
    }
}
return res;
}
```

- 解析：后序遍历顺序为左-右-根，迭代版需判断右子树是否已访问。

59. 二叉树的层序遍历

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (root == nullptr) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int size = q.size(); // 当前层的节点数
        vector<int> level;
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front();
            q.pop();
            level.push_back(node->val);
            // 左子树入队
            if (node->left != nullptr) {
                q.push(node->left);
            }
            // 右子树入队
            if (node->right != nullptr) {
                q.push(node->right);
            }
        }
        res.push_back(level);
    }
}
```

```
    return res;
}
```

- 解析：使用队列实现，每次处理一层的所有节点，时间复杂度 $O(n)$ 。

60. 二叉树的最大深度

```
// 递归版
int maxDepth(TreeNode* root) {
    if (root == nullptr) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}

// 迭代版（层序遍历）
int maxDepth(TreeNode* root) {
    if (root == nullptr) return 0;
    queue<TreeNode*> q;
    q.push(root);
    int depth = 0;

    while (!q.empty()) {
        int size = q.size();
        depth++; // 每处理一层，深度加1
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front();
            q.pop();
            if (node->left != nullptr) q.push(node->left);
            if (node->right != nullptr) q.push(node->right);
        }
    }
    return depth;
}
```

- 解析：递归版计算左右子树最大深度加1，迭代版通过层序遍历计数层数。

61. 二叉树的最小深度

```
int minDepth(TreeNode* root) {
    if (root == nullptr) return 0;
    // 左子树为空，返回右子树深度+1
    if (root->left == nullptr) {
        return 1 + minDepth(root->right);
    }
    // 右子树为空，返回左子树深度+1
    if (root->right == nullptr) {
        return 1 + minDepth(root->left);
    }
    // 左右子树都不为空，返回较小深度+1
}
```

```
    return 1 + min(minDepth(root->left), minDepth(root->right));  
}
```

- 解析：注意与最大深度的区别，需处理其中一个子树为空的情况。

62. 对称二叉树

```
// 递归版  
bool isSymmetricHelper(TreeNode* left, TreeNode* right) {  
    if (left == nullptr && right == nullptr) return true;  
    if (left == nullptr || right == nullptr) return false;  
    return (left->val == right->val)  
        && isSymmetricHelper(left->left, right->right)  
        && isSymmetricHelper(left->right, right->left);  
}  
  
bool isSymmetric(TreeNode* root) {  
    if (root == nullptr) return true;  
    return isSymmetricHelper(root->left, root->right);  
}  
  
// 迭代版  
bool isSymmetric(TreeNode* root) {  
    if (root == nullptr) return true;  
    queue<TreeNode*> q;  
    q.push(root->left);  
    q.push(root->right);  
  
    while (!q.empty()) {  
        TreeNode* left = q.front();  
        q.pop();  
        TreeNode* right = q.front();  
        q.pop();  
  
        if (left == nullptr && right == nullptr) continue;  
        if (left == nullptr || right == nullptr) return false;  
        if (left->val != right->val) return false;  
  
        // 按对称顺序入队  
        q.push(left->left);  
        q.push(right->right);  
        q.push(left->right);  
        q.push(right->left);  
    }  
    return true;  
}
```

- 解析：判断左右子树是否对称，递归版比较左左与右右、左右与右左，迭代版使用队列按对称顺序处理。

63. 路径总和

```
bool hasPathSum(TreeNode* root, int targetSum) {
    if (root == nullptr) return false;
    // 叶子节点, 检查是否满足条件
    if (root->left == nullptr && root->right == nullptr) {
        return root->val == targetSum;
    }
    // 递归检查左右子树
    return hasPathSum(root->left, targetSum - root->val)
        || hasPathSum(root->right, targetSum - root->val);
}
```

- 解析: 递归计算从根到叶子节点的路径和是否等于目标值, 时间复杂度 $O(n)$ 。

64. 从中序与后序遍历序列构造二叉树

```
TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
    if (inorder.empty() || postorder.empty()) return nullptr;
    // 后序遍历最后一个元素是根节点
    int root_val = postorder.back();
    TreeNode* root = new TreeNode(root_val);

    // 找到根节点在中序遍历中的位置
    int idx = 0;
    while (idx < inorder.size() && inorder[idx] != root_val) {
        idx++;
    }

    // 分割中序遍历
    vector<int> left_in(inorder.begin(), inorder.begin() + idx);
    vector<int> right_in(inorder.begin() + idx + 1, inorder.end());

    // 分割后序遍历
    vector<int> left_post(postorder.begin(), postorder.begin() + left_in.size());
    vector<int> right_post(postorder.begin() + left_in.size(), postorder.end() - 1);

    // 递归构建左右子树
    root->left = buildTree(left_in, left_post);
    root->right = buildTree(right_in, right_post);

    return root;
}
```

- 解析: 后序遍历最后一个元素是根节点, 中序遍历中根节点左侧是左子树, 右侧是右子树, 递归构建二叉树。

65. 二叉搜索树的最近公共祖先

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (root == nullptr) return nullptr;
    // 两个节点都在左子树
    if (p->val < root->val && q->val < root->val) {
        return lowestCommonAncestor(root->left, p, q);
    }
    // 两个节点都在右子树
    else if (p->val > root->val && q->val > root->val) {
        return lowestCommonAncestor(root->right, p, q);
    }
    // 一个在左，一个在右，当前节点是最近公共祖先
    else {
        return root;
    }
}
```

- 解析：利用BST特性，若p和q都小于根则在左子树，都大于根则在右子树，否则根就是最近公共祖先。

66. 二叉树的右视图

```
vector<int> rightSideView(TreeNode* root) {
    vector<int> res;
    if (root == nullptr) return res;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int size = q.size();
        // 记录每层最后一个节点（右视图）
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front();
            q.pop();
            if (i == size - 1) { // 最后一个节点
                res.push_back(node->val);
            }
            if (node->left != nullptr) q.push(node->left);
            if (node->right != nullptr) q.push(node->right);
        }
    }
    return res;
}
```

- 解析：层序遍历，记录每层最后一个节点，即为右视图看到的节点。

67. 验证二叉搜索树

```

// 递归版，使用上下界
bool isValidBSTHelper(TreeNode* root, long long lower, long long upper) {
    if (root == nullptr) return true;
    // 检查当前节点是否在[lower, upper]范围内
    if (root->val <= lower || root->val >= upper) return false;
    // 左子树范围[lower, root->val], 右子树范围[root->val, upper]
    return isValidBSTHelper(root->left, lower, root->val)
        && isValidBSTHelper(root->right, root->val, upper);
}

bool isValidBST(TreeNode* root) {
    return isValidBSTHelper(root, LLONG_MIN, LLONG_MAX);
}

// 迭代版，中序遍历是否递增
bool isValidBST(TreeNode* root) {
    stack<TreeNode*> st;
    TreeNode* curr = root;
    long long prev_val = LLONG_MIN; // 记录前一个节点的值

    while (curr != nullptr || !st.empty()) {
        while (curr != nullptr) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top();
        st.pop();
        // 中序遍历应递增
        if (curr->val <= prev_val) return false;
        prev_val = curr->val;
        curr = curr->right;
    }
    return true;
}

```

- 解析：BST中序遍历应为递增序列，或通过递归检查每个节点是否在合理范围内。

68. 二叉树的序列化与反序列化

```

// 序列化：前序遍历
string serialize(TreeNode* root) {
    if (root == nullptr) return "#,";
    string res = to_string(root->val) + ",";
    res += serialize(root->left);
    res += serialize(root->right);
    return res;
}

// 辅助函数：将字符串分割为节点值列表
vector<string> split(const string& s) {

```

```

    vector<string> res;
    stringstream ss(s);
    string token;
    while (getline(ss, token, ',')) {
        res.push_back(token);
    }
    return res;
}

// 反序列化: 递归构建
TreeNode* deserializeHelper(vector<string>& data, int& index) {
    if (index >= data.size() || data[index] == "#") {
        index++;
        return nullptr;
    }
    TreeNode* root = new TreeNode(stoi(data[index]));
    index++;
    root->left = deserializeHelper(data, index);
    root->right = deserializeHelper(data, index);
    return root;
}

TreeNode* deserialize(string data) {
    vector<string> nodes = split(data);
    int index = 0;
    return deserializeHelper(nodes, index);
}

```

- 解析：序列化使用前序遍历，空节点用特殊符号（如"#")表示，反序列化时递归重建二叉树。

69. 克隆图

```

class Node {
public:
    int val;
    vector<Node*> neighbors;
    Node() {
        val = 0;
        neighbors = vector<Node*>();
    }
    Node(int _val) {
        val = _val;
        neighbors = vector<Node*>();
    }
    Node(int _val, vector<Node*> _neighbors) {
        val = _val;
        neighbors = _neighbors;
    }
};

Node* cloneGraph(Node* node) {

```

```

    if (node == nullptr) return nullptr;
    unordered_map<Node*, Node*> visited; // 原节点到克隆节点的映射
    queue<Node*> q;

    // 创建第一个节点
    Node* clone_node = new Node(node->val);
    visited[node] = clone_node;
    q.push(node);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();

        // 处理每个邻居
        for (Node* neighbor : curr->neighbors) {
            if (visited.find(neighbor) == visited.end()) {
                // 未访问过, 创建克隆节点
                visited[neighbor] = new Node(neighbor->val);
                q.push(neighbor);
            }
            // 连接克隆节点的邻居
            visited[curr]->neighbors.push_back(visited[neighbor]);
        }
    }

    return clone_node;
}

```

- 解析：BFS遍历图，使用哈希表记录已克隆的节点，避免重复创建，时间复杂度 $O(V + E)$ 。

70. 岛屿数量

```

void dfs(vector<vector<char>>& grid, int i, int j) {
    int m = grid.size();
    int n = grid[0].size();
    // 越界或不是陆地, 返回
    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] != '1') {
        return;
    }
    // 标记为已访问
    grid[i][j] = '0';
    // 递归访问上下左右
    dfs(grid, i - 1, j);
    dfs(grid, i + 1, j);
    dfs(grid, i, j - 1);
    dfs(grid, i, j + 1);
}

int numIslands(vector<vector<char>>& grid) {
    if (grid.empty() || grid[0].empty()) return 0;
    int m = grid.size();

```

```
int n = grid[0].size();
int count = 0;

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (grid[i][j] == '1') {
            count++;
            dfs(grid, i, j); // 淹没整个岛屿
        }
    }
}
return count;
}
```

- 解析：DFS或BFS遍历，遇到陆地则计数并将整个连通区域标记为已访问，时间复杂度 $O(mn)$ 。

七、动态规划（71-85题）

71. 斐波那契数列

```
// 递归版（效率低，有重复计算）
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

// 动态规划版（迭代）
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

- 解析：斐波那契数列定义为 $F(n) = F(n-1) + F(n-2)$ ，动态规划版避免重复计算，时间复杂度 $O(n)$ 。

72. 爬楼梯

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2, c;
    for (int i = 3; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
}
```

```
    }  
    return b;  
}
```

- 解析：到达第n阶的方法数 = 到达第n-1阶的方法数 + 到达第n-2阶的方法数，时间复杂度O(n)。

73. 三角形最小路径和

```
int minimumTotal(vector<vector<int>>& triangle) {  
    int n = triangle.size();  
    // 从倒数第二行开始向上计算  
    for (int i = n - 2; i >= 0; i--) {  
        for (int j = 0; j <= i; j++) {  
            // 当前位置的最小路径和 = 当前值 + 下一行相邻两个位置的较小值  
            triangle[i][j] += min(triangle[i+1][j], triangle[i+1][j+1]);  
        }  
    }  
    return triangle[0][0];  
}
```

- 解析：自底向上动态规划，修改原数组存储最小路径和，空间复杂度O(1)。

74. 最大子序和

```
int maxSubArray(vector<int>& nums) {  
    if (nums.empty()) return 0;  
    int current_sum = nums[0];  
    int max_sum = nums[0];  
    for (int i = 1; i < nums.size(); i++) {  
        // 当前最大子序和 = max(当前元素, 当前元素 + 之前的最大子序和)  
        current_sum = max(nums[i], current_sum + nums[i]);  
        max_sum = max(max_sum, current_sum);  
    }  
    return max_sum;  
}
```

- 解析：动态规划，current_sum表示以当前元素结尾的最大子序和，时间复杂度O(n)。

75. 最长递增子序列

```
int lengthOfLIS(vector<int>& nums) {  
    if (nums.empty()) return 0;  
    int n = nums.size();  
    vector<int> dp(n, 1); // dp[i]表示以nums[i]结尾的最长递增子序列长度  
    int max_len = 1;
```

```

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[i] > nums[j]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        max_len = max(max_len, dp[i]);
    }
    return max_len;
}

// 优化版, 时间复杂度O(n log n)
int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for (int num : nums) {
        // 找到第一个大于等于num的位置
        auto it = lower_bound(tails.begin(), tails.end(), num);
        if (it == tails.end()) {
            tails.push_back(num);
        } else {
            *it = num;
        }
    }
    return tails.size();
}

```

- 解析：动态规划版时间复杂度 $O(n^2)$ ，优化版使用贪心+二分查找，时间复杂度 $O(n \log n)$ 。

76. 最长公共子序列

```

int longestCommonSubsequence(string text1, string text2) {
    int m = text1.size();
    int n = text2.size();
    // dp[i][j]表示text1[0..i-1]和text2[0..j-1]的最长公共子序列长度
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1[i - 1] == text2[j - 1]) {
                // 字符相同, 长度+1
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                // 字符不同, 取较长的
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}

```

- 解析：二维动态规划，根据字符是否相同更新状态，时间复杂度 $O(mn)$ 。

77. 零钱兑换

```
int coinChange(vector<int>& coins, int amount) {
    // dp[i]表示兑换金额i所需的最少硬币数
    vector<int> dp(amount + 1, amount + 1); // 初始化为一个较大值
    dp[0] = 0; // 金额0需要0个硬币

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```

- 解析：动态规划， $dp[i] = \min(dp[i - \text{coin}] + 1)$ for all $\text{coin} \leq i$ ，时间复杂度 $O(\text{amount} * n)$ ， n 为硬币种类数。

78. 零钱兑换 II

```
int change(int amount, vector<int>& coins) {
    // dp[i]表示兑换金额i的方法数
    vector<int> dp(amount + 1, 0);
    dp[0] = 1; // 金额0有1种方法（不选任何硬币）

    for (int coin : coins) {
        for (int i = coin; i <= amount; i++) {
            dp[i] += dp[i - coin];
        }
    }

    return dp[amount];
}
```

- 解析：动态规划，外层循环遍历硬币，内层循环遍历金额，避免重复计算，时间复杂度 $O(\text{amount} * n)$ 。

79. 不同路径

```
int uniquePaths(int m, int n) {
    // dp[i][j]表示到达(i,j)的路径数
    vector<vector<int>> dp(m, vector<int>(n, 1)); // 第一行和第一列都只有1种路径
}
```

```

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            // 到达(i,j)的路径数 = 到达(i-1,j)的路径数 + 到达(i,j-1)的路径数
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }

    return dp[m - 1][n - 1];
}

// 优化空间复杂度为O(n)
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }
    return dp[n - 1];
}

```

- 解析：动态规划，路径数由上方和左方路径数之和决定，优化版使用一维数组，空间复杂度O(n)。

80. 不同路径 II

```

int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
    int m = obstacleGrid.size();
    int n = obstacleGrid[0].size();
    // 起点或终点有障碍物，路径数为0
    if (obstacleGrid[0][0] == 1 || obstacleGrid[m-1][n-1] == 1) {
        return 0;
    }

    vector<vector<int>> dp(m, vector<int>(n, 0));
    // 初始化第一行
    for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) {
        dp[0][j] = 1;
    }
    // 初始化第一列
    for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) {
        dp[i][0] = 1;
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (obstacleGrid[i][j] == 0) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }
    }
}

```

```
        return dp[m - 1][n - 1];
    }
```

- 解析：在不同路径基础上，处理障碍物的情况，遇到障碍物则路径数为0。

81. 最小路径和

```
int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();

    // 初始化第一行
    for (int j = 1; j < n; j++) {
        grid[0][j] += grid[0][j - 1];
    }
    // 初始化第一列
    for (int i = 1; i < m; i++) {
        grid[i][0] += grid[i - 1][0];
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            // 当前路径和 = 当前值 + min(上方路径和, 左方路径和)
            grid[i][j] += min(grid[i - 1][j], grid[i][j - 1]);
        }
    }

    return grid[m - 1][n - 1];
}
```

- 解析：动态规划，修改原数组存储最小路径和，时间复杂度 $O(mn)$ ，空间复杂度 $O(1)$ 。

82. 编辑距离

```
int minDistance(string word1, string word2) {
    int m = word1.size();
    int n = word2.size();
    // dp[i][j]表示word1[0..i-1]转换为word2[0..j-1]的最少操作数
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    // 初始化：一个字符串为空，操作数为另一个字符串的长度
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1[i - 1] == word2[j - 1]) {
                // 字符相同，不需要操作
                dp[i][j] = dp[i - 1][j - 1];
            }
        }
    }
}
```

```

        } else {
            // 取插入、删除、替换的最小操作数 + 1
            dp[i][j] = min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]}) +
1;
        }
    }
}

return dp[m][n];
}

```

- 解析：经典动态规划问题，考虑插入、删除、替换三种操作，时间复杂度 $O(mn)$ 。

83. 打家劫舍

```

int rob(vector<int>& nums) {
    if (nums.empty()) return 0;
    int n = nums.size();
    if (n == 1) return nums[0];

    // dp[i]表示前i个房子能偷到的最大金额
    vector<int> dp(n);
    dp[0] = nums[0];
    dp[1] = max(nums[0], nums[1]);

    for (int i = 2; i < n; i++) {
        // 偷第i个房子: dp[i-2] + nums[i]
        // 不偷第i个房子: dp[i-1]
        dp[i] = max(dp[i - 2] + nums[i], dp[i - 1]);
    }

    return dp[n - 1];
}

// 优化空间复杂度为O(1)
int rob(vector<int>& nums) {
    if (nums.empty()) return 0;
    int n = nums.size();
    if (n == 1) return nums[0];

    int a = nums[0], b = max(nums[0], nums[1]);
    for (int i = 2; i < n; i++) {
        int c = max(a + nums[i], b);
        a = b;
        b = c;
    }

    return b;
}

```

- 解析：动态规划，每个房子有偷或不偷两种选择，取最大值，时间复杂度 $O(n)$ 。

84. 打家劫舍 II

```
// 辅助函数：计算线性排列的房子的最大金额
int robLinear(vector<int>& nums, int start, int end) {
    if (start == end) return nums[start];
    int a = nums[start];
    int b = max(nums[start], nums[start + 1]);
    for (int i = start + 2; i <= end; i++) {
        int c = max(a + nums[i], b);
        a = b;
        b = c;
    }
    return b;
}

int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];
    // 分成两种情况：不偷第一个房子 或 不偷最后一个房子
    return max(robLinear(nums, 0, n - 2), robLinear(nums, 1, n - 1));
}
```

- 解析：环形排列的房子，第一个和最后一个不能同时偷，分为两种情况计算，取最大值。

85. 最长回文子串

```
string longestPalindrome(string s) {
    int n = s.size();
    if (n < 2) return s;

    // dp[i][j]表示s[i..j]是否为回文串
    vector<vector<bool>> dp(n, vector<bool>(n, false));
    int start = 0, max_len = 1;

    // 长度为1的子串都是回文串
    for (int i = 0; i < n; i++) {
        dp[i][i] = true;
    }

    // 长度为2的子串
    for (int i = 0; i < n - 1; i++) {
        if (s[i] == s[i + 1]) {
            dp[i][i + 1] = true;
            start = i;
            max_len = 2;
        }
    }
}
```

```
// 长度大于2的子串
for (int len = 3; len <= n; len++) {
    for (int i = 0; i <= n - len; i++) {
        int j = i + len - 1; // 子串结束索引
        // s[i..j]是回文串当且仅当s[i] == s[j]且s[i+1..j-1]是回文串
        if (s[i] == s[j] && dp[i + 1][j - 1]) {
            dp[i][j] = true;
            start = i;
            max_len = len;
        }
    }
}

return s.substr(start, max_len);
}
```

- 解析：动态规划，判断子串是否为回文串，时间复杂度 $O(n^2)$ ，空间复杂度 $O(n^2)$ 。

八、排序与查找 (86-95题)

86. 二分查找

```
int search(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // 避免溢出
        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

- 解析：在有序数组中查找目标值，每次缩小一半范围，时间复杂度 $O(\log n)$ 。

87. 搜索插入位置

```
int searchInsert(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
```

```
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
// 循环结束时left即为插入位置
return left;
}
```

- 解析：二分查找变种，找不到目标值时返回应该插入的位置，时间复杂度 $O(\log n)$ 。

88. 在排序数组中查找元素的第一个和最后一个位置

```
vector<int> searchRange(vector<int>& nums, int target) {
    vector<int> res = {-1, -1};
    if (nums.empty()) return res;

    // 查找第一个出现的位置
    int left = 0, right = nums.size() - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    if (nums[left] != target) return res;
    res[0] = left;

    // 查找最后一个出现的位置
    right = nums.size() - 1;
    while (left < right) {
        int mid = left + (right - left + 1) / 2; // 向上取整
        if (nums[mid] > target) {
            right = mid - 1;
        } else {
            left = mid;
        }
    }
    res[1] = right;

    return res;
}
```

- 解析：两次二分查找，分别寻找第一个和最后一个出现的位置，时间复杂度 $O(\log n)$ 。

89. 归并排序

```
// 合并两个有序数组
void merge(vector<int>& nums, int left, int mid, int right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (nums[i] <= nums[j]) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    while (i <= mid) temp[k++] = nums[i++];
    while (j <= right) temp[k++] = nums[j++];
    // 复制回原数组
    for (k = 0; k < temp.size(); k++) {
        nums[left + k] = temp[k];
    }
}

// 归并排序递归函数
void mergeSort(vector<int>& nums, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(nums, left, mid);           // 排序左半部分
    mergeSort(nums, mid + 1, right);      // 排序右半部分
    merge(nums, left, mid, right);        // 合并
}

// 归并排序入口
void mergeSort(vector<int>& nums) {
    mergeSort(nums, 0, nums.size() - 1);
}
```

- 解析：分治法，将数组分成两半分别排序，再合并，时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。

90. 快速排序

```
// 分区函数：返回基准元素的最终位置
int partition(vector<int>& nums, int left, int right) {
    int pivot = nums[right]; // 选择最右元素作为基准
    int i = left - 1;        // 小于基准的元素的边界
    for (int j = left; j < right; j++) {
        if (nums[j] <= pivot) {
            i++;
            swap(nums[i], nums[j]);
        }
    }
    swap(nums[i + 1], nums[right]); // 将基准元素放到正确位置
    return i + 1;
}
```

```
// 快速排序递归函数
void quickSort(vector<int>& nums, int left, int right) {
    if (left < right) {
        int pivot_idx = partition(nums, left, right);
        quickSort(nums, left, pivot_idx - 1); // 排序左半部分
        quickSort(nums, pivot_idx + 1, right); // 排序右半部分
    }
}

// 快速排序入口
void quickSort(vector<int>& nums) {
    quickSort(nums, 0, nums.size() - 1);
}
```

- 解析：分治法，选择基准元素，将数组分为小于和大于基准的两部分，递归排序，平均时间复杂度 $O(n \log n)$ ，最坏 $O(n^2)$ 。

91. 堆排序

```
// 堆调整：维护最大堆性质
void heapify(vector<int>& nums, int n, int i) {
    int largest = i; // 根节点
    int left = 2 * i + 1; // 左孩子
    int right = 2 * i + 2; // 右孩子

    if (left < n && nums[left] > nums[largest]) {
        largest = left;
    }
    if (right < n && nums[right] > nums[largest]) {
        largest = right;
    }

    if (largest != i) {
        swap(nums[i], nums[largest]);
        heapify(nums, n, largest); // 递归调整
    }
}

// 堆排序
void heapSort(vector<int>& nums) {
    int n = nums.size();

    // 构建最大堆
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(nums, n, i);
    }

    // 提取最大元素并调整堆
    for (int i = n - 1; i > 0; i--) {
        swap(nums[0], nums[i]); // 将当前最大元素放到数组末尾
    }
}
```

```
        heapify(nums, i, 0);    // 调整剩余元素为最大堆
    }
}
```

- 解析：先构建最大堆，再反复提取堆顶元素并调整堆，时间复杂度 $O(n \log n)$ ，空间复杂度 $O(1)$ 。

92. 冒泡排序

```
void bubbleSort(vector<int>& nums) {
    int n = nums.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false; // 优化：标记是否发生交换
        // 每轮将最大元素"冒泡"到末尾
        for (int j = 0; j < n - 1 - i; j++) {
            if (nums[j] > nums[j + 1]) {
                swap(nums[j], nums[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break; // 没有交换，说明已排序完成
    }
}
```

- 解析：重复交换相邻的逆序元素，每轮将最大元素移到末尾，时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ 。

93. 选择排序

```
void selectionSort(vector<int>& nums) {
    int n = nums.size();
    for (int i = 0; i < n - 1; i++) {
        // 找到[i..n-1]中的最小值索引
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (nums[j] < nums[min_idx]) {
                min_idx = j;
            }
        }
        // 将最小值交换到位置i
        swap(nums[i], nums[min_idx]);
    }
}
```

- 解析：每轮选择最小元素放到已排序部分的末尾，时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ 。

94. 插入排序

```
void insertionSort(vector<int>& nums) {
    int n = nums.size();
    for (int i = 1; i < n; i++) {
        int key = nums[i];
        int j = i - 1;
        // 将大于key的元素向后移动
        while (j >= 0 && nums[j] > key) {
            nums[j + 1] = nums[j];
            j--;
        }
        // 插入key到正确位置
        nums[j + 1] = key;
    }
}
```

- 解析：将元素逐个插入到已排序部分的正确位置，时间复杂度 $O(n^2)$ ，空间复杂度 $O(1)$ ，适合近乎有序的数据。

95. 计数排序

```
void countingSort(vector<int>& nums) {
    if (nums.empty()) return;
    // 找到最大值和最小值
    int min_val = *min_element(nums.begin(), nums.end());
    int max_val = *max_element(nums.begin(), nums.end());
    int range = max_val - min_val + 1;

    // 计数数组
    vector<int> count(range, 0);
    // 结果数组
    vector<int> output(nums.size());

    // 计数每个元素出现的次数
    for (int num : nums) {
        count[num - min_val]++;
    }

    // 计算前缀和，确定每个元素的位置
    for (int i = 1; i < range; i++) {
        count[i] += count[i - 1];
    }

    // 构建输出数组（从后往前，保持稳定性）
    for (int i = nums.size() - 1; i >= 0; i--) {
        output[count[nums[i] - min_val] - 1] = nums[i];
        count[nums[i] - min_val]--;
    }

    // 复制回原数组
```

```
    nums = output;
}
```

- 解析：非比较排序，适合范围不大的整数排序，时间复杂度 $O(n + \text{range})$ ，空间复杂度 $O(n + \text{range})$ 。

以下是对上一份面试题的补充，继续涵盖经典算法、LeetCode高频题、《编程珠玑》问题及校园招聘热点，总计超过100道题，按模块整理如下：

十一、位运算

41. 位1的个数 (LeetCode 191)

题目：编写函数计算无符号整数二进制表示中1的个数（汉明重量）。

解析：

- 技巧： $n \& (n-1)$ 可消除最后一个1，循环操作直至 n 为0，计数次数即为1的个数。时间 $O(k)$ (k 为1的个数)，空间 $O(1)$ 。

C++代码：

```
int hammingWeight(uint32_t n) {
    int count = 0;
    while (n) {
        n &= n - 1; // 清除最后一个1
        count++;
    }
    return count;
}
```

42. 2的幂 (LeetCode 231)

题目：判断整数 n 是否为2的幂次方。

解析：

- 特性：2的幂的二进制只有一个1，故 $n \& (n-1) == 0$ （需排除 $n=0$ ）。时间 $O(1)$ ，空间 $O(1)$ 。

C++代码：

```
bool isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

43. 两整数之和 (LeetCode 371)

题目：不使用运算符 $+$ 和 $-$ ，计算两整数之和。

解析：

- 位运算： $a \oplus b$ 得无进位和， $(a \& b) \ll 1$ 得进位，循环至进位为0。时间 $O(1)$ （整数位数固定），空间 $O(1)$ 。

C++代码：

```
int getSum(int a, int b) {
    while (b != 0) {
        unsigned int carry = (unsigned int)(a & b) << 1; // 计算进位
        a ^= b; // 无进位和
        b = carry;
    }
    return a;
}
```

十二、回溯算法

44. 全排列 (LeetCode 46)

题目：生成数组所有可能的全排列（元素无重复）。

解析：

- 回溯：通过交换元素实现，固定第 i 位元素（与 $i \sim n-1$ 位交换），递归生成剩余元素的排列。时间 $O(n!)$ ，空间 $O(n)$ （递归栈）。

C++代码：

```
vector<vector<int>>> permute(vector<int>& nums) {
    vector<vector<int>>> res;
    backtrack(nums, 0, res);
    return res;
}

void backtrack(vector<int>& nums, int start, vector<vector<int>>>& res) {
    if (start == nums.size()) {
        res.push_back(nums);
        return;
    }
    for (int i = start; i < nums.size(); ++i) {
        swap(nums[start], nums[i]); // 交换固定当前位
        backtrack(nums, start + 1, res);
        swap(nums[start], nums[i]); // 回溯
    }
}
```

45. 子集 (LeetCode 78)

题目：生成数组所有可能的子集（元素无重复）。

解析：

- 回溯：每个元素可选或不选，递归时记录当前子集。时间 $O(2^n)$ ，空间 $O(n)$ 。

C++代码：

```
vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> res;
    vector<int> path;
    backtrack(nums, 0, path, res);
    return res;
}

void backtrack(vector<int>& nums, int start, vector<int>& path,
vector<vector<int>>& res) {
    res.push_back(path); // 记录当前子集
    for (int i = start; i < nums.size(); ++i) {
        path.push_back(nums[i]);
        backtrack(nums, i + 1, path, res); // 下一个元素从i+1开始（避免重复）
        path.pop_back();
    }
}
```

46. 括号生成 (LeetCode 22)

题目：生成所有有效的n对括号组合。

解析：

- 回溯：保证左括号数量 $\geq$ 右括号，左括号数 $\leq n$ 时可加左括号，右括号数 $<$ 左括号时可加右括号。时间 $O(4^n/\sqrt{n})$ （卡特兰数），空间 $O(n)$ 。

C++代码：

```
vector<string> generateParenthesis(int n) {
    vector<string> res;
    string path;
    backtrack(n, 0, 0, path, res);
    return res;
}

void backtrack(int n, int left, int right, string& path, vector<string>& res) {
    if (path.size() == 2 * n) {
        res.push_back(path);
        return;
    }
    if (left < n) { // 可加左括号
        path.push_back('(');
        backtrack(n, left + 1, right, path, res);
        path.pop_back();
    }
    if (right < left) { // 可加右括号
        path.push_back(')');
    }
}
```

```
        backtrack(n, left, right + 1, path, res);
        path.pop_back();
    }
}
```

十三、数学问题

47. 斐波那契数 (LeetCode 509)

题目：计算第n个斐波那契数 ($F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$)。

解析：

- 迭代：用变量存储前两项，避免递归重复计算。时间 $O(n)$ ，空间 $O(1)$ 。

C++代码：

```
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; ++i) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

48. 质数的个数 (LeetCode 204)

题目：统计小于n的所有质数的数量。

解析：

- 埃氏筛法：标记非质数，从2开始，将其倍数标记为非质数。时间 $O(n \log \log n)$ ，空间 $O(n)$ 。

C++代码：

```
int countPrimes(int n) {
    if (n <= 2) return 0;
    vector<bool> isPrime(n, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i < n; ++i) {
        if (isPrime[i]) {
            // 标记i的倍数为非质数
            for (int j = i * i; j < n; j += i) {
                isPrime[j] = false;
            }
        }
    }
}
```

```
        return count(isPrime.begin(), isPrime.end(), true);  
    }
```

49. x的平方根 (LeetCode 69)

题目：计算x的平方根，返回整数部分（不使用库函数）。

解析：

- 二分查找：在[0, x]范围内找最大的mid使 $mid^2 \leq x$ 。时间 $O(\log x)$ ，空间 $O(1)$ 。

C++代码：

```
int mySqrt(int x) {  
    if (x == 0) return 0;  
    int left = 1, right = x;  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
        int sqrt = x / mid; // 避免mid*mid溢出  
        if (sqrt == mid) {  
            return mid;  
        } else if (sqrt < mid) {  
            right = mid - 1;  
        } else {  
            left = mid + 1;  
        }  
    }  
    return right;  
}
```

十四、《编程珠玑》扩展问题

50. 寻找重复元素 (第二章)

题目：在大小为n的数组中，所有元素均在1~n-1范围内，且至少有一个重复元素，找出任意一个重复元素（不修改数组，空间 $O(1)$ ）。

解析：

- 快慢指针（环形链表）：将数组视为链表 $i \rightarrow \text{nums}[i]$ ，重复元素对应环的入口。时间 $O(n)$ ，空间 $O(1)$ 。

C++代码：

```
int findDuplicate(vector<int>& nums) {  
    int slow = nums[0], fast = nums[nums[0]];  
    // 找相遇点  
    while (slow != fast) {  
        slow = nums[slow];  
        fast = nums[nums[fast]];  
    }
```

```

    }
    // 找环入口
    fast = 0;
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}

```

51. 字符串匹配（第六章）

题目：实现KMP算法，在文本串中查找模式串的位置。

解析：

- 前缀函数（部分匹配表）：预处理模式串得到`next`数组，记录最长前缀后缀匹配长度，匹配失败时按`next`回溯，避免重复比较。时间 $O(m+n)$ ，空间 $O(m)$ （ m 为模式串长度）。

C++代码：

```

int strStr(string haystack, string needle) {
    if (needle.empty()) return 0;
    int n = haystack.size(), m = needle.size();
    // 构建next数组
    vector<int> next(m, 0);
    for (int i = 1, j = 0; i < m; ++i) {
        while (j > 0 && needle[i] != needle[j]) j = next[j-1];
        if (needle[i] == needle[j]) j++;
        next[i] = j;
    }
    // KMP匹配
    for (int i = 0, j = 0; i < n; ++i) {
        while (j > 0 && haystack[i] != needle[j]) j = next[j-1];
        if (haystack[i] == needle[j]) j++;
        if (j == m) return i - m + 1;
    }
    return -1;
}

```

52. 外部排序（第十章）

题目：假设有多个有序文件，如何合并为一个有序文件（内存有限）。

解析：

- 败者树/堆：
 - 每次从每个文件读取一块数据到内存，构建小根堆（或败者树）。
 - 取出堆顶元素写入结果文件，从其来源文件读取下一个元素补充到堆中。
 - 重复直至所有元素处理完毕。
 时间 $O(N \log k)$ （ N 为总元素数， k 为文件数），空间 $O(k)$ 。

十五、LeetCode高频题（续）

53. 合并区间（LeetCode 56）

题目：合并重叠的区间列表（如 $[[1,3],[2,6]] \rightarrow [[1,6]]$ ）。

解析：

- 排序+遍历：按区间起点排序，遍历合并重叠或相邻区间。时间 $O(n \log n)$ （排序），空间 $O(n)$ （存储结果）。

C++代码：

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};
    // 按起点排序
    sort(intervals.begin(), intervals.end(), [](const auto& a, const auto& b) {
        return a[0] < b[0];
    });
    vector<vector<int>> res;
    res.push_back(intervals[0]);
    for (int i = 1; i < intervals.size(); ++i) {
        auto& last = res.back();
        if (intervals[i][0] <= last[1]) { // 重叠, 合并
            last[1] = max(last[1], intervals[i][1]);
        } else { // 不重叠, 加入结果
            res.push_back(intervals[i]);
        }
    }
    return res;
}
```

54. 最小路径和（LeetCode 64）

题目：在 $m \times n$ 网格中，从左上角到右下角的路径（只能右/下移动）的最小路径和。

解析：

- DP: $dp[i][j]$ 为到 (i,j) 的最小和, $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$ 。空间优化为 $O(n)$ 。时间 $O(mn)$ ，空间 $O(n)$ 。

C++代码：

```
int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    vector<int> dp(n);
    dp[0] = grid[0][0];
    // 第一行
    for (int j = 1; j < n; ++j) {
        dp[j] = dp[j-1] + grid[0][j];
    }
}
```

```
// 其余行
for (int i = 1; i < m; ++i) {
    dp[0] += grid[i][0]; // 第一列
    for (int j = 1; j < n; ++j) {
        dp[j] = grid[i][j] + min(dp[j], dp[j-1]);
    }
}
return dp[n-1];
}
```

55. 买卖股票的最佳时机 (LeetCode 121)

题目：只能买卖一次，求最大利润。

解析：

- 贪心：记录最低买入价，遍历计算当前卖出利润，更新最大值。时间 $O(n)$ ，空间 $O(1)$ 。

C++代码：

```
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for (int price : prices) {
        minPrice = min(minPrice, price);
        maxProfit = max(maxProfit, price - minPrice);
    }
    return maxProfit;
}
```

56. 相交链表 (LeetCode 160)

题目：找到两个链表相交的起始节点。

解析：

- 双指针：p1从headA出发，p2从headB出发，到达尾部后切换到对方头部，相遇点即为相交节点（若不相交，最终都为null）。时间 $O(m+n)$ ，空间 $O(1)$ 。

C++代码：

```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
    if (!headA || !headB) return nullptr;
    ListNode *p1 = headA, *p2 = headB;
    while (p1 != p2) {
        p1 = p1 ? p1->next : headB;
        p2 = p2 ? p2->next : headA;
    }
    return p1;
}
```

57. 二叉树的直径 (LeetCode 543)

题目：计算二叉树的直径（任意两节点路径长度的最大值，路径可不过根）。

解析：

- 递归：直径=左子树深度+右子树深度，递归计算每个节点的深度并更新最大直径。时间 $O(n)$ ，空间 $O(n)$ （递归栈）。

C++代码：

```
int diameterOfBinaryTree(TreeNode* root) {
    int maxDiameter = 0;
    depth(root, maxDiameter);
    return maxDiameter;
}

int depth(TreeNode* node, int& maxDiameter) {
    if (!node) return 0;
    int left = depth(node->left, maxDiameter);
    int right = depth(node->right, maxDiameter);
    maxDiameter = max(maxDiameter, left + right); // 更新直径
    return 1 + max(left, right); // 返回深度
}
```

58. 多数元素 (LeetCode 169)

题目：找出数组中出现次数 $> \lfloor n/2 \rfloor$ 的元素。

解析：

- 摩尔投票法：遍历数组，相同元素计数+1，不同-1，计数为0时更换候选元素，最终候选元素即为多数。时间 $O(n)$ ，空间 $O(1)$ 。

C++代码：

```
int majorityElement(vector<int>& nums) {
    int candidate = nums[0], count = 1;
    for (int i = 1; i < nums.size(); ++i) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else {
            count += (nums[i] == candidate) ? 1 : -1;
        }
    }
    return candidate;
}
```

59. 零钱兑换 (LeetCode 322)

题目：用最少的硬币数凑成金额amount，硬币面额为coins数组。

解析：

- DP: $dp[i]$ 为凑成i的最少硬币数, $dp[i] = \min(dp[i - coin] + 1) \text{ (coin} \leq i \text{)}$ 。时间 $O(\text{amount} \times k)$ (k为硬币种类), 空间 $O(\text{amount})$ 。

C++代码：

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, amount + 1); // 初始化为不可能的值
    dp[0] = 0; // 0金额需0硬币
    for (int i = 1; i <= amount; ++i) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

60. 课程表 (LeetCode 207)

题目：判断是否能完成所有课程（存在先修课程依赖，如[0,1]表示修0前需先修1）。

解析：

- 拓扑排序 (BFS)：统计每个节点的入度，入度为0的节点入队，遍历并减少邻接节点入度，最后检查完成的课程数是否等于总课程数。时间 $O(n+m)$ (n课程数, m依赖数), 空间 $O(n+m)$ 。

C++代码：

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> adj(numCourses); // 邻接表
    vector<int> inDegree(numCourses, 0);
    // 构建图
    for (auto& p : prerequisites) {
        int course = p[0], pre = p[1];
        adj[pre].push_back(course);
        inDegree[course]++;
    }
    // 入度为0的节点入队
    queue<int> q;
    for (int i = 0; i < numCourses; ++i) {
        if (inDegree[i] == 0) q.push(i);
    }
    // BFS
    int count = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        count++;
        for (int v : adj[u]) {
            inDegree[v]--;
            if (inDegree[v] == 0) q.push(v);
        }
    }
    return count == numCourses;
}
```

```
        count++;
        for (int v : adj[u]) {
            if (--inDegree[v] == 0) {
                q.push(v);
            }
        }
    }
    return count == numCourses;
}
```

十六、C++进阶特性

61. 模板特化与偏特化

解析：

- 模板特化：为特定类型提供定制实现，如`template<> class A<int> { ... }`。
- 偏特化：为部分参数提供定制，如`template<typename T> class A<T*>`（指针类型特化）。

示例代码：

```
// 通用模板
template<typename T>
struct IsPointer {
    static const bool value = false;
};

// 偏特化（指针类型）
template<typename T>
struct IsPointer<T*> {
    static const bool value = true;
};

// 使用
cout << IsPointer<int>::value << endl; // 0
cout << IsPointer<int*>::value << endl; // 1
```

62. 右值引用与移动语义

解析：

- 右值引用（`&&`）：绑定临时对象（右值），用于实现移动语义。
- 移动构造/赋值：转移资源所有权，避免深拷贝，提高性能（如`vector`的`push_back`传入临时对象时）。

示例代码：

```
class MyString {
private:
    char* data;
```

```
public:
    // 移动构造函数
    MyString(MyString&& other) noexcept : data(other.data) {
        other.data = nullptr; // 原对象释放资源
    }
    // 移动赋值运算符
    MyString& operator=(MyString&& other) noexcept {
        if (this != &other) {
            delete[] data;
            data = other.data;
            other.data = nullptr;
        }
        return *this;
    }
    // ... 其他成员
};
```

63. 内存对齐

解析：

- 规则：变量地址需是其对齐数的倍数（通常为类型大小或编译器默认值），结构体对齐数为成员最大对齐数。
- 作用：提高CPU访问效率，避免跨总线访问。

示例：

```
struct A {
    char c; // 1字节, 对齐1
    int i;   // 4字节, 对齐4 → 结构体对齐4, 总大小8 (c后补3字节)
};
cout << sizeof(A) << endl; // 输出8
```

十七、更多高频算法题（补充至100+）

64. 最长连续序列 (LeetCode 128)

65. 螺旋矩阵 (LeetCode 54)

66. 编辑距离 (LeetCode 72)

67. 打家劫舍 (LeetCode 198)

68. 单词搜索 (LeetCode 79)

69. 最大矩形 (LeetCode 85)

70. 验证二叉搜索树 (LeetCode 98)

- 71. 二叉树的最近公共祖先 (LeetCode 236)
- 72. 组合总和 (LeetCode 39)
- 73. 接雨水 (LeetCode 42)
- 74. 寻找峰值 (LeetCode 162)
- 75. 分割回文串 (LeetCode 131)
- 76. 颜色分类 (LeetCode 75)
- 77. 目标和 (LeetCode 494)
- 78. 滑动窗口最大值 (LeetCode 239)
- 79. 不同路径 (LeetCode 62)
- 80. 验证回文串 (LeetCode 125)
- 81. 二叉搜索树中第K小的元素 (LeetCode 230)
- 82. 移除链表元素 (LeetCode 203)
- 83. 字符串相加 (LeetCode 415)
- 84. 移除元素 (LeetCode 27)
- 85. 有效的数独 (LeetCode 36)
- 86. 最大数 (LeetCode 179)
- 87. 有序矩阵中第K小的元素 (LeetCode 378)
- 88. 两数相除 (LeetCode 29)
- 89. 字符串的排列 (LeetCode 567)
- 90. 把二叉搜索树转换为累加树 (LeetCode 538)
- 91. 丑数 (LeetCode 263)
- 92. 找不同 (LeetCode 389)
- 93. 存在重复元素 (LeetCode 217)
- 94. 找零 (LeetCode 322变种)
- 95. 二进制手表 (LeetCode 401)

96. 区间子数组个数 (LeetCode 795)

97. 回文链表 (LeetCode 234)

98. 链表的中间结点 (LeetCode 876)

99. 路径总和 (LeetCode 112)

100. 二叉树的层平均值 (LeetCode 637)

101. 最大深度 (LeetCode 104)

102. 对称二叉树 (LeetCode 101)

总结

以上100+道题覆盖了C/C++算法开发工程师面试的核心考点，包括：

1. **基础算法**：排序、查找、递归、回溯、贪心、动态规划等。
2. **数据结构**：数组、链表、栈、队列、树（二叉树、BST）、图、哈希表等。
3. **编程技巧**：位运算、双指针、滑动窗口、前缀和等。
4. **C++特性**：STL、智能指针、模板、内存管理、移动语义等。
5. **经典问题**：《编程珠玑》中的优化问题、LeetCode高频题。

嵌入式开发

一、单片机基础 (1-20题)

1. 单片机 (MCU) 的基本组成部分有哪些？

- **解析**：单片机是将CPU、存储器、外设集成在单一芯片上的微型计算机，核心组成包括：
 - **中央处理器 (CPU)**：执行指令（如8051的8位CPU、ARM Cortex-M的32位CPU）；
 - **存储器**：程序存储器（ROM/Flash）和数据存储器（RAM）；
 - **定时器/计数器**：用于定时、计数功能；
 - **中断系统**：处理外部/内部事件的异步响应；
 - **I/O接口**：GPIO（通用输入输出）；
 - **通信外设**：UART、SPI、I2C等；
 - **特殊外设**：ADC（模数转换）、DAC（数模转换）、PWM（脉冲宽度调制）等。

2. 哈佛架构与冯·诺依曼架构的区别？单片机通常采用哪种？

- **解析**：
 - **冯·诺依曼架构**：程序和数据共用同一地址空间和总线，读写指令和数据需分时进行（如早期计算机）；
 - **哈佛架构**：程序存储器和数据存储器分离，有独立总线，可同时读写指令和数据，执行效率更高。
- 单片机（如8051、STM32）普遍采用**哈佛架构**，适合实时控制场景。

3. 单片机的复位电路作用及常见类型？

- **作用：**使单片机从初始状态开始运行（清除寄存器、PC指针归零）。
- **常见类型：**
 - **上电复位：**电源上电时通过电容充电产生复位信号；
 - **手动复位：**通过按键强制拉低/拉高复位引脚；
 - **** watchdog复位**：**程序跑飞时， watchdog定时器溢出触发复位。

4. 单片机的时钟源有哪些？各自特点？

- **解析：**
 - **内部RC振荡器：**成本低，无需外部元件，但精度低（误差 $\pm 10\%$ ）；
 - **外部晶体振荡器（晶振）：**精度高（误差ppm级），适合需要精确定时/通信的场景（如UART波特率）；
 - **外部RC振荡器：**精度介于内部RC和晶振之间，成本适中；
 - **PLL（锁相环）：**将低频时钟倍频为高频（如STM32用8MHz晶振经PLL倍频到72MHz）。

5. GPIO的工作模式有哪些？（以8051或STM32为例）

- **8051 GPIO：**基本输入（高阻）、输出（推挽）；
- **STM32 GPIO：**更丰富，包括：
 - 输入模式：浮空输入、上拉输入、下拉输入、模拟输入；
 - 输出模式：推挽输出（高低电平都能驱动负载）、开漏输出（仅低电平驱动，高电平需外部上拉，适合I2C等线与场景）；
 - 复用功能：将GPIO复用为外设引脚（如UART_TX、SPI_SCK）。

6. 什么是中断？中断的处理流程是什么？

- **中断：**外部/内部事件（如按键、定时器溢出）打断CPU当前任务，转而去处理事件的机制，提高实时性。
- **处理流程：**
 1. **中断请求：**外设产生中断信号；
 2. **中断响应：**CPU暂停当前指令，保存上下文（PC、寄存器）；
 3. **中断服务程序（ISR）：**执行中断处理代码；
 4. **中断返回：**恢复上下文，继续执行原任务。

7. 中断优先级的作用？如何设置中断优先级？

- **作用：**当多个中断同时发生时，CPU优先处理高优先级中断。
- **设置方式：**
 - **硬件优先级：**部分单片机有固定优先级（如8051的外部中断0 > 定时器0 > 外部中断1 > 定时器1）；
 - **软件配置：**如STM32通过NVIC（嵌套向量中断控制器）设置抢占优先级和子优先级，数值越小优先级越高。

8. 定时器/计数器的工作模式有哪些？应用场景？

- **工作模式：**

- **定时模式**：通过内部时钟计数，用于产生固定延时（如1ms定时）；
- **计数模式**：通过外部引脚输入脉冲计数（如统计脉冲个数）；
- **PWM模式**：输出占空比可调的脉冲（如电机调速、LED调光）；
- **输入捕获**：测量外部脉冲的频率/周期（如测频、红外解码）。
- **应用场景**：延时函数、周期性任务调度、电机控制、频率测量等。

9. UART、SPI、I2C的区别及应用场景？

特性	UART（异步串行）	SPI（同步串行）	I2C（同步串行）
信号线	TX、RX（共地）	SCK、MOSI、MISO、CS	SDA、SCL（上拉电阻）
同步方式	异步（波特率匹配）	同步（SCK时钟线）	同步（SCL时钟线）
通信方向	全双工	全双工	半双工
设备数量	点对点（多机需额外控制）	一主多从（通过CS选从）	一主多从（通过地址）
速率	较低（如115200bps）	较高（如几十Mbps）	中等（如400kbps、1Mbps）
应用场景	调试打印、GPS模块	显示屏、Flash存储	传感器（温湿度、加速度）

10. ADC的主要参数及作用？

- **主要参数**：
 - **分辨率**：转换结果的位数（如12位ADC，可区分 $2^{12}=4096$ 个电平）；
 - **量程**：可测量的电压范围（如0-3.3V）；
 - **转换速率**：每秒完成转换的次数（如1MHz表示每秒100万次）；
 - **精度**：实际值与理论值的偏差（通常用LSB表示）。
- **作用**：将模拟信号（如传感器输出的电压）转换为数字信号，供CPU处理。

11. 什么是PWM？占空比和频率对输出的影响？

- **PWM（脉冲宽度调制）**：通过周期性高低电平切换，用占空比表示模拟量的数字信号。
- **占空比**：高电平时间占周期的比例（如50%占空比表示高电平与低电平时间相等），决定输出平均电压（如3.3V电源，50%占空比平均1.65V）；
- **频率**：每秒脉冲数，频率过低会导致输出抖动（如LED闪烁），过高可能超出外设响应速度（如电机驱动需匹配电机特性）。

12. 单片机低功耗模式的实现方式？

- **核心思路**：关闭不使用的外设、降低时钟频率、进入休眠模式。
- **常见措施**：
 - 关闭未使用的外设时钟（如ADC、SPI）；
 - 降低CPU主频（如从80MHz降至16MHz）；
 - 进入休眠模式（如STM32的STOP模式、STANDBY模式，仅保留必要电路供电）；
 - 使用外部中断唤醒（如按键、RTC闹钟）。

13. 什么是看门狗（Watchdog）？作用是什么？

- **看门狗**：一个定时器，需CPU定期喂狗（重置计数器），若超时未喂狗则触发复位。
- **作用**：防止程序跑飞（如死循环、异常），使系统自动恢复。
- **类型**：独立看门狗（IWDG，用内部低速时钟，不受主时钟影响）、窗口看门狗（WWDG，有时钟窗口限制，更安全）。

14. 单片机中常用的存储类型有哪些？各自用途？

- **Flash（程序存储器）**：存放程序代码、常量（掉电不丢失）；
- **RAM（随机存储器）**：存放变量、堆栈（掉电丢失，读写速度快）；
- **EEPROM**：存放需要掉电保存的用户数据（如配置参数，可擦写次数多）；
- **寄存器**：CPU内部的高速存储，用于临时数据处理、外设备置。

15. 什么是堆栈？单片机中堆栈的作用？

- **栈**：先进后出（FILO）的内存区域，用于：
 - 保存函数调用时的返回地址；
 - 传递函数参数；
 - 保存局部变量；
 - 中断发生时保存上下文。
- **堆**：动态分配的内存区域（需手动申请/释放），用于存储不确定大小的数据（如链表节点）。
- **注意**：单片机RAM较小，堆使用需谨慎，避免溢出。

16. 如何实现单片机的延时函数？

- **软件延时**：通过空循环消耗CPU时间（简单但精度低，受主频影响）：

```
void delay_ms(uint32_t ms) {
    for (uint32_t i = 0; i < ms; i++)
        for (uint32_t j = 0; j < 72000; j++); // 假设72MHz主频下约1ms
}
```

- **定时器延时**：利用定时器中断（精度高，不阻塞CPU）：

```
uint32_t delay_flag = 0;
void TIM_IRQHandler() {
    if (TIM_GetITStatus(TIM2, TIM_IT_Update)) {
        delay_flag = 1;
        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}
void delay_ms(uint32_t ms) {
    delay_flag = 0;
    TIM_SetCounter(TIM2, 0);
    TIM_EnableIT(TIM2, TIM_IT_Update);
    while (!delay_flag); // 等待定时器中断
}
```

17. 单片机中常用的调试方式有哪些？

- **仿真器调试：**通过J-Link、ST-Link连接单片机，单步执行、断点调试、查看寄存器/内存；
- **打印调试：**通过UART输出调试信息到上位机（如串口助手）；
- **LED/蜂鸣器指示：**用硬件状态表示程序运行阶段（如初始化完成亮LED）；
- **示波器/逻辑分析仪：**测量引脚电平变化（如PWM波形、通信时序）。

18. 什么是总线？单片机中常见的总线有哪些？

- **总线：**连接CPU与外设的公共信号线，用于数据、地址、控制信号的传输。
- **常见总线：**
 - 内部总线：CPU与片内外设的连接（如STM32的AHB、APB总线）；
 - 外部总线：扩展外部设备（如地址总线、数据总线、控制总线）；
 - 通信总线：UART、SPI、I2C（见题9）。

19. 如何提高单片机系统的抗干扰能力？

- **硬件措施：**
 - 电源滤波（加电容、电感）；
 - 信号线加粗、短接，远离干扰源（如电机）；
 - 接地处理（单点接地、模拟地与数字地分离）；
 - 关键引脚加下拉/上拉电阻、TVS管（防浪涌）。
- **软件措施：**
 - 数字信号多次采样（去抖）；
 - 数据传输加校验（CRC、校验和）；
 - 使用看门狗复位异常；
 - 关键代码加冗余（如双备份）。

20. 51单片机与STM32的主要区别？

特性	51单片机	STM32（以F1为例）
内核	8位CPU（如Intel 8051）	32位Cortex-M3
主频	通常≤12MHz	最高72MHz
存储器	小容量（如4KB Flash）	大容量（如512KB Flash）
外设	基础（UART、定时器）	丰富（DMA、ADC、SPI等）
架构	哈佛架构	哈佛架构
开发方式	汇编/C，寄存器配置	C/C++，库函数/HAL
应用场景	简单控制（如玩具、小家电）	复杂嵌入式系统（工业控制、物联网）

二、STM32开发（21-40题）

21. STM32的主要系列及应用场景？

- **F1系列：**基础型，Cortex-M3内核，主频72MHz，适合中低端控制（如家电、传感器节点）；

- **F4系列**：高性能型，Cortex-M4内核（带FPU），主频168MHz，适合需要浮点运算、高速外设的场景（如电机控制、音频处理）；
- **F7系列**：高性能型，Cortex-M7内核，主频216MHz，带Cache，适合图形显示、高速数据处理；
- **H7系列**：超高性能型，Cortex-M7内核，主频400MHz，适合复杂工业控制、AI边缘计算；
- **L系列**：低功耗型，Cortex-M0+/M4，适合电池供电设备（如穿戴设备、物联网传感器）。

22. STM32的时钟树结构？PLL的作用？

- **时钟树**：STM32时钟源经分频/倍频后分配给CPU和外设，核心路径：外部高速时钟（HSE）/内部高速时钟（HSI）→ PLL（锁相环）→ 系统时钟（SYSCLK）→ 分频给AHB总线、APB1、APB2外设。
- **PLL作用**：将低频时钟（如8MHz HSE）倍频为高频（如72MHz、168MHz），提升CPU和外设运行速度。

23. STM32的GPIO配置步骤（以HAL库为例）？

1. 使能GPIO端口时钟（如`__HAL_RCC_GPIOA_CLK_ENABLE();`）；
2. 定义GPIO初始化结构体（`GPIO_InitTypeDef GPIO_InitStruct`）；
3. 配置结构体参数（引脚Pin、模式Mode、速度Speed、上下拉Pull）；
4. 调用初始化函数（`HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);`）。

24. STM32的中断系统（NVIC）的核心功能？

- **NVIC（嵌套向量中断控制器）**是Cortex-M内核的中断管理组件，功能包括：
 - 中断优先级管理（支持抢占优先级和子优先级）；
 - 中断使能/禁用；
 - 中断嵌套（高优先级中断可打断低优先级中断）；
 - 中断向量表（存储中断服务程序入口地址）。

25. STM32的DMA控制器作用及优势？

- **DMA（直接存储器访问）**：无需CPU干预，直接在外设与存储器（或存储器之间）传输数据。
- **优势**：解放CPU，提高数据传输效率（如ADC采集、SPI通信时，CPU可处理其他任务）。
- **常用场景**：ADC多通道采集、UART大数据量收发、SPI读取Flash。

26. STM32的ADC工作模式有哪些？如何配置多通道采集？

- **工作模式**：单次转换、连续转换、扫描模式、间断模式。
- **多通道采集配置（扫描模式）**：
 1. 使能ADC和GPIO时钟；
 2. 配置ADC通道对应的GPIO为模拟输入；
 3. 配置ADC初始化结构体（扫描模式`ScanConvMode = ENABLE`，连续转换`ContinuousConvMode = ENABLE`）；
 4. 配置通道序列（`ADC_ChannelConfTypeDef`，设置通道、采样时间、序列编号）；
 5. 启动ADC（`HAL_ADC_Start_DMA(&hadc1, (uint32_t*)&adc_buf, 通道数);`），通过DMA传输数据到缓冲区。

27. STM32的定时器有哪些类型？各有什么特点？

- **基本定时器 (TIM6、TIM7)**：仅支持定时和触发DAC，无GPIO输出；
- **通用定时器 (TIM2-TIM5、TIM9-TIM14)**：支持定时、计数、PWM、输入捕获，功能丰富；
- **高级定时器 (TIM1、TIM8)**：在通用定时器基础上，支持互补PWM（带死区控制）、刹车功能，适合电机驱动（如三相电机）。

28. STM32的UART如何配置中断接收？（以HAL库为例）

1. 使能UART和GPIO时钟；
2. 配置UART引脚（TX为复用推挽，RX为浮空/上拉输入）；
3. 初始化UART结构体（波特率、数据位、停止位、校验位）；
4. 使能UART接收中断（`__HAL_UART_ENABLE_IT(&huart1, UART_IT_RXNE);`）；
5. 在中断服务程序（`USART1_IRQHandler`）中调用HAL库回调（`HAL_UART_IRQHandler(&huart1);`）；
6. 重写接收回调函数（`void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)`）处理数据。

29. STM32的SPI通信中，NSS引脚的作用？硬件NSS与软件NSS的区别？

- **NSS（从设备选择）**：用于SPI主设备选择从设备（低电平有效）。
- **硬件NSS**：使用专用NSS引脚（如SPI1_NSS），主设备输出NSS信号自动控制从设备；
- **软件NSS**：通过普通GPIO模拟NSS信号，主设备手动控制GPIO高低电平选择从设备（更灵活，适合多从设备场景）。

30. STM32的I2C通信中，如何处理 arbitration lost（仲裁丢失）？

- **仲裁丢失**：I2C总线多主设备通信时，两主设备同时发送数据，总线判定优先级低的设备丢失仲裁。
- **处理方法**：
 1. 检测到仲裁丢失标志（`I2C_FLAG_ARLO`）；
 2. 清除标志位，复位I2C外设；
 3. 重新发起通信（通常加入重试机制，避免频繁冲突）。

31. STM32的Bootloader（启动程序）的作用？启动模式有哪些？

- **Bootloader作用**：芯片上电后首先运行的程序，用于选择启动方式（加载用户程序或进入ISP模式）。
- **启动模式**（由BOOT0和BOOT1引脚电平决定）：
 - BOOT0=0：从主Flash启动（正常运行用户程序）；
 - BOOT0=1, BOOT1=0：从系统存储器启动（通过UART/SPI下载程序，ISP模式）；
 - BOOT0=1, BOOT1=1：从SRAM启动（用于调试）。

32. STM32的HAL库、标准外设库（SPL）、LL库的区别？

- **标准外设库 (SPL)**：早期库，按外设模块化设计（如`stm32f10x_gpio.c`），函数名如`GPIO_Init()`，已逐步淘汰；
- **HAL库（硬件抽象层）**：跨系列兼容（同一函数适用于F1/F4/H7），封装程度高，函数名如`HAL_GPIO_Init()`，适合快速开发，但效率略低；
- **LL库（底层库）**：轻量级，接近寄存器操作，效率高，函数名如`LL_GPIO_Init()`，适合对性能要求高的场景，可与HAL库混合使用。

33. STM32CubeMX的作用？使用流程？

- **作用：**图形化配置工具，用于生成STM32初始化代码（引脚配置、时钟树、外设初始化），支持HAL/LL库，可导出到Keil、IAR、Makefile等开发环境。
- **使用流程：**
 1. 选择STM32型号（如STM32F103C8T6）；
 2. 配置引脚功能（GPIO、UART、SPI等）；
 3. 配置时钟树（HSE、PLL、外设时钟）；
 4. 配置外设参数（如UART波特率、定时器频率）；
 5. 生成工程代码（选择开发环境）；
 6. 在生成的工程中编写应用逻辑。

34. STM32中如何实现低功耗模式？（以STOP模式为例）

1. 关闭不使用的外设时钟；
2. 配置唤醒源（如外部中断、RTC闹钟）；
3. 调用HAL库低功耗函数：

```
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI); // 进入STOP模式，等待中断唤醒
```

4. 唤醒后需重新配置时钟（STOP模式下PLL关闭，需重新使能）。

35. STM32的RTC（实时时钟）作用及配置步骤？

- **作用：**提供独立于主时钟的实时时间（年/月/日/时/分/秒），支持闹钟、周期性中断，适合低功耗场景（由备用电源供电）。
- **配置步骤：**
 1. 使能PWR和BKP时钟，允许访问备用寄存器（`HAL_PWR_EnableBkUpAccess();`）；
 2. 配置RTC时钟源（LSE外部低速晶振或LSI内部低速时钟）；
 3. 初始化RTC（设置分频系数，使RTC时钟为1Hz）；
 4. 设置时间（`HAL_RTC_SetTime()`）和日期（`HAL_RTC_SetDate()`）；
 5. （可选）配置闹钟中断（`HAL_RTC_SetAlarm_IT()`）。

36. STM32中使用FreeRTOS的基本步骤？

1. 通过CubeMX使能FreeRTOS（选择CMSIS-RTOS v2接口）；
2. 创建任务（`osThreadNew()`），定义任务函数（`void task1(void *argument)`）；
3. 配置任务优先级、栈大小；
4. （可选）创建信号量、队列等同步机制；
5. 在`main()`函数中启动调度器（`osKernelStart()`）；
6. 任务函数中通过`osDelay()`让出CPU，或通过同步机制阻塞/唤醒。

37. STM32的GPIO引脚复用如何配置？

- **步骤：**
 1. 使能对应外设时钟（如复用为UART1_TX，需使能USART1时钟）；
 2. 配置GPIO为复用功能模式（`GPIO_Mode_AF_PP`或`GPIO_Mode_AF_OD`）；

3. (部分型号) 配置复用映射 (如STM32F4需通过`GPIO_PinAFConfig()`指定复用功能)。

38. STM32的硬件断点和软件断点的区别?

- **硬件断点**: 利用CPU的调试寄存器实现, 数量有限 (如Cortex-M3支持6个), 可在任意地址设置, 适合ROM/Flash代码;
- **软件断点**: 通过在断点地址插入特殊指令 (如ARM的`BKPT`) 实现, 数量无限制, 但仅适用于可修改的内存 (RAM), 对Flash代码需先下载到RAM。

39. STM32的ADC校准的作用及步骤?

- **作用**: 消除ADC的偏移误差和增益误差, 提高转换精度。
- **步骤**:
 1. 使能ADC电源;
 2. 启动校准 (`HAL_ADCEx_Calibration_Start(&hadc1);`);
 3. 等待校准完成 (HAL库函数会阻塞直到完成);
 4. 校准完成后再启动ADC转换。

40. STM32中如何实现SPI从模式?

1. 配置SPI为从模式 (`SPI_Mode = SPI_Mode_Slave`);
2. 配置NSS引脚为输入 (硬件NSS或软件模拟);
3. 使能SPI接收中断 (`SPI_IT_RXNE`);
4. 当主设备发送数据时, 从设备通过中断接收数据, 并可通过`SPI_I2S_SendData()`发送应答数据。

三、嵌入式Linux开发 (41-65题)

41. 嵌入式Linux系统的组成部分?

- **引导程序 (Bootloader)**: 如U-Boot, 初始化硬件、加载内核;
- **Linux内核**: 进程管理、内存管理、设备驱动、文件系统等核心功能;
- **根文件系统 (RootFS)**: 包含应用程序、库、配置文件 (如/bin、/lib、/etc);
- **应用程序**: 用户开发的业务逻辑程序。

42. Bootloader的作用? U-Boot的主要功能?

- **Bootloader作用**: 系统上电后第一个运行的程序, 负责:
 - 硬件初始化 (CPU、内存、时钟、串口);
 - 加载内核到内存并启动;
 - 提供命令行接口 (用于烧写系统、修改启动参数)。
- **U-Boot功能**: 支持多架构 (ARM、x86)、命令集丰富 (`bootm`启动内核、`tftp`下载文件)、可配置性强 (通过Kconfig)、支持脚本 (自动启动)。

43. 嵌入式Linux的启动流程?

1. **上电复位**: CPU执行固化在ROM中的初始程序 (如SPL, 第二阶段Bootloader);
2. **Bootloader初始化**: U-Boot初始化内存、外设, 设置内核启动参数;
3. **加载内核**: U-Boot将内核镜像 (zImage/uImage) 从Flash加载到RAM;

4. **启动内核**：跳转到内核入口地址，内核解压、初始化（进程管理、内存管理、驱动）；
5. **挂载根文件系统**：内核启动init进程，挂载根文件系统（通过`root=`参数指定）；
6. **启动用户程序**：init进程按`/etc/inittab`或`systemd`配置启动应用程序。

44. 交叉编译的概念及作用？如何配置交叉编译工具链？

- **交叉编译**：在主机（如x86 PC）上编译目标机（如ARM嵌入式设备）可执行程序的过程。
- **作用**：嵌入式设备资源有限（CPU弱、存储小），无法本地编译大型程序。
- **配置方法**：
 - 下载交叉工具链（如`arm-linux-gnueabi-gcc`）；
 - 配置环境变量：`export PATH=$PATH:/path/to/toolchain/bin`；
 - 编译时指定目标架构：`arm-linux-gnueabi-gcc main.c -o main`。

45. 常用的嵌入式Linux构建工具？（Buildroot、Yocto）

- **Buildroot**：轻量级构建工具，通过Kconfig配置（类似Linux内核），一键生成Bootloader、内核、根文件系统，适合快速原型开发；
- **Yocto Project**：功能强大，支持定制化程度高，生成的系统更规范，但学习曲线陡，适合复杂产品开发。

46. Linux文件系统的目录结构及各目录作用？

- **/bin**：基本用户命令（如`ls`、`cp`）；
- **/sbin**：系统管理命令（如`reboot`、`ifconfig`）；
- **/etc**：配置文件（如`/etc/passwd`、`/etc/network/interfaces`）；
- **/lib**：共享库（如`libc.so`）和内核模块；
- **/dev**：设备文件（如`/dev/ttyS0`串口、`/dev/mmcblk0`SD卡）；
- **/proc**：虚拟文件系统，反映系统状态（如`/proc/cpuinfo`、`/proc/meminfo`）；
- **/sys**：虚拟文件系统，用于访问硬件设备（如`/sys/class/gpio`）；
- **/root**：root用户的家目录；
- **/home**：普通用户的家目录；
- **/tmp**：临时文件目录（重启后清空）。

47. 嵌入式Linux中常用的文件系统类型？

- **JFFS2**：日志型文件系统，适合NOR Flash，支持磨损平衡、断电恢复；
- **YAFFS2**：专为NAND Flash设计，支持大页NAND，磨损平衡；
- **EXT4**：通用文件系统，支持大容量、日志功能，适合SD卡、eMMC；
- **SquashFS**：只读压缩文件系统，节省存储空间，适合存放根文件系统；
- **tmpfs**：基于内存的文件系统，速度快，用于`/tmp`目录（掉电丢失）。

48. Linux进程与线程的区别？

- **进程**：资源分配的基本单位，有独立的地址空间、文件描述符等，进程间通信（IPC）需特殊机制（管道、共享内存）；
- **线程**：调度的基本单位，共享所属进程的资源（地址空间、文件描述符），线程间通信通过共享内存即可，开销小。
- **关系**：一个进程可包含多个线程，线程是进程的一部分。

49. Linux中常用的IPC（进程间通信）方式？

- **管道 (Pipe)**：半双工，用于父子进程通信；
- **命名管道 (FIFO)**：可用于无亲缘关系进程通信，有文件名；
- **信号 (Signal)**：用于通知进程事件（如SIGINT中断、SIGKILL终止）；
- **共享内存 (Shared Memory)**：多个进程共享同一块内存，速度最快；
- **消息队列 (Message Queue)**：按消息类型传递数据，异步通信；
- **信号量 (Semaphore)**：用于同步和互斥（如保护共享资源）；
- **套接字 (Socket)**：可用于同一主机或不同主机的进程通信（网络编程）。

50. Linux线程同步的方式有哪些？

- **互斥锁 (Mutex)**：pthread_mutex_t，保证同一时间只有一个线程访问共享资源；
- **条件变量 (Condition Variable)**：pthread_cond_t，用于线程间等待/通知（如生产者-消费者模型）；
- **信号量 (Semaphore)**：sem_t，可用于限制同时访问资源的线程数；
- **读写锁 (Read-Write Lock)**：pthread_rwlock_t，允许多个读线程同时访问，写线程独占。

51. 什么是Linux的动态链接和静态链接？区别？

- **静态链接**：编译时将库函数代码直接嵌入可执行文件，优点是不依赖外部库，缺点是文件体积大、更新库需重新编译；
- **动态链接**：编译时仅记录库引用，运行时加载共享库（.so文件），优点是文件体积小、库更新无需重新编译，缺点是依赖库存在。
- **编译方式**：静态链接用-static选项（gcc main.c -o main -static），动态链接默认。

52. Linux中如何查看进程状态？常用命令？

- **ps**：查看进程快照（ps aux显示所有进程，ps -ef显示进程关系）；
- **top**：实时查看进程资源占用（CPU、内存），按q退出；
- **pstree**：以树状显示进程间关系；
- **进程状态**：R（运行）、S（睡眠）、D（不可中断睡眠）、Z（僵尸）、T（停止）。

53. Linux中如何查看和配置网络？

- **查看IP地址**：ifconfig 或 ip addr；
- **查看路由表**：route -n 或 ip route；
- **查看网络连接**：netstat -tuln 或 ss -tuln；
- **配置IP**：ifconfig eth0 192.168.1.100 netmask 255.255.255.0 或 ip addr add 192.168.1.100/24 dev eth0；
- **配置默认网关**：route add default gw 192.168.1.1 或 ip route add default via 192.168.1.1；
- **永久配置**：修改/etc/network/interfaces（Debian系）或/etc/sysconfig/network-scripts/ifcfg-eth0（RedHat系）。

54. 嵌入式Linux中如何实现开机自启动程序？

- **方法1**：修改/etc/rc.local（在exit 0前添加程序路径，如/usr/bin/myapp &）；

- **方法2**：添加系统服务（systemd）：
 1. 创建服务文件`/etc/systemd/system/myapp.service`：

```
[Unit]
Description=My Application
[Service]
ExecStart=/usr/bin/myapp
Restart=always
[Install]
WantedBy=multi-user.target
```

2. 启用服务：`systemctl enable myapp.service`;

- **方法3**：在`/etc/inittab`中添加（适合sysvinit系统）：`::respawn:/usr/bin/myapp`。

55. Makefile的基本结构？如何编写简单的Makefile？

- **基本结构**：

目标：依赖
命令（Tab开头）

- **示例**（编译`main.c`和`led.c`生成`app`）：

```
CC = arm-linux-gnueabi-hf-gcc # 交叉编译器
CFLAGS = -Wall -O2 # 编译选项（警告、优化）
LDFLAGS = -lm # 链接选项（链接数学库）
OBJS = main.o led.o # 目标文件
TARGET = app # 最终可执行文件

$(TARGET): $(OBJS)
$(CC) $(OBJS) -o $@ $(LDFLAGS)

%.o: %.c
$(CC) $(CFLAGS) -c $< -o $@

clean:
rm -f $(OBJS) $(TARGET)
```

- 执行：`make`编译，`make clean`清理。

56. Linux中GDB调试的常用命令？

- `gdb <程序>`：启动GDB；
- `run (r)`：运行程序；
- `break <行号/函数名> (b)`：设置断点；
- `next (n)`：单步执行（不进入函数）；

- `step (s)` : 单步执行 (进入函数) ;
- `continue (c)` : 继续运行到下一个断点;
- `print <变量> (p)` : 打印变量值;
- `backtrace (bt)` : 查看函数调用栈;
- `quit (q)` : 退出GDB。

57. 嵌入式Linux中如何访问GPIO?

- **方法1:** 通过sysfs接口 (用户空间) :
 1. 导出GPIO: `echo 18 > /sys/class/gpio/export;`
 2. 设置方向: `echo out > /sys/class/gpio/gpio18/direction;`
 3. 写值: `echo 1 > /sys/class/gpio/gpio18/value` (输出高电平) ;
 4. 取消导出: `echo 18 > /sys/class/gpio/unexport。`
- **方法2:** 通过设备树配置GPIO, 编写驱动程序, 提供设备文件供应用访问。

58. 嵌入式Linux中如何进行串口通信? (C语言示例)

```
#include <stdio.h>
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>

int main() {
    int fd = open("/dev/ttyS0", O_RDWR | O_NOCTTY); // 打开串口
    if (fd < 0) { perror("open"); return -1; }

    struct termios opt;
    tcgetattr(fd, &opt);
    // 配置波特率 (115200)
    cfsetispeed(&opt, B115200);
    cfsetospeed(&opt, B115200);
    // 配置数据位 (8位)、无校验、停止位 (1位)
    opt.c_cflag &= ~PARENB;
    opt.c_cflag &= ~CSTOPB;
    opt.c_cflag &= ~CSIZE;
    opt.c_cflag |= CS8;
    // 使能接收
    opt.c_cflag |= CREAD;
    tcsetattr(fd, TCSANOW, &opt);

    char buf[] = "Hello UART!";
    write(fd, buf, sizeof(buf)); // 发送数据

    char rbuf[100];
    int len = read(fd, rbuf, sizeof(rbuf)); // 接收数据
    if (len > 0) printf("Received: %.*s\n", len, rbuf);

    close(fd);
    return 0;
}
```

59. Linux中select/poll/epoll的作用？区别？

- **作用：**I/O多路复用，允许一个进程同时监控多个文件描述符（如socket、串口），当其中一个或多个就绪时再处理，提高效率。
- **区别：**
 - **select：**监控文件描述符数量有限（通常1024），每次调用需拷贝描述符集，效率低；
 - **poll：**无数量限制，但仍需拷贝描述符集，效率一般；
 - **epoll：**Linux特有，通过事件驱动，无需拷贝，支持水平触发/边缘触发，效率高（适合高并发，如服务器）。

60. 嵌入式Linux中如何实现定时器？

- **方法1：**`sleep()/usleep()`：简单但精度低，阻塞进程；
- **方法2：**`timer_create()/timer_settime()`：高精度定时器，通过信号或线程回调；
- **方法3：**`select()`：利用其超时参数实现非阻塞定时；
- **方法4：**`gettimeofday()`：手动计算时间差（如循环中检查是否到达定时时间）。

61. 什么是BusyBox？作用是什么？

- **BusyBox：**一个集成了数百个Linux常用命令（如`ls`、`cp`、`sh`）的精简工具集，编译后体积小（几百KB），适合嵌入式系统（根文件系统空间有限）。
- **作用：**提供基本的命令行工具，简化嵌入式Linux的根文件系统构建。

62. Linux中如何查看内核版本和CPU信息？

- **内核版本：**`uname -r`（如4.1.15）或`cat /proc/version`；
- **CPU信息：**`cat /proc/cpuinfo`（包含架构、核心数、主频等）。

63. 嵌入式Linux中常见的调试工具？

- **串口调试：**通过`minicom/screen`连接串口，查看启动日志和命令行输出；
- **网络调试：**`telnet/ssh`远程登录，`tftp`传输文件，`tcpdump`抓包；
- **内核调试：**`printk`输出日志（通过`dmesg`查看），`kgdb`远程调试内核；
- **应用调试：**`gdb`调试应用程序，`strace`跟踪系统调用，`ltrace`跟踪库函数调用。

64. 如何减小嵌入式Linux应用程序的体积？

- 编译时加优化选项（`-Os`：优化体积）；
- 动态链接（避免静态链接的库冗余）；
- 去除调试信息（`strip <程序名>`）；
- 使用`-ffunction-sections -fdata-sections`和`-Wl,--gc-sections`移除未使用的函数/数据；
- 选择精简库（如uClibc、musl替代glibc）。

65. 嵌入式Linux中如何挂载外部存储设备（如U盘、SD卡）？

- 查看设备文件：`fdisk -l`（如U盘识别为`/dev/sda1`，SD卡为`/dev/mmcblk0p1`）；
- 创建挂载点：`mkdir /mnt/usb`；

- 挂载: `mount /dev/sda1 /mnt/usb` (默认按文件系统类型自动识别) ;
- 卸载: `umount /mnt/usb` (需先退出挂载目录) ;
- 永久挂载: 修改`/etc/fstab`添加条目 (`/dev/sda1 /mnt/usb vfat defaults 0 0`) 。

四、Linux驱动开发 (66-100题)

66. Linux驱动程序的作用？驱动与应用程序的区别？

- **驱动作用**: 抽象硬件细节, 为应用程序提供统一的设备访问接口 (如`read()/write()`) , 是内核与硬件之间的桥梁。
- **区别**:
 - 运行空间: 驱动运行在内核空间 (ring 0) , 应用程序运行在用户空间 (ring 3) ;
 - 权限: 驱动可直接访问硬件和内核资源, 应用程序需通过系统调用;
 - 开发接口: 驱动使用内核API (如`kmalloc()`) , 应用程序使用C库 (如`malloc()`) ;
 - 崩溃影响: 驱动崩溃可能导致系统死机, 应用程序崩溃通常不影响系统。

67. Linux设备的分类？各自特点？

- **字符设备**: 按字节流顺序访问, 无缓冲 (如串口、GPIO) , 设备文件通常在`/dev`下 (如`/dev/ttyS0`) ;
- **块设备**: 按块 (如512B、4KB) 访问, 有缓冲 (如硬盘、SD卡) , 设备文件如`/dev/sda`;
- **网络设备**: 用于网络通信, 无传统设备文件, 通过套接字访问 (如`eth0`) 。

68. 什么是设备文件？主设备号和次设备号的作用？

- **设备文件**: 用户空间访问设备的接口, 存在于`/dev`目录, 不占用磁盘空间, 仅用于标识设备。
- **主设备号**: 标识设备类型 (如串口主设备号4) , 对应驱动程序;
- **次设备号**: 标识同一类型中的具体设备 (如`/dev/ttyS0`次设备号0, `/dev/ttyS1`次设备号1) 。
- 查看设备号: `ls -l /dev/ttyS0` (输出如`crw-rw---- 1 root dialout 4, 64 ...`, 4为主设备号, 64为次设备号) 。

69. Linux驱动的加载和卸载方式？

- **加载**:
 - `insmod`: 加载指定.ko文件 (`insmod mydriver.ko`) ;
 - `modprobe`: 自动解决依赖关系加载模块 (需模块在`/lib/modules/$(uname -r)`目录) 。
- **卸载**:
 - `rmmmod`: 卸载模块 (`rmmmod mydriver`, 无需.ko后缀) ;
 - `modprobe -r`: 卸载模块及依赖 (`modprobe -r mydriver`) 。
- 查看已加载模块: `lsmod`。

70. 字符设备驱动的核心数据结构？ (`file_operations`、`cdev`)

- **file_operations**: 定义驱动支持的操作函数 (接口) , 如:

```
struct file_operations {
    ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char __user *, size_t, loff_t *);
```

```
int (*open)(struct inode *, struct file *);
int (*release)(struct inode *, struct file *);
// 其他操作 (ioctl、mmap等)
};
```

- **cdev**: 字符设备结构体, 关联设备与file_operations:

```
struct cdev {
    struct kobject kobj;
    struct module *owner; // 模块所有者 (THIS_MODULE)
    const struct file_operations *ops; // 操作函数集
    dev_t dev; // 设备号
    unsigned int count; // 设备数量
};
```

71. 字符设备驱动的注册流程?

1. 分配设备号 (alloc_chrdev_region()或静态指定register_chrdev_region());
2. 初始化cdev结构体 (cdev_init(), 绑定file_operations);
3. 添加cdev到内核 (cdev_add());
4. 创建设备文件 (通过class_create()和device_create()自动创建设备节点)。

- 示例代码片段:

```
dev_t dev_num;
struct cdev my_cdev;
struct class *my_class;

// 1. 分配设备号
alloc_chrdev_region(&dev_num, 0, 1, "mydev"); // 次设备号0, 1个设备
// 2. 初始化cdev
cdev_init(&my_cdev, &fops);
my_cdev.owner = THIS_MODULE;
// 3. 添加cdev
cdev_add(&my_cdev, dev_num, 1);
// 4. 创建设备文件
my_class = class_create(THIS_MODULE, "myclass");
device_create(my_class, NULL, dev_num, NULL, "mydev"); // /dev/mydev
```

72. 什么是设备模型 (Device Model) ? 核心数据结构?

- **设备模型**: Linux内核中统一管理设备、驱动、总线的框架, 提供热插拔、电源管理、sysfs接口等功能。
- **核心数据结构**:
 - **kobject**: 基础对象, 提供引用计数、sysfs接口;
 - **kset**: kobject的集合, 用于组织对象;
 - **bus_type**: 总线类型 (如platform_bus_type), 负责设备与驱动的匹配;

- `device`: 表示一个硬件设备;
- `device_driver`: 表示一个驱动程序。

73. Platform总线驱动的特点？与传统总线（I2C/SPI）的区别？

- **Platform总线**: 虚拟总线，用于连接无物理总线的片上外设（如GPIO、定时器），基于设备树或平台设备注册。
- **特点**: 设备与驱动通过名字匹配（`device`的`name`与`driver`的`name`一致）。
- **与传统总线区别**:
 - 传统总线（I2C/SPI）: 有物理信号线，设备挂在实际总线上，驱动需遵循总线协议;
 - Platform总线: 无物理总线，仅为内核管理片上外设提供统一框架。

74. Platform驱动的流程？

1. 定义`platform_driver`结构体（包含`probe`、`remove`、`driver`等成员）;
 2. 实现`probe`函数（设备与驱动匹配时调用，初始化硬件、注册字符设备）;
 3. 实现`remove`函数（驱动卸载时调用，释放资源）;
 4. 注册驱动（`platform_driver_register()`）。
- 示例:

```
static int my_platform_probe(struct platform_device *pdev) {
    // 初始化硬件、注册字符设备等
    return 0;
}

static int my_platform_remove(struct platform_device *pdev) {
    // 释放资源
    return 0;
}

static struct platform_driver my_platform_driver = {
    .probe = my_platform_probe,
    .remove = my_platform_remove,
    .driver = {
        .name = "mydevice", // 与设备树中compatible匹配或平台设备name匹配
        .owner = THIS_MODULE,
    },
};

module_platform_driver(my_platform_driver);
```

75. 设备树（Device Tree）的作用？基本语法？

- **作用**: 描述硬件信息（如CPU、内存、外设地址、中断），使内核与硬件解耦（无需修改内核代码，仅修改设备树即可支持新硬件）。
- **基本语法**:
 - 节点: `node-name { ... };` (如`uart0: uart@12340000 { ... };`);

- 属性: `property = value`; (如`compatible = "vendor,uart"`; , 用于驱动匹配) ;
- 地址: `reg = <address size>`; (外设寄存器地址和大小) ;
- 中断: `interrupts = <irq_num irq_type>`;
- 示例片段:

```
uart0: serial@40002000 {
    compatible = "st,stm32-uart";
    reg = <0x40002000 0x400>; // 地址0x40002000, 大小0x400
    interrupts = <31 0>; // 中断号31, 触发类型0
    clocks = <&clk 0>;
};
```

76. 驱动中如何从设备树获取硬件信息?

- 使用内核提供的API从`platform_device`中获取:
 - 获取地址资源: `platform_get_resource(pdev, IORESOURCE_MEM, 0)`;
 - 获取中断资源: `platform_get_irq(pdev, 0)`;
 - 获取属性值: `of_property_read_string()`、`of_property_read_u32()`等。
- 示例:

```
static int my_probe(struct platform_device *pdev) {
    struct resource *res;
    // 获取寄存器地址资源
    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
    void __iomem *base = devm_ioremap_resource(&pdev->dev, res); // 映射到虚拟地址

    // 获取中断号
    int irq = platform_get_irq(pdev, 0);

    // 获取设备树属性
    const char *str;
    of_property_read_string(pdev->dev.of_node, "vendor,desc", &str);
    return 0;
}
```

77. Linux内核中的内存管理函数? (`kmalloc`、`vmalloc`等)

- `kmalloc(size, GFP_KERNEL)`: 分配连续物理内存, 用于内核短期使用, 速度快, 大小有限制 (通常 $\leq 128\text{KB}$) ;
- `vmalloc(size)`: 分配虚拟地址连续但物理地址可能不连续的内存, 用于大内存分配 (如几MB) , 速度较慢;
- `devm_kmalloc(dev, size, GFP_KERNEL)`: 设备管理的内存分配, 随设备释放自动释放, 避免内存泄漏;
- `kfree(ptr)`: 释放`kmalloc`分配的内存;
- `vfree(ptr)`: 释放`vmalloc`分配的内存。

78. 内核空间与用户空间的数据传输函数？为什么不能直接访问？

- **原因：**内核空间与用户空间有严格隔离，直接访问用户空间指针可能导致页错误（用户空间内存可能被换出）。
- **传输函数：**
 - 用户空间→内核空间：`copy_from_user(dst, src, n);`
 - 内核空间→用户空间：`copy_to_user(dst, src, n);`
 - 成功返回0，失败返回未复制的字节数。
- **示例：**

```
ssize_t my_write(struct file *filp, const char __user *buf, size_t len,
loff_t *off) {
    char *kernel_buf = kmalloc(len, GFP_KERNEL);
    if (copy_from_user(kernel_buf, buf, len)) { // 从用户空间复制数据
        kfree(kernel_buf);
        return -EFAULT;
    }
    // 处理数据...
    kfree(kernel_buf);
    return len;
}
```

79. Linux中断处理的顶半部（Top Half）和底半部（Bottom Half）的区别？

- **顶半部：**中断服务程序（ISR），响应中断后立即执行，处理紧急事务（如清除中断标志、读取状态），需快速完成（禁止中断嵌套）；
- **底半部：**处理耗时操作（如数据处理），在顶半部之后执行，允许中断嵌套。
- **底半部实现方式：**
 - 软中断（`softirq`）：内核级，适合高频率事件（如网络接收）；
 - tasklet：基于软中断，简化接口，单CPU上串行执行；
 - 工作队列（`workqueue`）：在进程上下文执行，可睡眠（适合需要调用`kmalloc(GFP_KERNEL)`等可能睡眠的操作）。

80. 如何在驱动中注册和处理中断？

1. 申请中断：`request_irq(irq, isr, flags, name, dev);`
 - `irq`：中断号；
 - `isr`：中断服务程序（顶半部）；
 - `flags`：中断标志（如`IRQF_TRIGGER_RISING`上升沿触发）；
 - `name`：中断名称（`/proc/interrupts`中可见）；
 - `dev`：传递给ISR的参数，用于共享中断。
2. 实现ISR（顶半部）：

```
static irqreturn_t my_isr(int irq, void *dev_id) {
    // 紧急处理（如清除中断）
    schedule_work(&my_work); // 调度底半部（工作队列）
}
```

```
    return IRQ_HANDLED;
}
```

3. 实现底半部（如工作队列）：

```
static struct work_struct my_work;
static void my_work_func(struct work_struct *work) {
    // 耗时处理（如数据处理）
}
```

4. 释放中断：`free_irq(irq, dev)`。

81. 什么是内核模块（Kernel Module）？模块的入口和出口函数？

- **内核模块**：可动态加载到内核的代码，扩展内核功能（如驱动程序），无需重新编译内核。
- **入口函数**：模块加载时调用（`module_init(init_func)`），用于初始化（如注册设备、申请资源）；
- **出口函数**：模块卸载时调用（`module_exit(exit_func)`），用于清理（如注销设备、释放资源）。
- 示例：

```
#include <linux/module.h>

static int __init mymodule_init(void) {
    printk(KERN_INFO "Module loaded\n");
    return 0;
}

static void __exit mymodule_exit(void) {
    printk(KERN_INFO "Module unloaded\n");
}

module_init(mymodule_init);
module_exit(mymodule_exit);
MODULE_LICENSE("GPL"); // 必须声明许可证（如GPL）
```

82. 内核中的同步机制有哪些？（自旋锁、互斥锁等）

- **自旋锁（`spinlock_t`）**：忙等待（不睡眠），适合短时间持有，中断上下文中使用（`spin_lock_irqsave()`）；
- **互斥锁（`struct mutex`）**：获取不到锁时睡眠，适合长时间持有，仅能在进程上下文使用；
- **信号量（`struct semaphore`）**：类似互斥锁，支持多个持有者（计数>0）；
- **读写锁（`rwlock_t`）**：读共享、写独占，适合读多写少场景；
- **原子变量（`atomic_t`）**：用于简单计数（如`atomic_inc()`、`atomic_read()`），避免锁开销。

83. 什么是内核定时器（`timer_list`）？使用步骤？

- **内核定时器**：基于系统时钟，在指定时间后执行回调函数（一次性或周期性）。

- **使用步骤:**

1. 定义定时器: `struct timer_list my_timer;;`
2. 初始化定时器: `init_timer(&my_timer);;`
3. 设置回调函数和参数:

```
my_timer.function = my_timer_func;
my_timer.data = (unsigned long)dev; // 传递参数
```

4. 设置超时时间 (jiffies为内核节拍, 1jiffy≈10ms@HZ=100) :

```
my_timer.expires = jiffies + HZ; // 1秒后执行
```

5. 启动定时器: `add_timer(&my_timer);;`
6. (可选) 修改超时时间: `mod_timer(&my_timer, jiffies + 2*HZ);;`
7. 回调函数实现:

```
static void my_timer_func(unsigned long data) {
    struct my_device *dev = (struct my_device *)data;
    // 定时任务...
    mod_timer(&dev->my_timer, jiffies + HZ); // 周期性执行
}
```

84. I2C驱动的框架? 核心数据结构?

- **I2C驱动框架:** 分为总线层、设备层、驱动层。
- **核心数据结构:**
 - `i2c_adapter`: 表示I2C控制器 (总线适配器) ;
 - `i2c_client`: 表示I2C从设备;
 - `i2c_driver`: 表示I2C设备驱动, 包含`probe`、`remove`、`id_table` (设备ID匹配) 等。
- **驱动注册步骤:**
 1. 定义`i2c_driver`结构体;
 2. 实现`probe`和`remove`函数;
 3. 注册驱动: `i2c_add_driver()`。
- 数据传输函数: `i2c_master_send()` (发送)、`i2c_master_recv()` (接收)、`i2c_transfer()` (复杂传输) 。

85. SPI驱动的框架? 与I2C驱动的区别?

- **SPI驱动框架:** 类似I2C, 核心结构包括`spi_master` (SPI控制器)、`spi_device` (SPI从设备)、`spi_driver` (SPI驱动) 。
- **与I2C驱动的区别:**
 - 通信方式: SPI是全双工, I2C是半双工;
 - 寻址方式: SPI通过片选 (CS) 线选设备, I2C通过地址;

- 传输速率：SPI速率更高（可达几十Mbps），I2C通常较低（≤1Mbps）；
- 驱动接口：SPI用`spi_write()/spi_read()`，I2C用`i2c_master_send()/i2c_master_recv()`。

86. 什么是设备文件系统（sysfs）？作用是什么？

- **sysfs**：基于内存的虚拟文件系统（挂载在`/sys`），以文件和目录形式展示内核设备模型（设备、驱动、总线）。
- **作用**：
 - 提供用户空间与内核的交互接口（如查看设备属性、修改配置）；
 - 展示设备拓扑结构（如`/sys/devices`下的设备树）；
 - 支持热插拔事件通知。

87. 驱动中如何创建sysfs接口？

- 通过`sysfs_create_file()`创建文件，结合`struct device_attribute`定义属性操作：

```
// 定义属性读写函数
static ssize_t my_attr_show(struct device *dev, struct device_attribute
*attr, char *buf) {
    return sprintf(buf, "%d\n", my_value); // 读取值
}

static ssize_t my_attr_store(struct device *dev, struct device_attribute
*attr, const char *buf, size_t count) {
    kstrtoint(buf, 10, &my_value); // 写入值
    return count;
}

// 定义设备属性
static DEVICE_ATTR(my_attr, 0664, my_attr_show, my_attr_store);

// 在probe函数中创建sysfs文件
device_create_file(&pdev->dev, &dev_attr_my_attr); // 创
建/sys/devices/.../my_attr
```

88. 什么是ioctl？如何在驱动中实现ioctl？

- **ioctl**：设备控制接口，用于实现read/write无法满足的复杂操作（如配置设备参数、发送命令）。
- **驱动中实现步骤**：
 1. 在`file_operations`中定义`unlocked_ioctl`函数：

```
long my_ioctl(struct file *filp, unsigned int cmd, unsigned long arg) {
    switch (cmd) {
        case MY_CMD_SET:
            // 处理命令（如从用户空间获取参数）
            if (copy_from_user(&value, (int __user *)arg, sizeof(int)))
                return -EFAULT;
            break;
    }
```

```
        case MY_CMD_GET:
            // 向用户空间返回数据
            if (copy_to_user((__user *)arg, &value, sizeof(int)))
                return -EFAULT;
            break;
        default:
            return -ENOTTY; // 不支持的命令
    }
    return 0;
}
```

2. 应用程序调用: `ioctl(fd, MY_CMD_SET, ¶m);`。

89. 内核调试的常用方法?

- **printk**: 内核打印函数, 优先级通过 `KERN_*` 指定 (如 `KERN_INFO`、`KERN_ERR`), 输出到 `dmesg`;
- **dump_stack()**: 打印函数调用栈, 用于定位死锁或异常;
- **ftrace**: 跟踪函数调用、中断等, 分析内核行为;
- **kgdb**: 远程调试内核, 通过串口或网络连接 GDB;
- **kprobe**: 动态调试工具, 在指定函数插入钩子, 无需重新编译内核。

90. 什么是Oops? 如何分析Oops信息?

- **Oops**: 内核发生严重错误 (如空指针访问、非法指令) 时的报错信息, 包含错误原因、寄存器状态、函数调用栈。
- **分析方法**:
 1. 查看Oops中的 **PC** (程序计数器) 值, 定位出错指令;
 2. 使用 `addr2line` 工具将地址转换为代码行 (`addr2line -e vmlinux pc_addr`);
 3. 结合调用栈 (**Backtrace**) 分析函数调用路径;
 4. 检查相关代码 (如指针未初始化、数组越界)。

91. 驱动开发中常见的错误码? (如 **-EIO**、**-ENOMEM**)

- **-EIO**: I/O 错误 (如硬件通信失败);
- **-ENOMEM**: 内存分配失败;
- **-EBUSY**: 设备忙 (如已被占用);
- **-EINVAL**: 无效参数;
- **-ENODEV**: 无此设备;
- **-EPERM**: 权限不足;
- **-ETIMEDOUT**: 超时。

92. 什么是内核线程? 如何创建内核线程?

- **内核线程**: 运行在内核空间的线程, 无用户空间上下文, 用于执行内核任务 (如刷新缓存、I/O 处理)。
- **创建方法**: `kthread_create()` 创建, `wake_up_process()` 启动:

```
#include <linux/kthread.h>
```

```
static int my_kthread_func(void *data) {
    while (!kthread_should_stop()) { // 检查是否需要退出
        // 线程任务 (如定时处理)
        msleep(1000);
    }
    return 0;
}

// 在驱动初始化中创建
struct task_struct *kthread;
kthread = kthread_create(my_kthread_func, NULL, "mykthread");
if (IS_ERR(kthread)) return PTR_ERR(kthread);
wake_up_process(kthread); // 启动线程

// 在驱动退出时停止
kthread_stop(kthread);
```

93. 什么是DMA？驱动中如何使用DMA传输数据？

- **DMA（直接存储器访问）**：无需CPU干预，外设直接与内存传输数据，提高效率。
- **驱动中使用步骤**：
 1. 申请DMA通道（如`dma_request_channel()`）；
 2. 分配DMA缓冲区（`dma_alloc_coherent()`，物理地址连续）；
 3. 配置DMA控制器（源地址、目的地址、传输长度、方向）；
 4. 启动DMA传输；
 5. 传输完成后（通过中断或轮询）处理数据；
 6. 释放DMA通道和缓冲区。

94. 什么是设备树叠加（Device Tree Overlay）？作用是什么？

- **设备树叠加**：动态加载的设备树片段，用于在不修改主设备树的情况下添加或修改硬件描述。
- **作用**：支持硬件可配置的系统（如开发板扩展模块），无需重新编译主设备树，只需加载对应overlay。

95. 如何处理驱动中的并发访问？

- **方法**：
 - 使用自旋锁或互斥锁保护共享资源（如全局变量、硬件寄存器）；
 - 对中断上下文和进程上下文共享的数据，使用`spin_lock_irqsave()`禁止中断；
 - 使用原子操作处理简单计数（如`atomic_t`）；
 - 避免在中断上下文中使用可能睡眠的操作（如`kmalloc(GFP_KERNEL)`）。

96. 什么是Kconfig和Makefile在驱动开发中的作用？

- **Kconfig**：用于配置内核选项，定义驱动的编译选项（如`config MY_DRIVER tristate "My Driver"`），通过`make menuconfig`选择是否编译为模块或内置；
- **Makefile**：指定驱动的编译规则（如`obj-$(CONFIG_MY_DRIVER) += mydriver.o`），当`CONFIG_MY_DRIVER`为`m`时编译为模块，为`y`时内置到内核。

97. 驱动中如何处理设备的热插拔？

- 依赖内核设备模型和udev（用户空间守护进程）：
 1. 驱动在设备插入时通过`device_add()`注册设备；
 2. 内核发送热插拔事件（通过netlink）；
 3. udev接收事件，根据`/etc/udev/rules.d`中的规则创建设备文件；
 4. 设备拔出时，驱动调用`device_del()`，udev删除设备文件。

98. 什么是中断共享？如何实现中断共享？

- **中断共享**：多个设备共享同一中断线（节省中断资源）。
- **实现条件**：
 - 中断控制器支持共享；
 - 驱动的中断服务程序能区分中断来源（如通过设备状态寄存器）。
- **注册方法**：`request_irq()`时指定`IRQF_SHARED`标志，且`dev`参数唯一：

```
request_irq(irq, my_isr, IRQF_SHARED | IRQF_TRIGGER_RISING, "mydev", dev);
```

- ISR中需检查是否为当前设备的中断，非本设备中断返回`IRQ_NONE`，否则返回`IRQ_HANDLED`。

99. 什么是内核版本兼容性？如何保证驱动在不同内核版本中兼容？

- **内核版本兼容性**：驱动程序在不同Linux内核版本（如4.1、5.4）上均可正常工作。
- **保证方法**：
 - 使用内核提供的兼容API（如`devm_`系列函数减少资源管理差异）；
 - 避免使用内核内部未公开的函数（可能随版本变化）；
 - 处理版本差异（如通过`#ifdef`判断内核版本，使用不同实现）：

```
#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 4, 0)
    // 5.4及以上版本的实现
#else
    // 旧版本的实现
#endif
```

- 参考内核文档中的`Documentation/ABI`了解接口稳定性。

100. 嵌入式Linux驱动开发的流程？

1. **硬件分析**：了解外设手册（寄存器地址、时序、中断等）；
2. **环境搭建**：安装交叉编译工具链、内核源码、调试工具；
3. **驱动框架选择**：根据设备类型（字符设备、I2C、SPI等）选择合适框架；
4. **编写驱动代码**：实现初始化、设备操作、中断处理等核心功能；
5. **编写Kconfig和Makefile**：配置编译选项；
6. **编译测试**：编译为模块，加载到目标板测试；
7. **调试优化**：通过`printk`、GDB等调试，优化性能和稳定性；
8. **集成到系统**：将驱动加入内核配置，随系统一起编译。